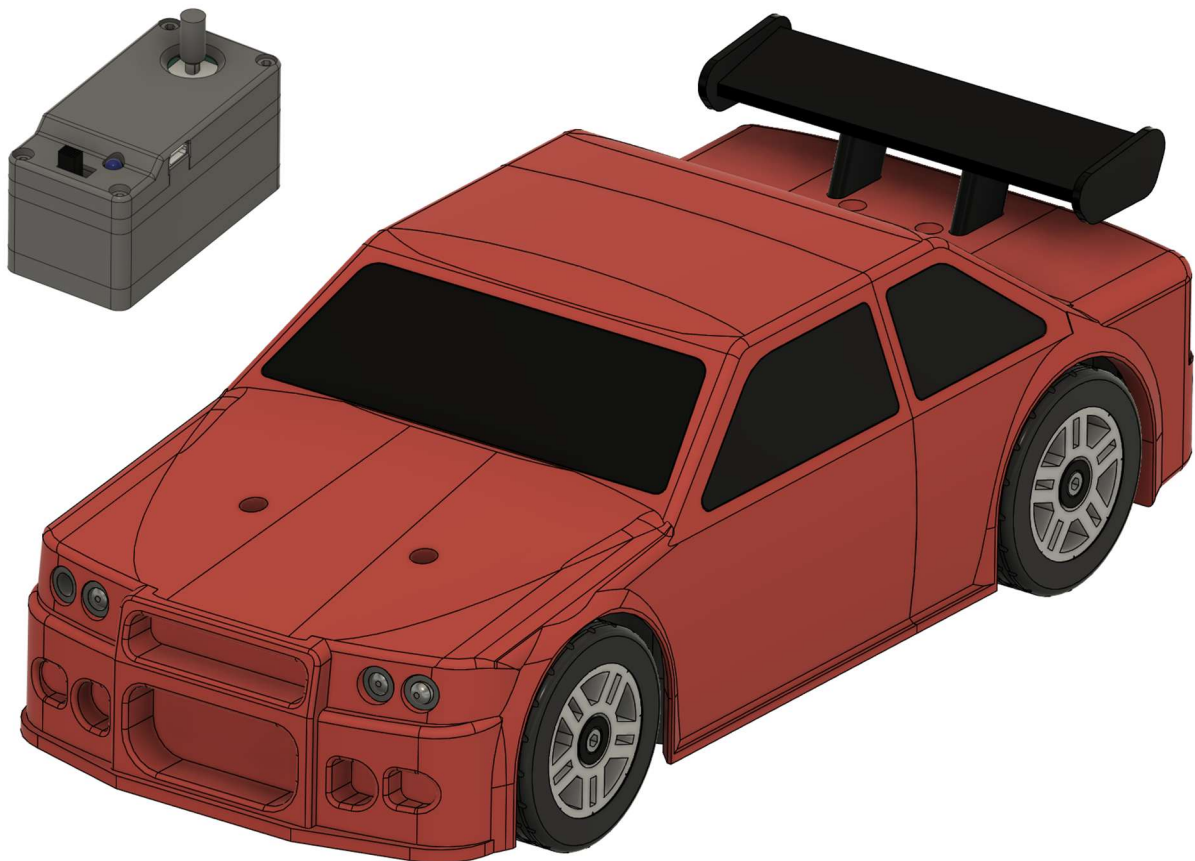


EasyRC – Open Source Fernsteuerungsplattform



Betriebsanleitung

Inhaltsverzeichnis

1.	Hinweise und Spezifikationen	4
1.1	Sicherheitshinweise	4
1.2	Spezifikationen der EasyRC-Plattform	5
2.	Teilelisten	6
2.1	Elektronikbauteile	6
2.2	Metallbauteile	8
2.3	Gedruckte Bauteile	9
3.	Elektronik	16
3.1	Einführung in die Elektronik	16
3.2	Fertigung und Bestückung der Leiterplatten	17
3.3	Crimpen und Verlöten der Elektronikbauteile	18
3.4	Komponentenliste	19
3.5	Übersicht Fernsteuerungselektronik	22
3.6	Übersicht Fahrzeugelektronik	25
4.	Bauanleitung Fernsteuerung und Fahrzeug	31
4.1	Fernbedienung	35
4.2	Antriebsstrang und Hinterräder	41
4.3	Vorderräder und Lenkung	51
4.4	Finalisierung Chassis	58
4.5	Karosserie	65
5.	Programmierung	73
5.1	Installation Arduino IDE	73
5.2	Mögliche und benötigte Code-Modifikationen	75
5.3	Programmieren der Microcontroller	77
6.	Code der Microcontroller	79
6.1	Fernsteuerung	79
6.2	Fahrzeug	80
7.	Erste Schritte und Bedienung des Fahrzeugs	82
7.1	Batterien einsetzen und Geräte anschalten	82
7.2	Lenkung fertig montieren	83
7.3	Fahrzeug fertig montieren	85
7.4	Steuerungsprinzip und Funktionen des Fahrzeugs	86
7.5	Trimmung der Lenkung	87
7.6	Feintuning der Fahreigenschaften	88

8.	Mögliche Fehler und Lösungsansätze	89
9.	Anhang	90
9.1	Änderung der Geräte-Adresse der EasyRC-Plattform	90
9.2	Änderung des Funkkanals der EasyRC-Plattform	91
10.	Noch umzusetzende Verbesserungen	92
11.	Impressum	93

1. Hinweise und Spezifikationen

1.1 Sicherheitshinweise

EasyRC ist eine open-source-Plattform für drahtlos ferngesteuerte Modellfahrzeuge, die als Bausatz aus 3D-gedruckten Bauteilen montiert werden. Das Benutzerhandbuch muss vor dem Zusammenbau und der Inbetriebnahme vollständig gelesen und verstanden werden, um Gefahrenquellen zu erkennen und Verletzungen oder Schäden zu vermeiden.

Für ferngesteuerte Modellfahrzeuge gilt grundsätzlich, dass diese nicht im öffentlichen Straßenverkehr bewegt werden dürfen. Bei ferngesteuerten Modellen besteht die Gefahr, dass der Steuernde die Kontrolle verliert, wenn die Drahtlosverbindung durch Überschreiten der maximalen Übertragungsreichweite oder Interferenzen gestört wird. EasyRC beinhaltet dafür eine Sicherheitsroutine, die beim Ausbleiben von Funksignalen der Fernsteuerung für wenigstens eine Sekunde die Motoren deaktiviert (die Fernsteuerung überträgt pro Sekunde 20 Datenpakete an das Modellfahrzeug, sodass diese Sicherung nicht durch einzelne verlorene Datenpakete ausgelöst werden kann). Diese Sicherheitsroutine bleibt jedoch funktionslos bei Störsignalen, die vom Modellfahrzeug fälschlicherweise als Signal der Fernsteuerung interpretiert werden. Dafür prüft das Modellfahrzeug die individuelle Geräte-Adresse (MAC) der Fernsteuerung, die mit jedem Datenpaket gesendet wird. Nur wenn diese mit der intern gespeicherten Geräte-Adresse der gekoppelten Fernsteuerung übereinstimmt, wird das Datenpaket weiterverarbeitet. Deshalb ist es beim gleichzeitigen Betrieb mehrerer EasyRC-Modelle wichtig, dass jedes mit einer anderen Geräte-Adresse programmiert wird, sodass jede Fernsteuerung spezifisch nur ein Fahrzeug steuert. Die Drahtlosverbindung basiert auf einer Funkfrequenz von 2,4 GHz, in der auch herkömmliches WLAN arbeitet. Daher besteht grundsätzlich in Häusern mit vielen verschiedenen Drahtlosnetzwerken ein erhöhtes Risiko für Verbindungsabbrüche. Sollte es zu häufigen Aussetzern kommen, kann der Sendekanal der EasyRC-Plattform geändert werden.

Die EasyRC-Plattform nutzt Batterien mit Systemspannungen von maximal ca. 12 V Gleichspannung. Diese Spannung ist ungefährlich für den Menschen; es kann zu keinen gefährlichen Stromschlägen kommen. Zudem ist die Platine des Fahrzeugs über eine Sicherung vor zu großem Stromfluss geschützt. Jedoch können Batterien bei unsachgemäßem Gebrauch (z.B. durch Überladen, Kurzschluss, mechanische Beschädigung) beschädigt werden, was zur Entzündung und zum Freisetzen gesundheitsschädlicher Inhaltsstoffe führen kann. Die im Modellsystem verwendeten Lithium-Ionen-Batterien dürfen deshalb nur mit dafür geeigneten Ladegeräten aufgeladen werden. Die maximale Spannung (Ladeschlussspannung) einer einzelnen Zelle beträgt 4.2 V. Diese Spannung darf nicht überschritten werden! Die Batterien dürfen nicht weiterverwendet werden, wenn sie mechanisch beschädigt sind oder überladen worden sind. Ein Kurzschluss muss unbedingt vermieden werden. Auch eine Tiefentladung (<2.5 V) kann die Zellen irreparabel beschädigen. Die Platinen der Fernsteuerung und des Fahrzeugs sind verpolungsgeschützt. Da das Fahrzeug mit drei in Serie geschalteten Batteriezellen betrieben wird, muss beim Einsetzen der Batterien dennoch unbedingt auf die richtige Ausrichtung (Polarität) geachtet werden, um Schäden an den Batterien zu vermeiden. Batterien und Elektronikkomponenten dürfen nicht im Restmüll entsorgt werden, sondern müssen entsprechend der örtlichen Möglichkeiten umweltgerecht entsorgt bzw. recycelt werden.

1.2 Spezifikationen der EasyRC-Plattform

1.2.1 Fernsteuerung

Abmessungen: 80x40x41 mm³ (LxBxH)

Gewicht: ca. 125 g (inklusive Batterie)

Microcontroller: ESP32-C3

Drahtloskommunikation: 2,4 GHz (ESP-NOW Protokoll)

Joystick: Alps 10k (Playstation 4 Modell)

Stromversorgung: 1x 18650 Li-Ion Batterie (3,7 V)

Energieverbrauch: ca. 0,5 W

Hinweise:

- Batterieanschluss verpolungsgeschützt
- Warnung bei niedriger Batteriespannung (<3,4 V): LED beginnt zu blinken
- Selbstkalibrierender Joystick

1.2.2 Fahrzeug

Abmessungen: 282x125x95 mm³ (LxBxH)

Maßstab: 1:16

Gewicht: ca. 1 kg (inklusive Batterien)

Microcontroller: ESP32-C3

Drahtloskommunikation: 2,4 GHz (ESP-NOW Protokoll)

Motortreiber: Pololu DRV8876 (QFN)

Antriebsmotor: JGA25-370 Bürstenmotor mit Getriebe (12 V, 1000 U/min)

Geschwindigkeit: max. 10 km/h

Lenkungsservo: MG90S Microservo mit Metallgetriebe

Stromversorgung: 3x 18650 Li-Ion Batterie (11.1 V)

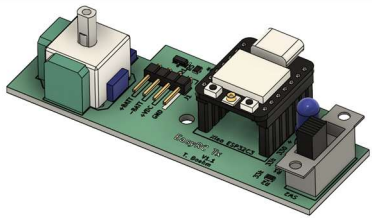
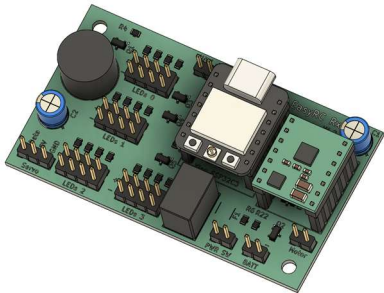
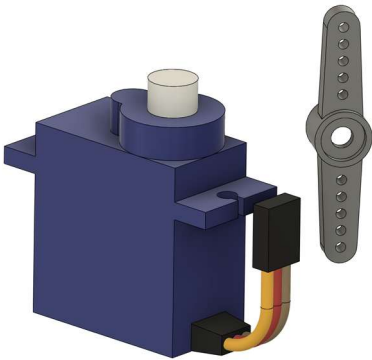
Stromverbrauch: ca. 0,8 W (Leerlauf); ca. 3-6 W (Fahrbetrieb)

Hinweise:

- Batterieanschluss verpolungsgeschützt
- Warnung bei niedriger Batteriespannung (<10,2 V): Motor deaktiviert, Buzzer piept
- Warnung bei fehlender Verbindung zur Fernsteuerung (>1 s): Motor deaktiviert, Buzzer piept
- Warnung bei ausbleibenden Nutzereingaben (kein Input für >1 min): Buzzer piept
- Selbstrückstellende Sicherung bei zu großem Stromfluss (>1.5 A)

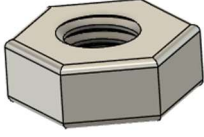
2. Teilelisten

2.1 Elektronikbauteile

Abbildung	Bezeichnung	Beschreibung
	1x EasyRC Tx Platine	Leiterplatte mit Komponenten für die Fernsteuerung. Beinhaltet den Joystick sowie den Microcontroller, der die Drahtlosverbindung herstellt. Komponentenliste in Kapitel 3.4.
	1x EasyRC Rx Platine	Leiterplatte mit Komponenten für das Auto. Beinhaltet den Microcontroller für die Fahrzeugsteuerung und den Motortreiber für den Antriebsmotor. Komponentenliste in Kapitel 3.4.
	1x RC-Servomotor, Typ MG90S, mit Servohorn Farbkodierung des Anschlusskabels: Gelb: Datenkanal Rot: +5 V Braun: GND	Motor mit integriertem Getriebe und Elektronik, der den Winkel der Motorachse in einem Winkelumfang von ca. 180° entsprechend seinem Eingangssignal ausrichtet. Das Servohorn dient als Verbindung zum bewegten Element und wird mit der Motorachse verschraubt. Wichtig: Auf richtige Anschlussrichtung achten. Verpolung zerstört den Servomotor!
	1x DC-Getriebemotor JGA25-370 (1000 U/min) Das Anschlusskabel kann farbkodiert werden: Rot: +12 V Schwarz: GND	Motor mit integriertem Getriebe mit 1:10 Übersetzung. Die Drehzahl bei 12 V beträgt 1.000 U/min. Zur stabilen Kraftübertragung ist die Motorwelle in D-Form ausgeführt. Die Drehrichtung des Motors hängt von der Anschlussorientierung ab.

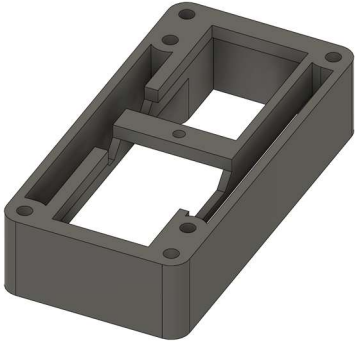
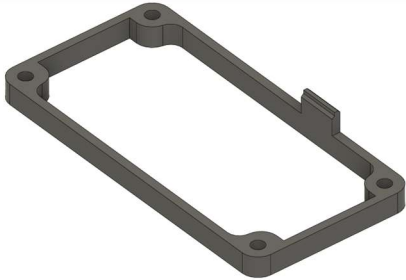
	8x bedrahtete 5-mm-LED Das Anschlusskabel sollte farbkodiert werden: Rot: +5 V* (langer Pin) Schwarz: GND (kurzer Pin) *Gilt nur für EasyRC: Die benötigten Vorwiderstände für die LEDs sind auf der EasyRC-Platine integriert.	Leuchtdiode in verschiedenen Farben (2x Weiß, 4x Gelb und 2x Rot). Typ: Quadrios 2111O182 (weiß) Quadrios 2111O171 (gelb) Quadrios 2111O172 (rot) Wichtig: Beim Verlöten mit Litzen ausreichend lange Kabel (30-40 cm) verwenden und auf die Anschlussrichtung achten – die LED leuchtet nicht, wenn sie falsch angeschlossen ist und kann dadurch beschädigt werden!
	4x 18650 Li Ion Batterie Spannungsfenster: 4.2 V (vollgeladen) 3.3 V (leer)	Typ: Lishen LR 1865SS 3.1 Ah Lithium-Ionen-Batterie für den Betrieb der EasyRC-Plattform. Die Batterien müssen über ein separat erhältliches Ladegerät aufgeladen werden. EasyRC prüft die Batteriespannung und warnt den Nutzer, wenn die Batterieladung zur Neige geht.
	Batteriehalter für Einzelzellen Benötigte Größen: 1x 3 18650 Zellen 1x 1 18650 Zelle	Typ: mit Kabeln ausgestattete Halter Wichtig: Vor dem Einsetzen der Batterien immer auf die richtige Polarität achten! Die Batterien und die angeschlossenen Komponenten können bei Verpolung beschädigt werden.
	1x An/Aus-Schalter	Typ: Marquardt 01801.1148-02 Schaltet das Fahrzeugmodell an und ab.

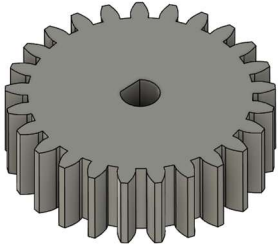
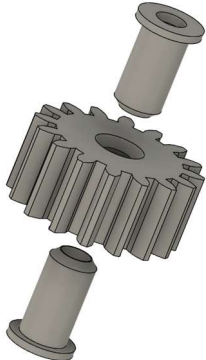
2.2 Metallbauteile

Abbildung	Bezeichnung	Beschreibung
	Zylinderkopfschraube M3 Benötigte Größen: 29x M3x8 mm 4x M3x12 mm 7x M3x16 mm 6x M3x20 mm 2x M3x22 mm 17x M3x30 mm 2x M3x35 mm 2x M3x40 mm	Schraube mit metrischem Gewinde mit 3 mm Durchmesser. Der Schraubenkopf wird mit einem 2,5 mm Innensechskantschlüssel angetrieben.
	64x Sechskantmutter M3	Mutter für Schrauben mit metrischem 3-mm-Gewinde.
	Senkkopfschraube M3 Benötigte Größe: 3x M3x10 mm	Schraube mit metrischem Gewinde mit 3 mm Durchmesser. Der Schraubenkopf wird mit einem 2,0 mm Innensechskantschlüssel angetrieben.
	Schraube M2.5 Benötigte Größe: 1x M2.5x6 mm	Schraube mit metrischem Gewinde mit 2,5 mm Durchmesser. Wird für die Befestigung des Servohorns benötigt.
	Rillenkugellager Benötigte Typen: 4x 605 (5x14x5 mm) 4x 608 (8x22x7 mm) 2x 6802 (15x24x5 mm)	Die Größenbezeichnungen der Lager geben Innendurchmesser x Außendurchmesser x Höhe an.

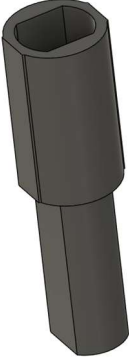
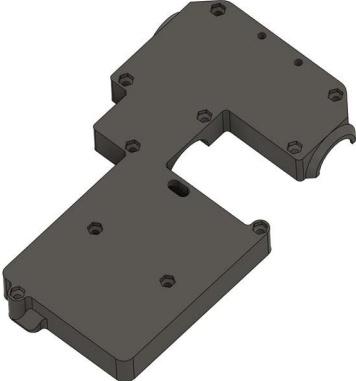
2.3 Gedruckte Bauteile

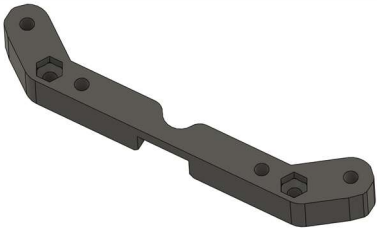
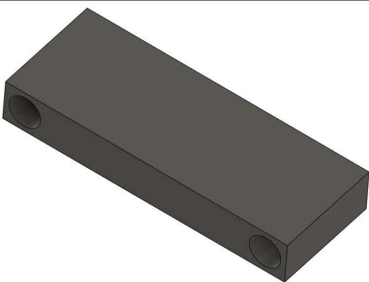
Richtungsangaben bei den Drucktipps beziehen sich immer auf die Ausrichtung des Bauteils im gesamten Modell, nicht auf die Ausrichtung in den Bildern in der linken Spalte!

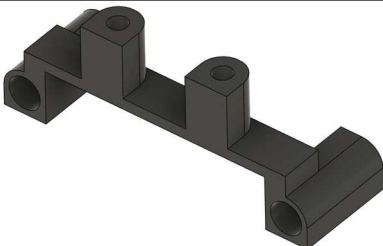
Abbildung	Bezeichnung, Kapitel	Drucktipps
	1x Fernbedienung, unterer Zwischenrahmen Kapitel 4.1.1	Oberseite auf Druckbett; Support empfohlen.
	1x Fernbedienung, Deckel Kapitel 4.1.2	Unterseite auf Druckbett; Support notwendig (organischer Support empfohlen).
	1x Fernbedienung, oberer Zwischenrahmen Kapitel 4.1.2	Unterseite auf Druckbett; kein Support notwendig.
	1x Fernbedienung, Boden Kapitel 4.1.2	Oberseite auf Druckbett; kein Support notwendig.

	1x Fernbedienung, Joystick-Verlängerung Kapitel 4.1.3	Oberseite auf Druckbett; kein Support notwendig
	1x Fernbedienung, Batterieabdeckung Kapitel 4.1.3	Unterseite auf Druckbett; kein Support notwendig
	1x Motorritzel (25 Zähne) Kapitel 4.2.1	Flache Seite auf Druckbett; kein Support notwendig. Bauteil am besten allein drucken!
	1x Zwischenzahnrad (16 Zähne) mit 2x Lageraufnahme. Kapitel 4.2.2	Zahnrad mit flacher Seite auf Druckbett (Bauteil am besten allein drucken); Lageraufnahme mit Außenseite auf Druckbett. Kein Support notwendig.
	1x Differentialzahnrad (24 Zähne) mit Abstandshalter Kapitel 4.2.3	Zahnrad mit flacher Seite auf Druckbett (Bauteil am besten allein drucken).

	2x Kronenzahnrad (15 Zähne) Kapitel 4.2.3	Zahnrad wird zweimal benötigt. Mit den Zähnen nach oben drucken. Support ist zwingend notwendig!
	2x Kronenzahnrad (10 Zähne) mit 1x Abstandshalter Kapitel 4.2.3	Zahnrad wird zweimal benötigt. Mit den Zähnen nach oben drucken. Kein Support notwendig.
	2x Differential-Körper Kapitel 4.2.3	Differential-Körper wird zweimal benötigt. Mit der Außenseite nach oben drucken. Support ist zwingend notwendig (organischer Support empfohlen).
	2x Hinterradfelge mit Reifen und Spacer Kapitel 4.2.4	Felge mit der Außenseite nach unten drucken; Support notwendig. Spacer mit der Innenseite nach unten drucken. Reifen aus möglichst weichem TPU drucken.

	2x Hinterradachse (in zwei Ausführungen mit 2.5 mm Längenunterschied) Kapitel 4.2.4	Achsen sollten aus TPU gedruckt werden (hartes TPU ist empfohlen). Achsen mit der abgeflachten Seite nach unten drucken. Kein Support notwendig.
	1x Chassis-Boden Kapitel 4.2.5	Unterseite nach unten; Support wird benötigt.
	1x Chassis-Abdeckung Kapitel 4.2.5	Oberseite nach unten; kein Support notwendig.
	2x Vorderradfelge mit Reifen, Sicherungsring und Spacer Kapitel 4.3.1	Felge mit der Außenseite nach unten drucken. Spacer mit der Außenseite nach unten drucken. Support für beide Teile notwendig. Reifen aus möglichst weichem TPU drucken.
	2x Vorderradaufhängung (gespiegelte Ausführungen für Links und Rechts) mit Spacer und Lenkgestänge-Verbinder Kapitel 4.3.1, 4.3.2	Vorderradaufhängung mit der Unterseite nach unten drucken. Lenkgestänge-Verbinder mit der Oberseite nach unten drucken. Spacer mit der Innenseite nach unten drucken. Kein Support notwendig.

	1x Lenkungsbügel Kapitel 4.3.2	Oberseite nach unten, kein Support notwendig.
	1x Chassis- Sicherungsbügel Kapitel 4.3.3	Oberseite nach unten, kein Support notwendig.
	1x Batteriefach-Abdeckung Kapitel 4.4.2	Unterseite nach unten, kein Support notwendig.
	1x Hinterer Stoßfänger Kapitel 4.4.3	Oberseite nach unten, kein Support notwendig.
	1x Vorderer Stoßfänger Kapitel 4.4.3	Unterseite nach unten, kein Support notwendig.

	1x Servohorn-Aufsatz Kapitel 4.4.5	Oberseite nach unten, kein Support notwendig. Aus TPU drucken! Der Servohorn-Aufsatz ist als kleinere und größere Ausführung vorhanden. Die für das Servohorn passende Größe muss gedruckt werden (siehe 4.4.5).
	1x Karosserie, Heck Kapitel 4.5.1	Verbindungsseite zur Front nach unten. Organischer Support notwendig. Rand zur besseren Haftung aktivieren.
	1x Heckspoiler-Halterung Kapitel 4.5.1	Innenseite (zur Fahrzeugfront zeigende Seite) nach unten. Kein Support notwendig.
	1x Heckspoiler Kapitel 4.5.1	Rückseite nach unten. Support notwendig. Rand für bessere Haftung aktivieren.
	1x Karosserie, Front Kapitel 4.5.2	Verbindungsseite zum Heck nach unten. Organischer Support notwendig. Rand zur besseren Haftung aktivieren.

	Scheiben (1x Front, 1x links vorn, 1x links hinten, 1x rechts vorn, 1x rechts hinten, 1x Heck) Kapitel 4.5.2	Scheiben mit den Außenseiten nach unten drucken. Kein Support notwendig.
	1x Scheibenhalterung Kapitel 4.5.2	Oberseite nach unten, Support empfohlen.
	1x Frontlichtverklebung mit 2x LED-Einsatz Kapitel 4.5.3	Frontlichtverklebung mit der Oberseite nach unten drucken, Support ist notwendig. LED-Einsatz mit den Innenseiten (zur Verklebung zeigend) nach unten drucken. LED-Einsatz wird zweimal benötigt.
	2x Rücklichtverklebung und Auspuff mit LED-Einsatz (LED-Einsatz in gespiegelten Ausführungen für Links und Rechts) Kapitel 4.5.4	Rücklichtverklebung und LED-Einsatz mit der Innenseite (zur Fahrzeugfront zeigende Seite) nach unten drucken. Kein Support notwendig. Die Verklebung wird zweimal benötigt.

3. Elektronik

3.1 Einführung in die Elektronik

EasyRC nutzt den ESP32 Microcontroller für die Fernsteuerung, um Nutzereingaben über den Joystick zu erfassen und diese drahtlos an das Fahrzeug zu senden. Im Fahrzeug befindet sich ebenfalls ein ESP32 Microcontroller, um diese Datenpakete zu empfangen und in Befehle für die Lenkung und den Antriebsmotor umzusetzen. Es wird eine kleine Entwicklerplatine, der Xiao ESP32-C3 der Firma SEEEDStudio genutzt, da diese Plattform alle benötigten Komponenten enthält, um den Microcontroller zu programmieren (via USB-C), die benötigte Stromversorgung für den Microcontroller (3.3 V) aus einer höheren Eingangsspannung bereitstellen kann und mit einfach erreichbaren Pins zur Verbindung mit Peripherie ausgestattet ist. Der ESP32 ist für ein solches System die ideale Wahl, da es sich hierbei um einen sehr kostengünstigen Microcontroller handelt, der alle benötigten Funktionen ohne externe Zusatzchips bereitstellt: Er verfügt nicht nur über digitale Ein- und Ausgänge (um einfache Schalter auszulesen bzw. An/Aus-Befehle auszugeben), sondern auch über analoge Eingänge, um Sensoren mit nicht-diskreten Messsignalen auszulesen und in digitale Informationen umzuwandeln. Dazu beinhaltet der ESP32 Analog-Digital-Wandler, die ein variables Messsignal wie den Joystick der Fernbedienung fein aufgelöst in verarbeitbare Zahlenwerte für den Microcontroller umwandeln. Für eine ebenso feine Ansteuerung des Lenkungsmotors und des Antriebsmotors verfügt der ESP32 über PWM-taugliche Ausgänge. PWM steht für Pulsweitenmodulation und ist eine Methode, um über einen digitalen Ausgang, der nur 1 oder 0 ausgeben kann, ein fein differenziertes Signal zu übertragen, indem die zeitliche Komponente berücksichtigt wird (wie lange je Ausgabezyklus ist der Ausgang auf 1 bzw. auf 0 gesetzt? („PWM-Tastgrad“)). Dabei kann der ESP32 den Ausgabezyklus, auch PWM-Frequenz genannt, in einem sehr breiten Intervall anpassen sowie gleichzeitig unterschiedliche Frequenzen auf unterschiedlichen Ausgängen bereitstellen. Zu guter Letzt beinhaltet der ESP32 auch ein Modul zur drahtlosen Kommunikation. Er funkt bei 2,4 GHz und kann so mit herkömmlichen WiFi-Netzwerken kommunizieren. In diesem Fall wird davon allerdings kein Gebrauch gemacht, sondern das ESP-eigene 2,4-GHz-Kommunikationsprotokoll ESP-NOW genutzt. Damit kann unabhängig von (nicht) vorhandenen WLAN eine Verbindung zwischen zwei ESP32-Microcontrollern aufgebaut werden, sodass die Fernsteuerung ihre Datenpakete direkt zum Fahrzeug schicken kann. Die Xiao ESP32-C3 Entwicklerplatine kommt außerdem mit einer mitgelieferten externen Antenne. Sie muss installiert werden, um die erforderliche Funkreichweite für ein ferngesteuertes Fahrzeug zu erreichen.

Der ESP32 enthält somit alle grundsätzlichen Funktionen für den Einsatz als Funk- und Kontrolleinheit für die EasyRC-Plattform, aber es werden dennoch noch Leiterplatten benötigt, die weitere Komponenten enthalten, um das Fahrzeug und die Fernsteuerung verwenden zu können. So werden beispielsweise für die LED-Beleuchtung des Fahrzeugs Transistoren als Schalter verwendet, da die Leuchtdioden mehr Strom benötigen, als der Microcontroller direkt bereitstellen kann. Er schaltet über den Transistor eine separate Stromversorgung für die LEDs. Für den Antriebsmotor wird ein separater Motortreiber benötigt, der eine ähnliche Funktion wie der Transistor für die LEDs erfüllt, aber noch mehr können muss: Er erlaubt es, die Drehrichtung und Geschwindigkeit eines Gleichstrom-Bürstenmotors mithilfe von nur zwei Pins (einer gibt die Drehrichtung vor, der andere die Geschwindigkeit) einzustellen. Dazu kommen kleinere Ergänzungen wie Widerstände als Spannungsteiler, um den ESP32 die Batteriespannung auslesen zu lassen, oder ein aus einem Transistor aufgebauter Verpolungsschutz, der die Platinen davor schützt, bei falsch angeschlossener Batterie zerstört zu werden.

3.2 Fertigung und Bestückung der Leiterplatten

Die Fertigungsdateien für die Leiterplatten sind auf GitHub verfügbar (<https://github.com/TRD-B/EasyRC>), können bei verschiedenen Auftragsfertigern wie JLCPCB, PCBWay oder Aisler bestellt werden und anschließend mit den benötigten Komponenten bestückt werden. Dazu sollte bleifreies Lötzinn verwendet werden (Gesundheitsschutz!). Die zu verlötenden Komponenten sind vorwiegend SMD-Bauteile, also Komponenten, die lediglich auf die Oberseite der Platine gelegt und anschließend mit der Platine verlötet werden. Das ist etwas schwieriger als THT-Bauteile, die durch die Platine gesteckt und auf der Rückseite verlötet werden, da sie beim Lötvorgang noch nicht fixiert sind. Die SMD-Bauteile sind aber kleiner als vergleichbare THT-Bauteile, sodass die Platinen kompakter gehalten werden können. Außerdem kommen nur Größen zum Einsatz, die noch groß genug sind, um ohne Spezialwerkzeug mit einem HandlötKolben verlötet zu werden (kleinste Größe: 0805 (2 mm x 1,25 mm)).

Als Faustregel für das Bestücken von Leiterplatten gilt, dass mit den kleinsten SMD-Bauteilen begonnen werden sollte. So arbeitet man sich von klein nach groß und erst am Schluss werden THT-Bauteile verlötet. So wird verhindert, dass bereits platzierte Komponenten bei den folgenden Schritten im Weg sind. Für ein komfortables Arbeiten ist eine „helfende Hand“, also eine mit flexiblen Armen ausgestattete Halterung für die Leiterplatte empfehlenswert. Die SMD-Bauteile werden am einfachsten verlötet, indem sie mit einer feinen Pinzette gehalten und dann am ersten Kontaktpad verlötet werden. Sobald sie an einer Stelle fixiert sind, halten sie ohne Pinzette und können an den verbleibenden Stellen verlötet werden. Wichtig ist, beim weiteren Löten nicht den ersten Kontakt wieder zu schmelzen, da sie ansonsten verschoben werden. Die verwendeten SMD-Bauteile lassen sich leicht richtig auf den Platinen orientieren: Widerstände und Keramik-Kondensatoren besitzen nur zwei Kontakte und sind „unpolar“, setzen also keine bestimmte Einbaurichtung voraus. Die Transistoren besitzen drei Kontakte, können jedoch aufgrund ihrer Geometrie nicht falsch positioniert werden (wenn die drei Kontakte auf den Pads der Leiterplatte aufliegen, ist der Transistor richtig ausgerichtet). Bei den THT-Komponenten muss mehr auf die richtige Positionierung geachtet werden: Der Piezo-Buzzer ist auf einer Seite mit „+“ beschriftet und auch die Elektrolyt-Kondensatoren sind polarisierte Bauteile, müssen also mit der Anode (kurzer Pin) zum Pluspol und der Kathode (langer Pin; weißer breiter Streifen auf dem Gehäuse mit schwarzem „-“) zum Minuspol ausgerichtet werden. Dasselbe gilt für die LEDs, deren Anschlusspins ebenfalls polarisiert sind (Anode kurz; Kathode lang). Beim Verlöten der Stiftleisten, die zum Anschließen externer Bauteile benötigt werden, kann eine Buchsenleiste als Hilfe verwendet werden, um die Stiftleiste vertikal ausgerichtet zu halten. Alternativ kann dafür auch eine Pinzette verwendet werden. Als letztes sollten die großen Bauteile (Joystick-Modul der Fernsteuerung und Spannungswandler, Piezo-Buzzer sowie Elektrolytkondensatoren der Fahrzeugplatine) verlötet werden.

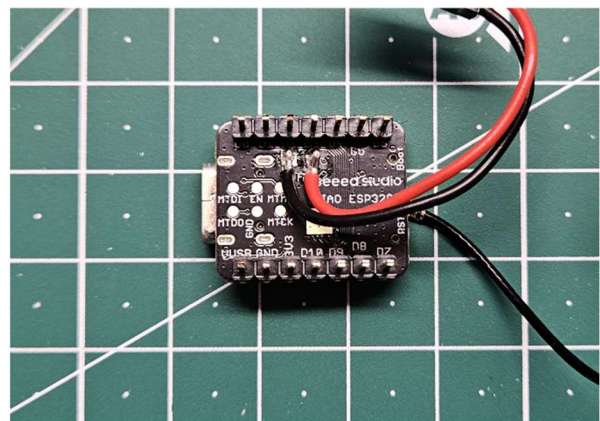
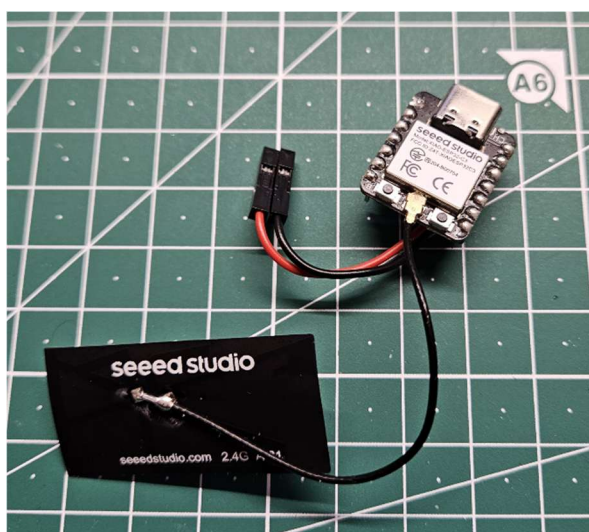
Nach dem Verlöten der Komponenten muss geprüft werden, dass sämtliche Kontakte korrekt hergestellt worden sind (jeder Kontakt muss mechanisch stabil verlötet sein: ein leichtes Drücken mit dem Finger gegen die Komponente darf diese nicht ablösen) und, dass keine Lötbrücken zwischen verschiedenen Kontakten vorhanden sind (Lötzinnverbindungen, die benachbarte Kontaktpads überbrücken). Erst wenn das nach optischer Prüfung gewährleistet worden ist, dürfen die Platinen mit den aufsteckbaren ESP32-C3 Leiterplatten sowie dem Pololu Motortreiber bestückt und mit Strom versorgt werden. Bei den aufsteckbaren Leiterplatten muss auf die richtige Ausrichtung geachtet werden, da sie andernfalls (bei Montage um 180° gedreht) nicht funktionieren und beschädigt werden können.

3.3 Crimpen und Verlöten der Elektronikbauteile

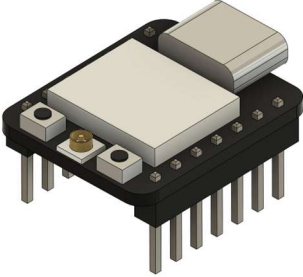
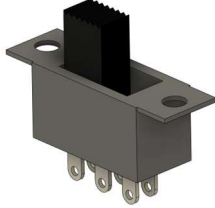
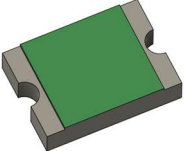
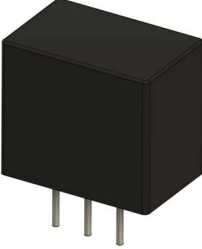
Zur Verbindung zwischen den Leiterplatten und den damit verbundenen Elektronikbauteilen (siehe Kapitel 2.1) sollten Kabel mit einem Querschnitt von 0.25 mm^2 verwendet werden. Das ist zwar mehr als für die LEDs notwendig ist, reicht aber für den Motor und den An/Aus-Schalter des Fahrzeugs aus und so muss nur mit einer Größe gearbeitet werden. Es empfiehlt sich, eine Kupferlitze mit roter und eine mit schwarzer Isolierung zu nutzen, um Bauteile mit Polarisierung (z.B. LEDs) ohne Probleme korrekt gepolt anschließen zu können: Als gängige Konvention wird Rot für Plus und Schwarz für Minus eingesetzt.

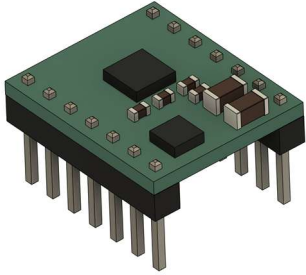
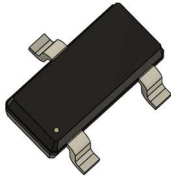
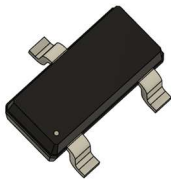
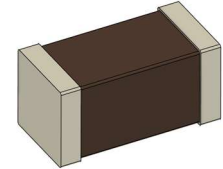
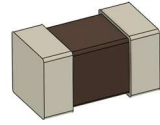
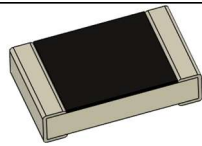
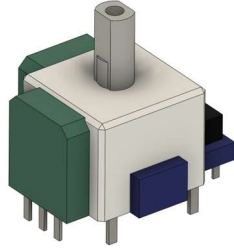
Die Elektronikbauteile werden bauteilseitig mit den Litzen verlötet und platinenseitig mit 2.54 mm Dupont-Steckern (weiblich, Buchse) gecrimpt, sodass sie einfach ein- und ausgesteckt werden können. Beim Crimpen sollte darauf geachtet werden, dass die Verbindung mechanisch stabil ist: Die Litze darf sich auch unter Zug nicht aus dem Crimp-Pin lösen. Je nach verwendetem Crimp-Werkzeug muss die Isolierung der Litze separat gecrimpt werden (Crimpen in zwei Schritten) oder wird in einem Arbeitsschritt mit vercrimpt. Bei letzterem kann es vorkommen, dass die Isolierung nicht mit ausreichend Druck gecrimpt wird, sodass noch mit einer Zange nachgecrimpt werden muss, um den Pin ohne Probleme in das Buchsengehäuse einführen zu können.

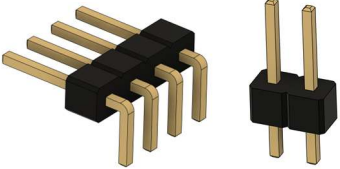
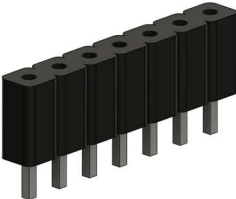
Die beiden folgenden Bilder zeigen den Xiao ESP32-C3 der Fernsteuerung. Die Antenne muss auch am Microcontroller des Fahrzeugs installiert werden, aber nur der Microcontroller für die Fernsteuerung benötigt verlötete Kabel an den Energieversorgungspads auf der Unterseite der Platine. Bei der Fernsteuerung sollte darauf geachtet werden, dass die Anschlussleitungen zwischen ESP32-C3 und Platine bzw. Batteriehalter und Platine so kurz ausgeführt werden, dass sich das Gehäuse problemlos schließen lässt (siehe auch Kapitel 4.1.1). Beim Fahrzeug sollte hingegen eher mit etwas längeren Leitungen gearbeitet werden, da hier ein Verstauen der Kabel einfacher möglich ist. Vor allem die LEDs des Fahrzeugs benötigen lange Anschlussleitungen (empfehlenswert sind 30-40 cm), um das Auf- und Absetzen der Karosserie ohne ein versehentliches Abstecken der LEDs zu ermöglichen. Der Motor und der An/Ausschalter des Fahrzeugs benötigen keine unterschiedlich farbkodierten Kabel, da die Anschlussrichtung beim Schalter keine Rolle spielt und die Drehrichtung des Motors, die von der Polarisierung abhängt, einfach im Code der Fernsteuerung korrigiert werden kann (Kapitel 5.2).



3.4 Komponentenliste

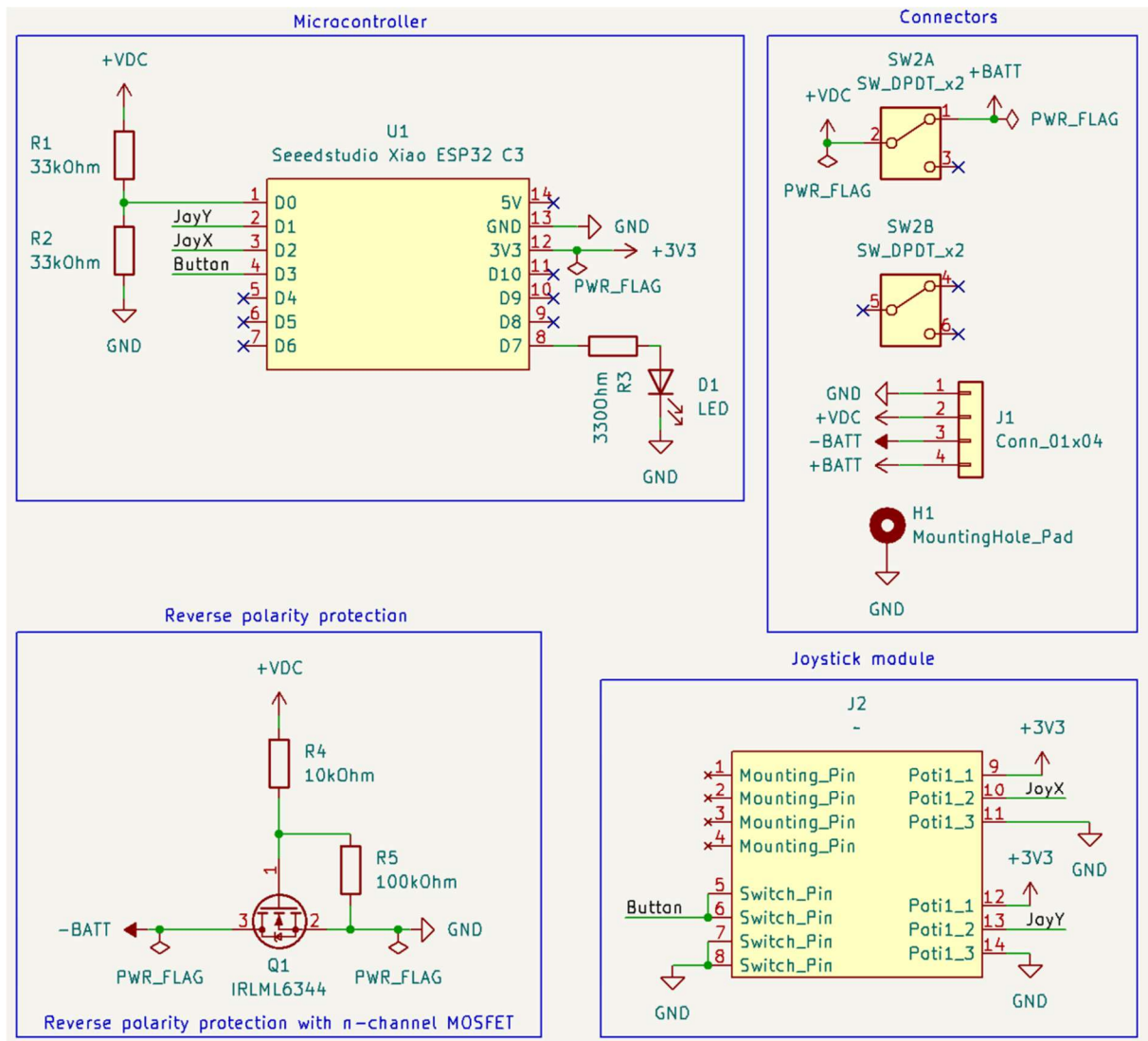
Abbildung	Bezeichnung	Beschreibung
	2x Microcontroller-Entwicklerplatine	Typ: SEEED Studio Xiao ESP32-C3
	1x Miniatur-Schiebeschalter	6-Pol, Typ Wentronic S 166. Schieber-Höhe: 6-10 mm
	1x Selbstrückstellende PTC-Sicherung, 1,5 A	Bauform: SMD 1812 Typ: ESKA FSMD1812 (FSMD150-24R)
	1x DC-DC Wandler, 5 V 1 A	3-Pol Bauform, Pin-kompatibel mit LMxx-Linearreglern Typ: Traco TSR1-2450E
	2x Elektrolyt-Kondensator, 25 V 100 µF	6,3 mm Durchmesser, 2,5 mm Rastermaß. Typ: Panasonic EEUFM1E101
	1x Piezo-Buzzer	12 mm Durchmesser, 6,5 mm Rastermaß Typ: HITPOINT PB1220PE05Q
	1x 5mm LED bedrahtet, grün	Typ: Quadrios 2111O169

	1x Motortreiber Breakout-Board	Typ: Pololu DRV8876QFN
	6x MOSFET n-Kanal, 5 A 30 V	Bauform: SMD SOT-23 Typ: Infineon IRLML 6344
	1x Zener-Diode, 8,2 V	Bauform: SMD SOT-23 Typ: CDIL BZX84C8,2
	2x Keramik-Kondensator, 10 V 22 μ F	Bauform: SMD 1206 Typ: Murata GRM31CR71A226KE15L
	1x Keramik-Kondensator, 50 V 100 nF	Bauform: SMD 0805 Typ: Walsin 0805B104K500CT
	Widerstand, 125 mW Benötigte Widerstandswerte: 1x 330 Ohm 16x 470 Ohm 4x 1 kOhm 1x 10 kOhm 3x 22 kOhm 2x 33 kOhm 2x 100 kOhm	Bauform: SMD 0805 Typ: Walsin WR08X
	Joystick-Modul (kompatibel mit PS4-Controller)	Typ: ALPS 10kOhm

	Stiftleiste, 2,54 mm Rastermaß Benötigte Größen: 1x 1x4 (gewinkelt) 4x 1x2 (gerade) 1x 1x3 (gerade) 4x 2x4 (gerade)	Typ: W+P 946-14-004-00 (1x4 gewinkelt) MPE 087-1-002-0-S-XS0-1260 (1x20, kann gekürzt werden) MPE 087-2-040-0-S-XS0-1260 (2x20, kann gekürzt werden)
	1x Kurzschlussbrücke, 2,54 mm Rastermaß	Typ: Frei Jumper 2,54 SW
	Buchsenleiste, 2,54 mm Rastermaß. Benötigte Größen: 4x 1x7 (8.5 mm Höhe) 2x 1x7 (5 mm Höhe)	Typ: MPE 094-1-007-0-NFX-YS0 (8.5 mm Höhe) Frei BL 1X10G 2,54 (5 mm Höhe, muss auf 1x7 gekürzt werden) Für die Fernbedienung werden niedrigere Leisten benötigt, um die Bauhöhe der Platine kleiner zu halten. Dazu müssen die Stifte der Microcontroller-Platine gekürzt werden.

3.5 Übersicht Fernsteuerungselektronik

3.5.1 Schaltplan



Netzbezeichnungen:

+BATT: Batterie-Anode (1S Lilon)

+VDC: Batterie-Anode nach An/Aus-Schalter

+3V3: Geregelte 3.3 V (DC-DC Wandler in Xiao ESP32-C3 integriert)

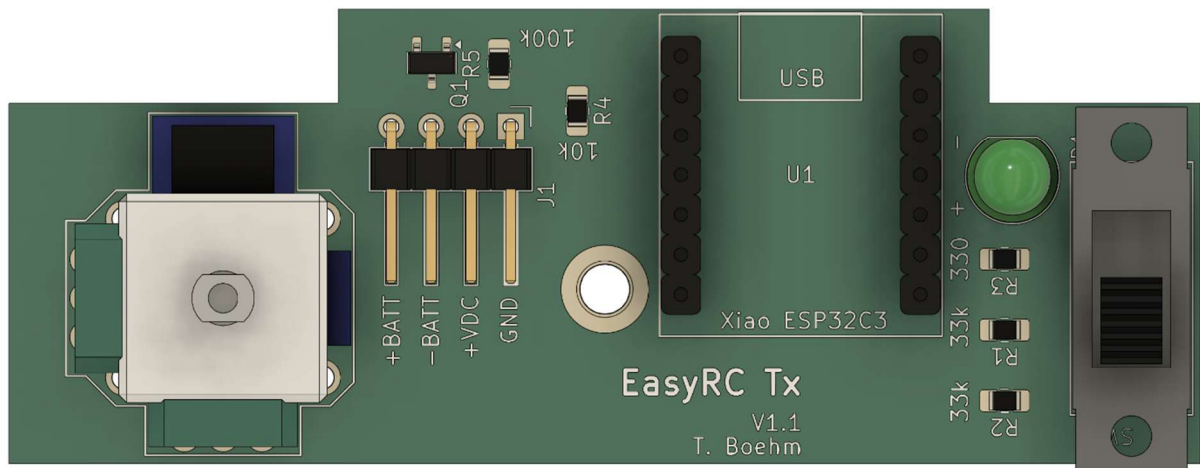
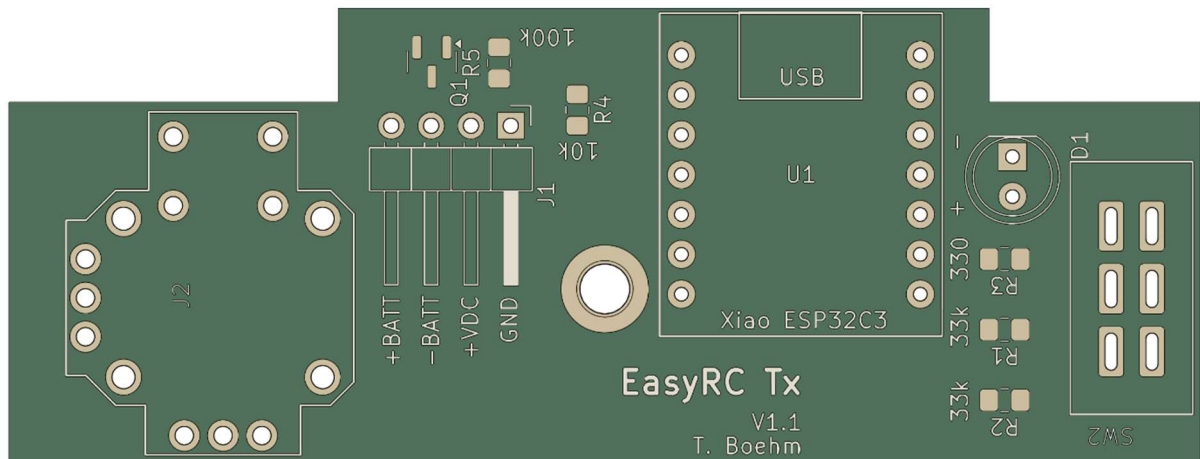
-BATT: Batterie-Kathode

GND: Batterie-Kathode nach Verpolungsschutz

Erläuterungen zu den Ausschnitten des Schaltplans:

- **Microcontroller:** Umfasst die Ein- und Ausgänge des Xiao ESP32-C3. An Pin D0 wird die Batteriespannung über einen Spannungsteiler halbiert, um sicher mittels des analogen Eingangs des Microcontrollers gemessen werden zu können. Die Pins D1 bis D3 werden mit dem Joystick-Modul verbunden. Da als Analogpins die Pins D0 bis D2 verwendet werden können, sind D1 und D2 für die beiden Achsen des Joysticks vorgesehen. D3 wird als digitaler Eingang für den Druckknopf des Joystick-Moduls genutzt. Die LED an Pin D7 wird mit einem Vorwiderstand betrieben, um zu verhindern, dass die LED durchbrennt oder der Microcontroller durch zu viel Stromabgabe beschädigt wird (der Stromfluss sollte nicht mehr als 10-15 mA betragen). Der Funktionsumfang der Pins kann hier nachgelesen werden:
https://wiki.seeedstudio.com/XIAO_ESP32C3_Getting_Started/#
- **Connectors:** Hier werden die verschiedenen Ein- und Ausgänge der Platine abgebildet. Der Miniaturschalter verfügt über zwei galvanisch getrennte Schalter, von denen nur einer benötigt wird, um die Stromzufuhr zur Platine an- und abzuschalten. Der 4-Pin-Anschluss wird dazu genutzt, um die Platine mit der Batterie sowie mit den Stromversorgungs pads auf der Unterseite des Xiao ESP32-C3 zu verbinden. Die Stromversorgungspins des Xiao ESP32-C3 erlauben es, den Microcontroller direkt mit einer einzelnen Lilon-Batterie zu betreiben, da die Entwicklerplatine über einen integrierten Spannungswandler verfügt, der aus der Batteriespannung stabile 3.3 V erzeugt.
- **Reverse polarity protection:** Als Verpolungsschutz für die Platine wird ein „falschherum“ verbauter n-Kanal MOSFET zwischen der Batterie-Kathode und der Platine verwendet. Das sorgt dafür, dass die Platine bei korrekt gepolter Batteriespannung über die Bodydiode des Transistors mit Strom versorgt werden kann. Sobald das erfolgt, liegt auch am Gate eine positive Spannung an, sodass der Transistor durchschaltet und der Spannungsabfall über die Bodydiode des Transistors als Verlust wegfällt. Sollte die Batterie verpolt angeschlossen werden, sperrt die Bodydiode des Transistors den Stromfluss und am Gate liegt eine negative Spannung an (+VDC entspricht der negativen Batteriespannung), sodass kein Strom zwischen Source und Drain fließen kann. Der Transistor verhält sich als ideale Diode.
- **Joystick module:** Die beiden Potentiometer des Joysticks werden mit 3.3 V aus dem ESP32-C3 gespeist, sodass die Analogausgänge eine für den Microcontroller messbare Spannung zwischen Logiklevel (3.3 V) und gemeinsamer Masse (0 V) bereitstellen. Der Druckknopf wird mit Masse und dem Microcontroller verbunden, sodass dieser mithilfe eines internen Pullup-Widerstands die Zustände 0 und 1 einnehmen kann (je nachdem, ob der Knopf gedrückt wird oder nicht).

3.5.2 Leiterplatte

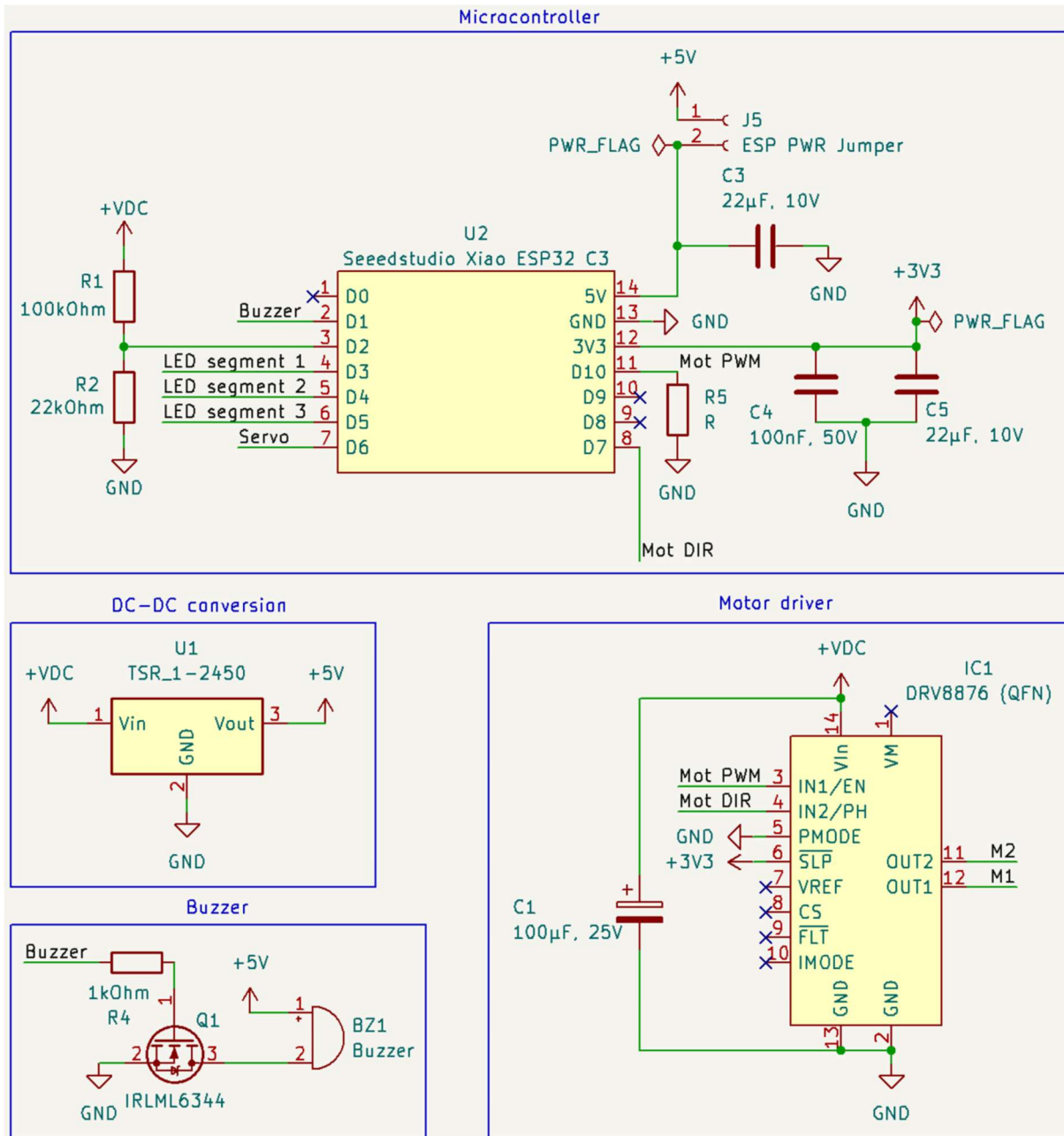


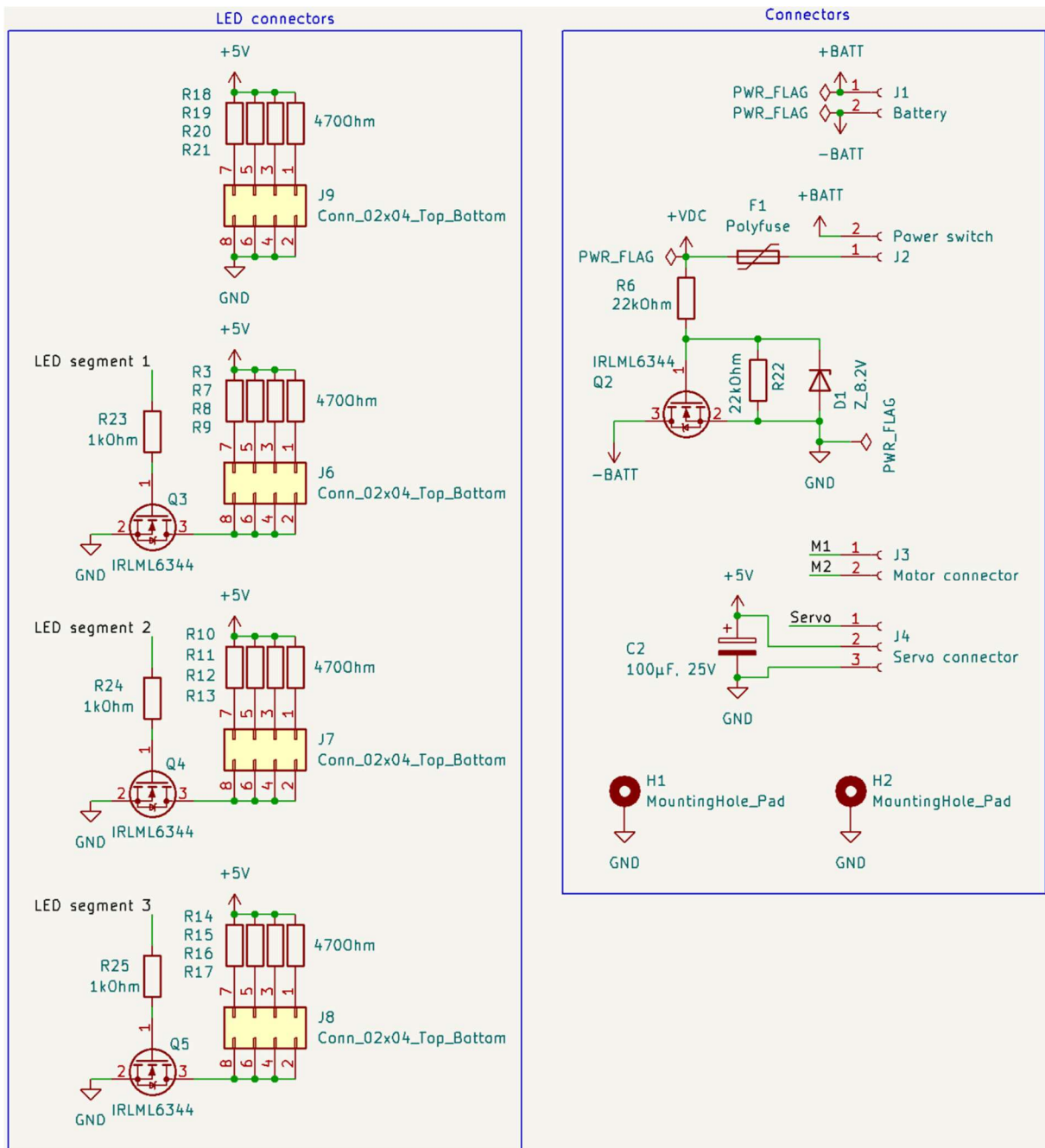
Bezeichnung	Komponente
D1	5 mm LED, grün
J1	1x4 2,54 mm gewinkelte Stiftleiste
J2	Alps 10k Joystick-Modul
Q1	SOT-23 MOSFET IRLML 6344
R1	0805 Widerstand 33 kOhm
R2	0805 Widerstand 33 kOhm
R3	0805 Widerstand 330 Ohm
R4	0805 Widerstand 10 kOhm
R5	0805 Widerstand 100 kOhm
SW2	Wentronic S 166 6-Pol Miniaturschalter
U1	2x 1x7 2,54 mm Buchsenleiste (5 mm Bauhöhe)
Anmerkung: Die Annotation der Leiterplatten-Version 1.1 muss noch verbessert werden.	

Ein Foto der mit Microcontroller bestückten und mit der Batteriehalterung verbundenen Platine ist in Kapitel 4.1.1 abgebildet.

3.6 Übersicht Fahrzeugelektronik

3.6.1 Schaltplan





Netzbezeichnungen:

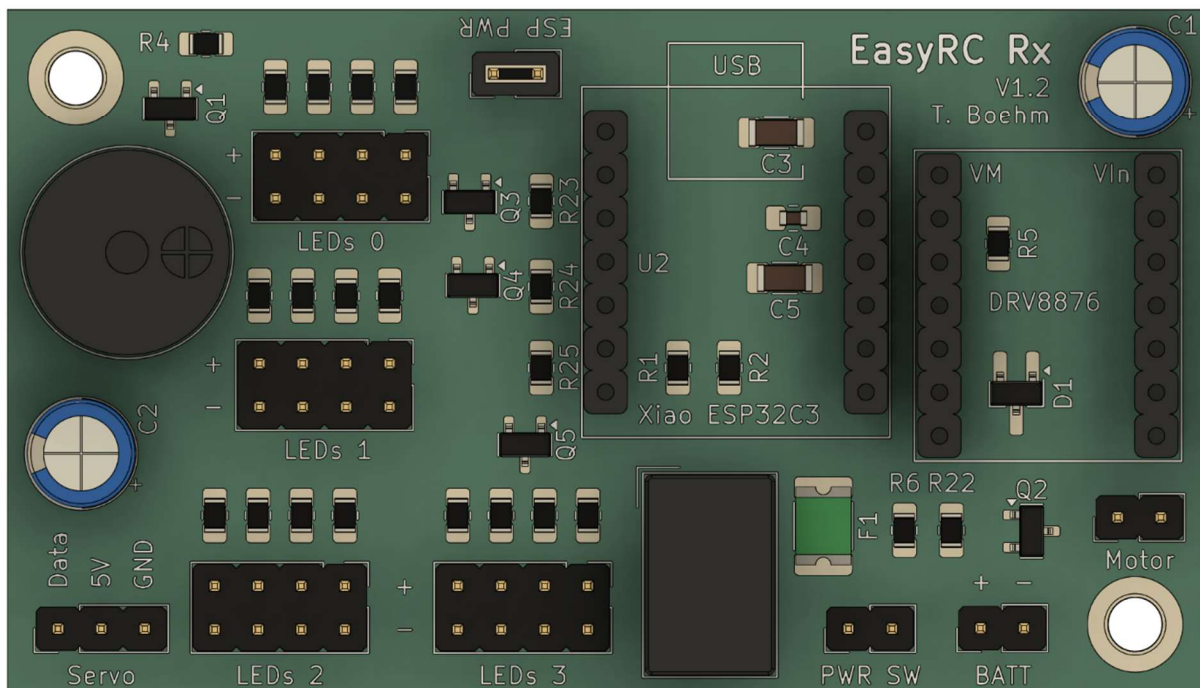
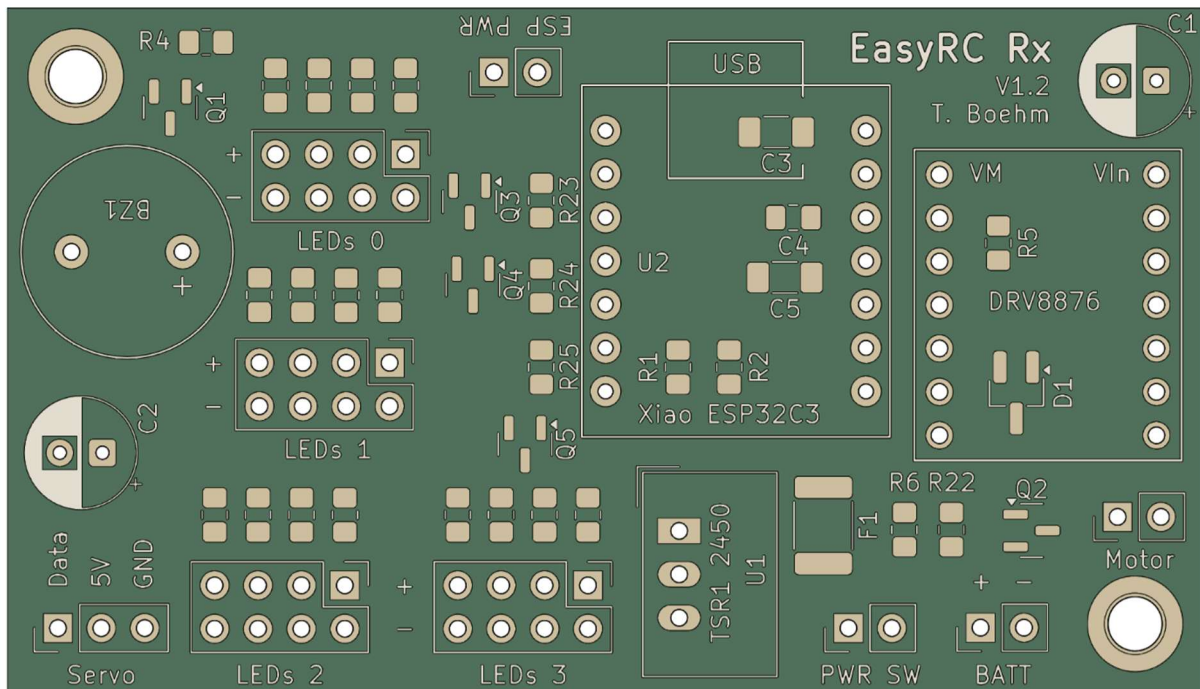
- +BATT: Batterie-Anode (3S Lilon)
- +VDC: Batterie-Anode nach An/Aus-Schalter
- +5V: Geregelte 5V (DC-DC Wandler)
- +3V3: Geregelte 3.3 V (DC-DC Wandler in Xiao ESP32-C3 integriert)
- BATT: Batterie-Kathode
- GND: Batterie-Kathode nach Verpolungsschutz

Erläuterungen zu den Ausschnitten des Schaltplans:

- **Microcontroller:** Umfasst die Ein- und Ausgänge des Xiao ESP32-C3. An Pin D2 wird die Batteriespannung über einen Spannungsteiler reduziert, um sicher mittels des analogen Eingangs des Microcontrollers gemessen werden zu können. Die Pins D1, D3 bis D7 und D10 dienen als Ausgänge. D1, D6 und D10 werden als PWM-Ausgänge verwendet; die anderen als einfache digitale Ausgänge. An D10 (PWM-Steuerung des Antriebsmotors) wird Widerstand R5 hinzugefügt, der als optionaler Pulldown-Widerstand dient. Wenn hier ein hochohmiger Widerstand eingesetzt wird, können unkontrollierte Motorbewegungen beim Hochfahren des Microcontrollers, wenn der Ausgang D10 noch undefiniert ist (floated), verhindert werden. Der 5 V-Eingang des Microcontrollers wird über eine 2-Pin-Stiftleiste unterbrochen, sodass die Verbindung zwischen dem Microcontroller und der 5 V-Leitung der Platine über das Entfernen eines Jumpers unterbrochen werden kann. So kann verhindert werden, dass bei einer bestehenden USB-Verbindung Strom vom PC zur Platine fließen kann, da dies den USB-Ausgang überlasten könnte. Am 5 V-Eingang und dem 3,3 V-Ausgang sind Kondensatoren verbaut, um Spannungsschwankungen zu minimieren und einen stabilen Betrieb zu gewährleisten.
- **DC-DC conversion:** Hier wird der Abwärtswandler gezeigt, der aus der 3S Lilon-Batterieversorgung der Platine stabile 5 V für den Microcontroller, den Servomotor der Lenkung, die LEDs und den Piezo-Buzzer erzeugt.
- **Motor driver:** Der Pololu-Motortreiber wird über zwei Pins des Microcontrollers mit Daten versorgt (Geschwindigkeit und Drehrichtung). Die weiteren Pins des Treibers werden fix mit Logikspannung und Masse verbunden oder bleiben unverbunden. In der Nähe des Motortreibers wird ein Stützkondensator verbaut, um bei schnellen Laständerungen des Antriebsmotors eine stabile Spannung aufrechtzuerhalten. Der Motor wird an die Ausgänge M1 und M2 des Motortreibers angeschlossen. Informationen zur Pinbelegung und Steuerung des Motortreibers können hier nachgelesen werden:
<https://www.pololu.com/product/4037>
- **Buzzer:** Der Piezo-Buzzer erzeugt Geräusche mit bestimmten Frequenzen, wenn eine wechselnde Spannung anliegt. Dazu wird ein PWM-Ausgang des Microcontrollers verwendet, sodass die am Piezo-Buzzer anliegende Spannung zwischen 0 V und 5 V wechselt, wobei die Frequenz über die PWM-Frequenz des Microcontrollers vorgegeben wird und damit die Tonhöhe des Piezo-Buzzers vorgibt. Da der Buzzer mehr Strom benötigt, als der Microcontroller bereitstellen kann und für den Betrieb mit 5 V statt den 3,3 V des Microcontrollers ausgelegt ist, wird ein Transistor als Schalter eingesetzt, der über den PWM-Ausgang des Microcontrollers an- und abgeschaltet wird.
- **LED connectors:** Die LED-Anschlüsse auf der Platine sind in vier Gruppen mit je vier Anschlussmöglichkeiten unterteilt. LED-Segment 0 kann nicht vom Microcontroller gesteuert werden, sondern erhält permanent Strom. Dieses Segment ist für die Frontscheinwerfer vorgesehen. Die Segmente 1 bis 3 werden (wie auch der Piezo-Buzzer) von einem über einen Transistor geschalteten Ausgang des Microcontrollers gesteuert und dienen deshalb als Blinker und Bremslicht. Jeder LED-Ausgang enthält einen eigenen Vorwiderstand, um den Stromfluss je LED zu begrenzen. Hierfür sind 470 Ohm vorgesehen, wobei dieser Wert auch angepasst werden kann, um die Lichtintensität einzustellen (bitte nicht den zulässigen Maximalstrom der LED überschreiten und die maximale thermische Verlustleistung der Vorwiderstände beachten!).
- **Connectors:** Hier sind die Platinenanschlüsse sowie der Verpolungsschutz der Platine dargestellt. Die positive Batterieverbinding wird vom Batterieanschluss an den An-

/Ausschalter weitergegeben, der dann wiederum die Platine mit Energie versorgt. Dabei wird eine Sicherung verbaut, die im Falle eines zu großen Stromflusses auslöst. Zudem wird dasselbe Verpolungsschutz-Konzept wie auch bei der Fernsteuerungsplatine genutzt. Hier wird zwingend ein Spannungsteiler benötigt, da der Transistor für eine maximale Gatespannung von 12 V ausgelegt ist und die Platine bei vollgeladenen Batterien 12,6 V erhält. Der Spannungsteiler halbiert diese Spannung, um in einem sicheren Bereich für den Transistor zu bleiben. Zudem wird eine Zener-Diode als zusätzliche Sicherung verwendet, um im Falle von induktiven Spannungsspitzen den Transistor zu schützen. In der Nähe des Servomotor-Anschlusses wird ein Stützkondensator verbaut, um Spannungseinbrüche bei Laständerungen des Lenkungsmotors zu minimieren.

3.6.2 Leiterplatte



Bezeichnung	Komponente
BATT	1x2 2,54 mm Stiftleiste
BZ1	Piezo-Buzzer (12 mm, 6,5 mm Rastermaß)
C1	Elektrolytkondensator, 6,3 mm, 2,5 mm Rastermaß, 100 µF, 25 V
C2	Elektrolytkondensator, 6,3 mm, 2,5 mm Rastermaß, 100 µF, 25 V
C3	1206 Keramikkondensator, 22 µF, 10 V
C4	0805 Keramikkondensator, 100 nF, 50 V
C5	1206 Keramikkondensator, 22 µF, 10 V
D1	SOT-23 Zenerdiode 8,2 V
DRV8876	2x 1x7 2,54 mm Buchsenleiste (8,5 mm Bauhöhe)
ESP PWR	1x2 2,54 mm Stiftleiste mit Kurzschlussbrücke
F1	1812 PTC-Sicherung 1,5 A
LEDs 0	2x4 2,54 mm Stiftleiste mit 4x 0805 Widerstand 470 Ohm
LEDs 1	2x4 2,54 mm Stiftleiste mit 4x 0805 Widerstand 470 Ohm
LEDs 2	2x4 2,54 mm Stiftleiste mit 4x 0805 Widerstand 470 Ohm
LEDs 3	2x4 2,54 mm Stiftleiste mit 4x 0805 Widerstand 470 Ohm
Motor	1x2 2,54 mm Stiftleiste
PWR SW	1x2 2,54 mm Stiftleiste
Q1	SOT-23 MOSFET IRLML 6344
Q2	SOT-23 MOSFET IRLML 6344
Q3	SOT-23 MOSFET IRLML 6344
Q4	SOT-23 MOSFET IRLML 6344
Q5	SOT-23 MOSFET IRLML 6344
R1	0805 Widerstand 100 kOhm
R2	0805 Widerstand 22 kOhm
R4	0805 Widerstand 1 kOhm
R5	(optional) 0805 Widerstand 10-100 kOhm
R6	0805 Widerstand 22 kOhm
R22	0805 Widerstand 22 kOhm
R23	0805 Widerstand 1 kOhm
R24	0805 Widerstand 1 kOhm
R25	0805 Widerstand 1 kOhm
Servo	1x3 2,54 mm Stiftleiste
U1	DC-DC Wandler Traco TSR1-2450E
U2	2x 1x7 2,54 mm Buchsenleiste (8,5 mm Bauhöhe)
Anmerkung: Die Annotation der Leiterplatte in Version 1.2 muss noch verbessert werden. Aus Platzgründen sind die Widerstände R3 und R7 bis R21 (LED-Vorwiderstände) auf der Platine nicht beschriftet.	

Die Übersicht der Anschlüsse ist in Kapitel 4.4.4 zu finden. Wichtig beim Aufsetzen des Xiao ESP32-C3 und des Motortreibers ist die richtige Ausrichtung: Für den Microcontroller ist die Ausrichtung des USB-Anschlusses auf der Leiterplatte angezeichnet und für den Motortreiber sind die Positionen der beiden Pins VM und VIn auf der Leiterplatte gekennzeichnet.

4. Bauanleitung Fernsteuerung und Fahrzeug

Für den Zusammenbau des Modellfahrzeugs und der Fernsteuerung werden die Komponenten aus den Teilelisten benötigt. Als Werkzeuge werden Innensechskantschlüssel in den Größen 2 und 2,5 mm mit Kugelkopf benötigt. Zudem werden eine kleine flache Zange und eine Pinzette benötigt bzw. sind hilfreich.

Als Material für den 3D-Druck der Bauteile wird PETG empfohlen, da dieser Kunststoff leicht zu drucken ist, eine ausreichende mechanische Stabilität bietet und UV-beständig ist. Alternativ kann allerdings auch PLA oder ein anderes gängiges Filament für mechanisch stabile, steife Drucke eingesetzt werden. Einige Bauteile müssen aus flexiblem TPU gedruckt werden. Diese sind in Kapitel 2.3 entsprechend gekennzeichnet und tragen „TPU“ im Namen. Für den Druck aller Bauteile ist ein Druckbett mit wenigstens 220 mm Länge notwendig. Empfehlungen für die Orientierung der Bauteile auf dem Druckbett sowie zum Support sind in den Hinweisen zu den gedruckten Bauteilen gelistet. Für alle Bauteile gilt, dass diese am besten mit einer 0,4 mm Düse sowie mit einer Schichthöhe von 0,2 mm gedruckt werden sollten. Eine Infill-Rate von 15 % ist ausreichend.

Zu den gedruckten Bauteilen muss angemerkt werden, dass diese keine perfekte Passform aufweisen. Das ergibt sich aus der Fertigung: 3D-Druck erzeugt in bestimmten Ausrichtungen keine perfekten Toleranzen (z.B. werden in Z-Richtung beim Drucken mit einer Schichthöhe von 0,2 mm Schritte in dieser Höhe ausgeführt, sodass sich die Form des Bauteils, sofern keine horizontale oder vertikale Kante vorliegt, der gewünschten Form nur annähern kann). Zudem benötigt 3D-Druck für bestimmte Geometrien Stützstrukturen, die die Form während des Drucks stabilisieren (man kann nicht in Luft drucken, sondern benötigt Kontakt zur Bodenplatte des Druckers, um eine Form stabil herstellen zu können). Diese Stützstrukturen, die auch Support genannt werden, verkleben teilweise mit den Bauteilen und sorgen deshalb für unsaubere Oberflächen. Die gedruckten Bauteile müssen vollständig von Support-Rückständen befreit werden, um eine korrekte Passung zu gewährleisten.

Teilweise kommt es auch zu Artefakten beim 3D-Druck, wie Stringing (es verbleiben dünne Kunststofffäden zwischen oder innerhalb von Bauteilen) oder Verzerrungen der Geometrie („Elefantenfuß“ oder Warping: Das Bauteil verzieht sich unmittelbar über der Bodenplatte des Druckers durch die auftretenden Temperatur- und Luftbewegungsschwankungen während des Druckprozesses). Die EasyRC-Komponenten wurden so entwickelt, dass diese Effekte möglichst vollständig vermieden oder aber in ihrer Auswirkung auf die Funktion vernachlässigbar sind. Dennoch kann es vorkommen, dass Bauteile nicht perfekt in ihre Position passen, rotierende Elemente in ihrem Gehäuse schleifen, oder Aussparungen für Schrauben und Muttern zu eng ausfallen, um Verschraubungen einfach durchführen zu können.

Deshalb ist es wichtig, beim Zusammenbau auf diese möglichen Fehler zu achten: Bevor Bauteile verschraubt werden, sollte geprüft werden, ob sich Schrauben einfach in die vorgesehenen Kanäle schieben lassen. Falls das nur schwer geht, sollte der Kanal ggf. mit einer längeren Schraube vorbearbeitet werden, indem das Schraubengewinde als Feile genutzt wird und durch Auf- und Abbewegungen überschüssiger Kunststoff abgetragen wird. Bei rotierenden Elementen sollte vor dem Verschrauben immer die Leichtgängigkeit geprüft werden. Fällt ein Schleifgeräusch auf, sollte geprüft werden, ob irgendwo ein Kunststofffaden oder eine Oberflächenungenauigkeit vorliegt, der oder die die Reibung erzeugt. Überschüssiges Material sollte vorsichtig mit einem Schlitzschraubendreher oder Schleifpapier entfernt werden. Der Vorteil 3D-gedruckter Bauteile

ist ihr geringer Preis und ihre schnelle Fertigung (bei kleinen Stückzahlen): Sollte ein Bauteil defekt sein, kann es innerhalb weniger Stunden neu gefertigt werden, sodass ein rascher Austausch beschädigter Komponenten möglich ist.

Das EasyRC-Modell wird mithilfe von metrischen Schrauben und Muttern zusammengesetzt. Es werden Muttern als Gewindeeinsätze verwendet, da kleine gedruckte Gewinde nur eine sehr geringe Stabilität aufweisen und deshalb nur wenige Male verwendet werden können, bevor sie abgerieben/abgetragen worden sind. Alternativ könnten gedruckte Bauteile auch mit Cyanacrylat-Kleber (Sekundenkleber) verbunden werden, was allerdings kein Lösen der Bauteile ohne ihre Zerstörung ermöglicht. Sekundenkleber oder Heißkleber können aber als schnelle Reparaturmöglichkeit für gebrochene Bauteile genutzt werden.

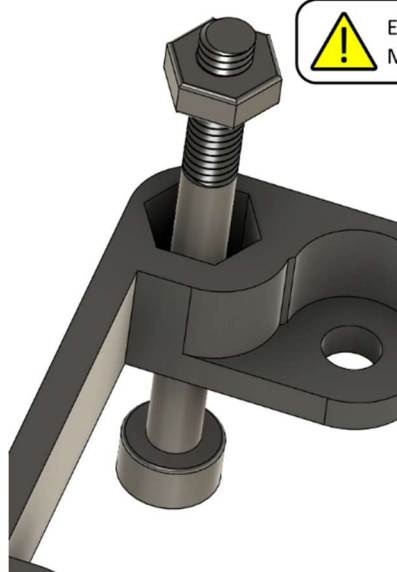
Bei den Schraubverbindungen ist es wichtig, die Muttern vor dem Verbinden der Komponenten korrekt in ihre jeweiligen Vertiefungen zu setzen. Das ist rein von Hand teilweise schwierig, was sowohl von der Position der Mutter in einem Bauteil als auch von den Toleranzen der Bauteile abhängt. Um das Einsetzen der Muttern zu vereinfachen, kann man diese auf eine Schraube drehen und mithilfe der Schraube im korrekten Winkel in die Vertiefung drücken. Alternativ setzt man die Mutter lose in die Vertiefung und greift sie von der gegenüberliegenden Seite mit einer Schraube, um sie in ihre Vertiefung zu ziehen. Diese beiden Möglichkeiten sind in der folgenden Abbildung dargestellt.



Sechskant-Vertiefungen sind als
Aufnahmepunkte für Muttern ausgelegt.
Die Mutter muss bis zum Boden der
Tasche geschoben werden, um korrekt
mit dem gegenüberliegenden Bauteil
verschraubt zu werden.



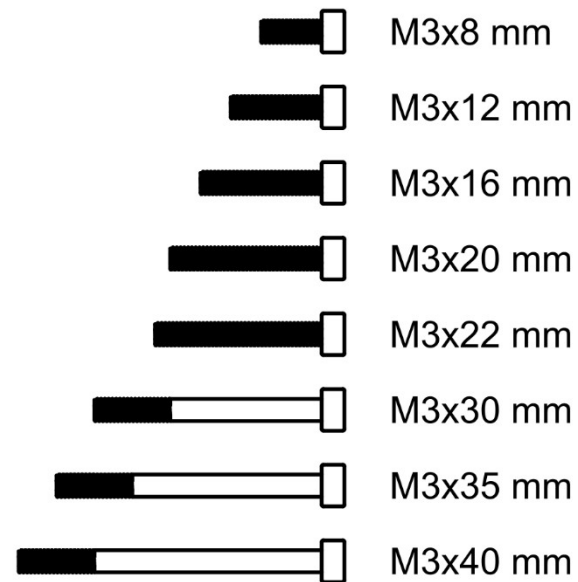
Eindrücken der
Mutter



Einziehen der
Mutter



Als Hilfestellung zum Finden der richtigen Schraubenlängen gibt es hier eine Größenliste. Wenn die Seite auf exaktem A4-Format ausgedruckt wird, entsprechen die gezeigten Größen genau den verschiedenen Schraubenlängen:

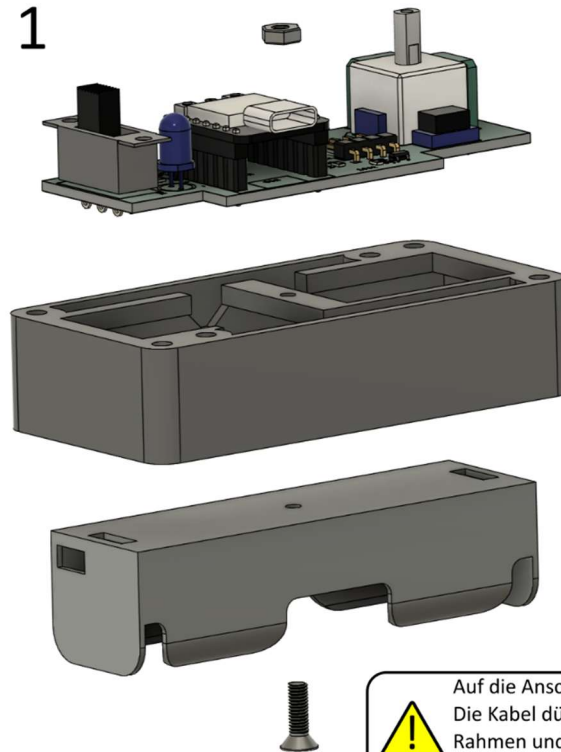


4.1 Fernbedienung

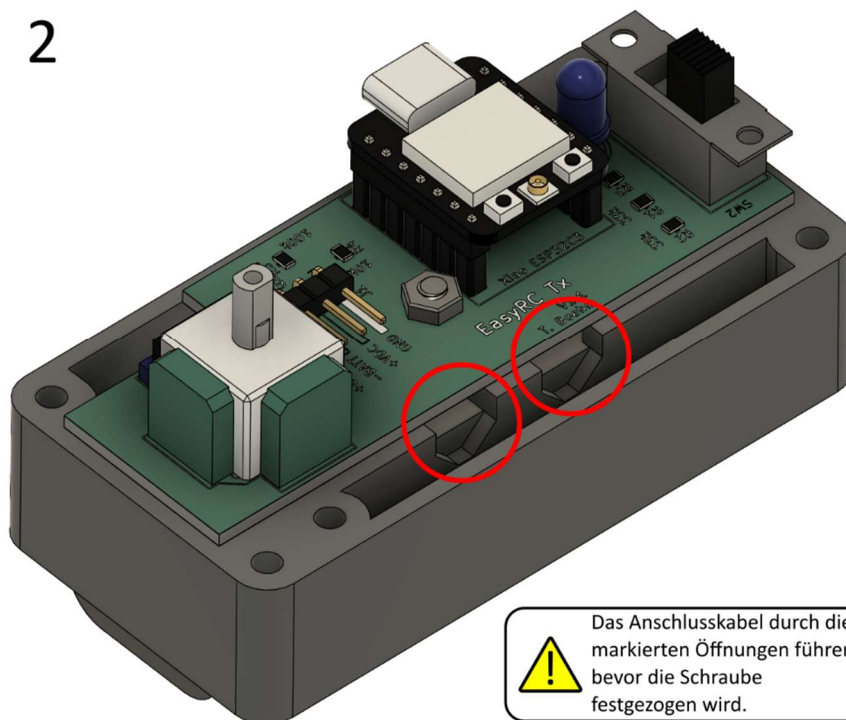
4.1.1 Einbau Leiterplatte und Batteriehalterung

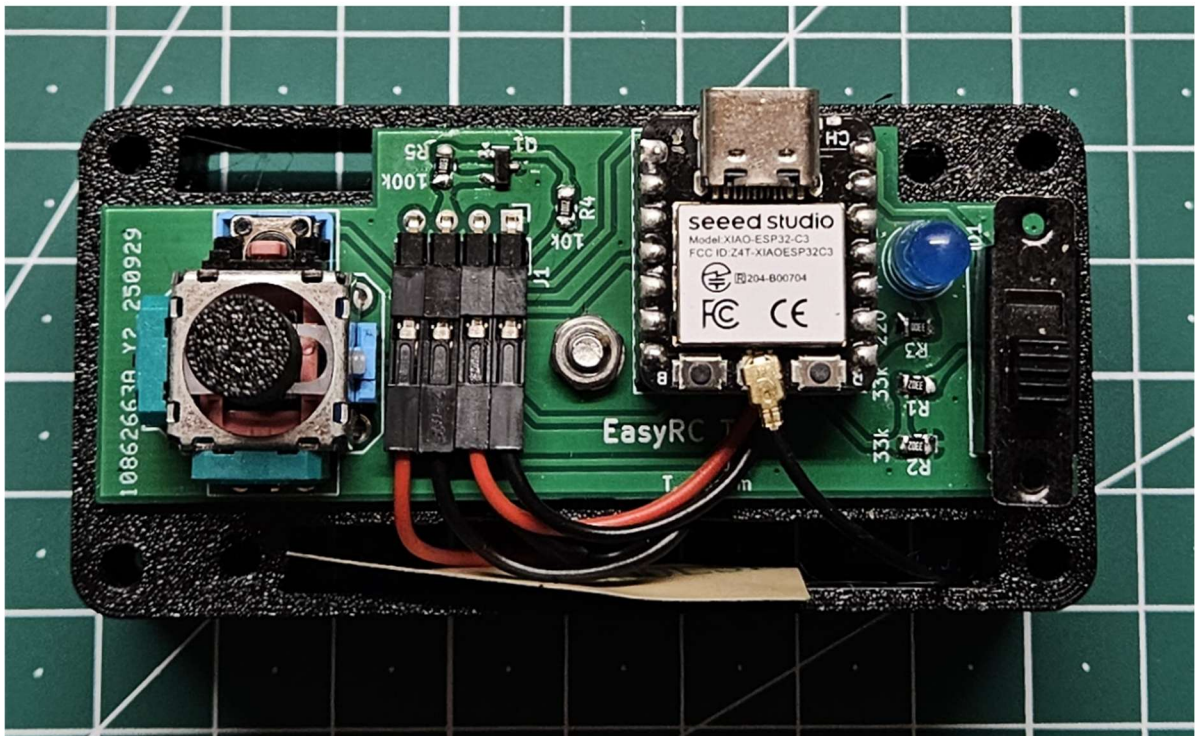
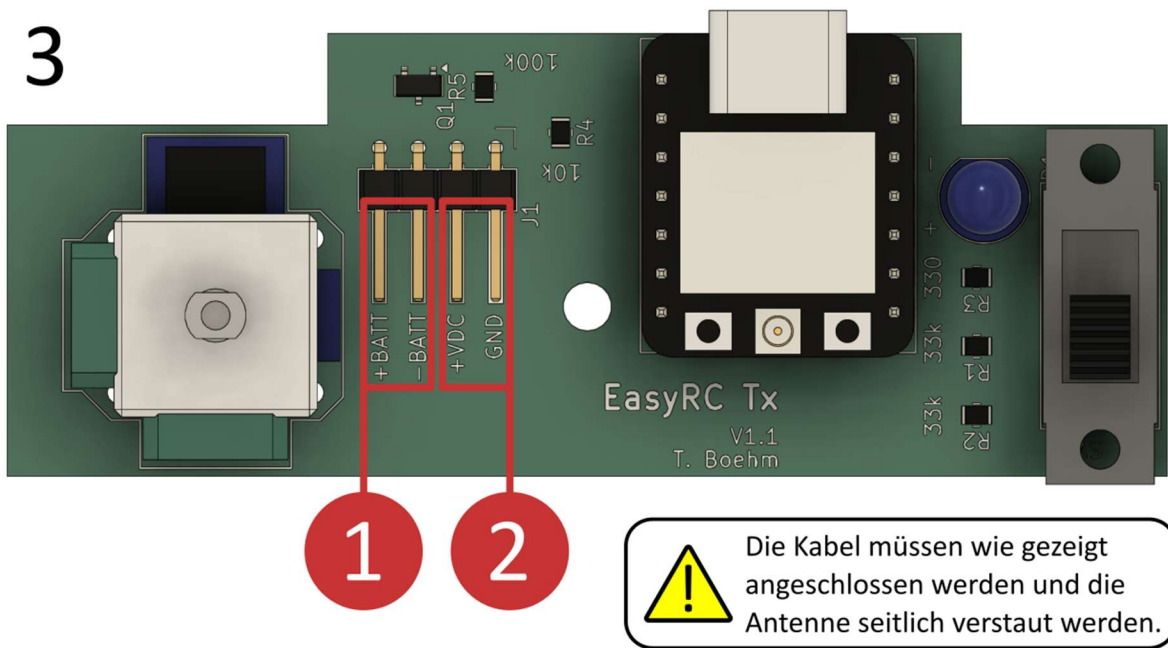
Benötigte Bauteile: Fernbedienung Zwischenrahmen unten, EasyRC Tx Leiterplatte, Batteriehalterung 1x 18650 Lilon Zelle, M3x10 mm Senkkopfschraube, M3 Mutter

1



2



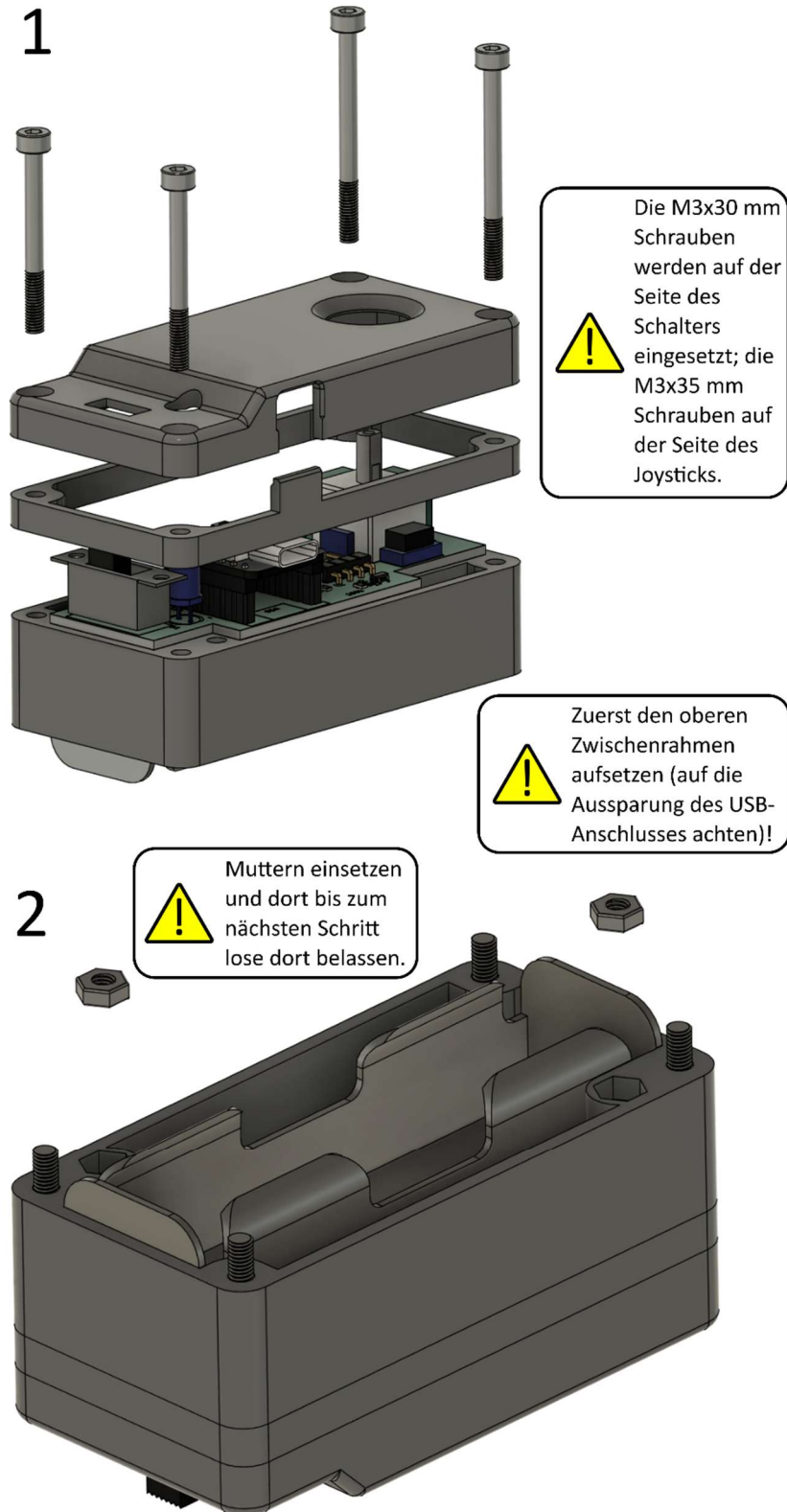


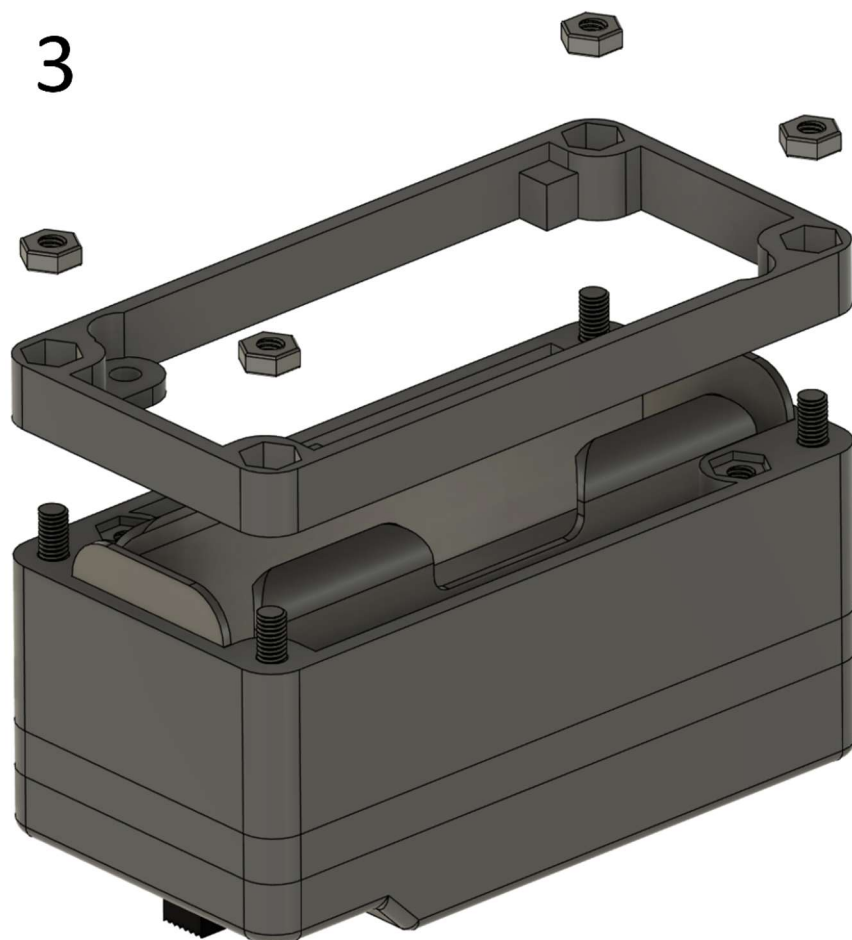
Anschlüsse: Die Batteriehalterung muss an (1) angeschlossen werden; das rote Kabel muss mit +BATT verbunden werden und das schwarze Kabel mit -BATT. Die Kabel vom Microcontroller müssen mit (2) verbunden werden; das rote Kabel wird an +VDC angeschlossen und das schwarze Kabel an GND.

Achtung: Der Batterieanschluss (1) ist verpolungsgeschützt, der Microcontroller aber nicht! Sollte (2) falschherum angeschlossen werden, wird der Microcontroller unmittelbar zerstört, sobald die Fernsteuerung angeschaltet wird!

4.1.2 Befestigung Deckel und Boden

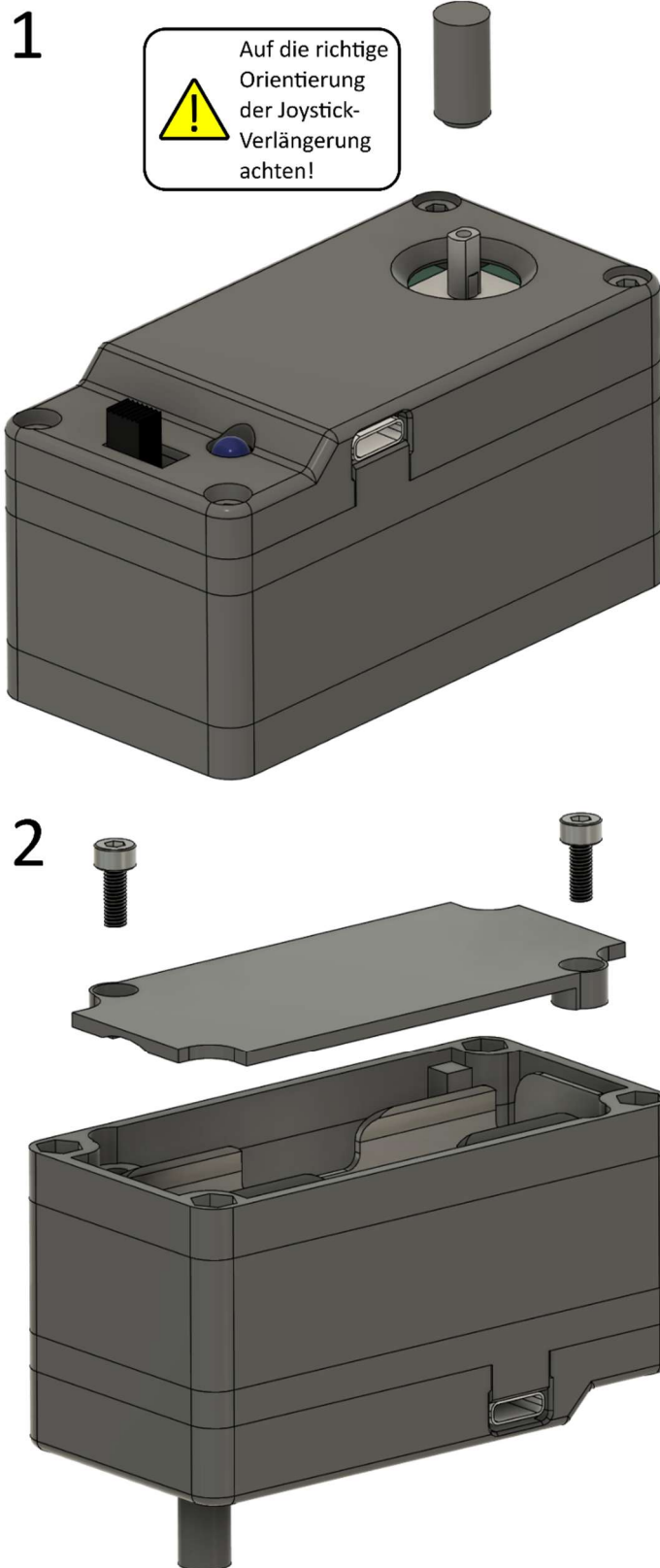
Benötigte Bauteile: Fernbedienung (vorbereitet), Fernbedienung Deckel, Fernbedienung Zwischenrahmen oben, Fernbedienung Boden, M3x30 mm Schraube (2x), M3x35 mm Schraube (2x), M3 Mutter (6x)



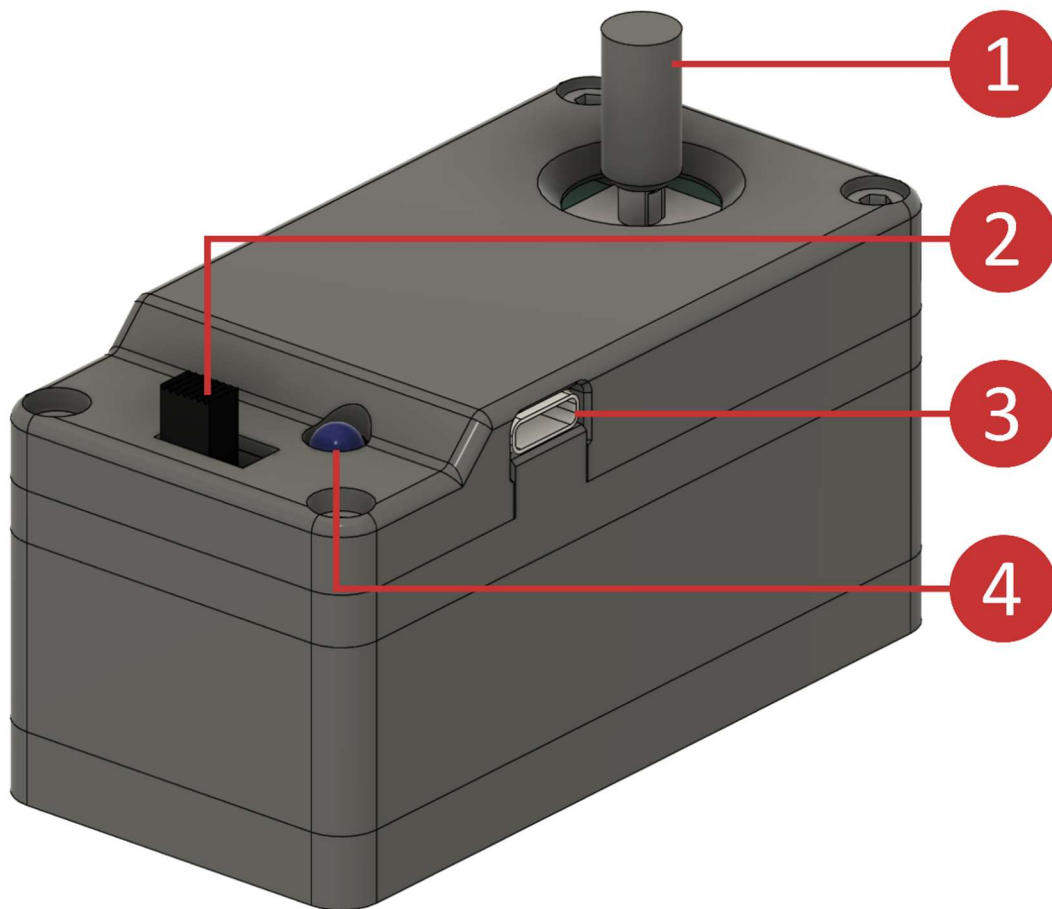


4.1.3 Abschluss Fernbedienung

Benötigte Bauteile: Fernbedienung (vorbereitet), Fernbedienung Joystick-Verlängerung, Fernbedienung Batterieabdeckung, M3x8 mm Schraube (2x)



4.1.4 Übersicht Fernbedienung



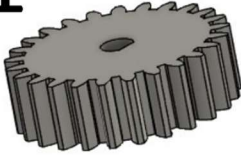
Zahl	Bezeichnung	Erklärung
1	Joystick	Joystick zur Steuerung des Fahrzeugs (vorwärts/rückwärts/links/rechts). Drücken des Joysticks löst die Hupe des Fahrzeugs aus.
2	An/Aus-Schieber	Position links: aus; Position rechts: an
3	USB-Anschluss	USB-Anschluss Typ C
4	Status-LED	Leuchtet durchgehend, wenn die Fernbedienung angeschaltet ist. Blinkt 2x pro Sekunde, wenn die Batterie schwach ist.

4.2 Antriebsstrang und Hinterräder

4.2.1 Vorbereitung Motor

Benötigte Bauteile: DC-Getriebemotor, Motorritzel

1



Das Motorritzel muss
entsprechend der D-Form
der Motorwelle gedreht sein!

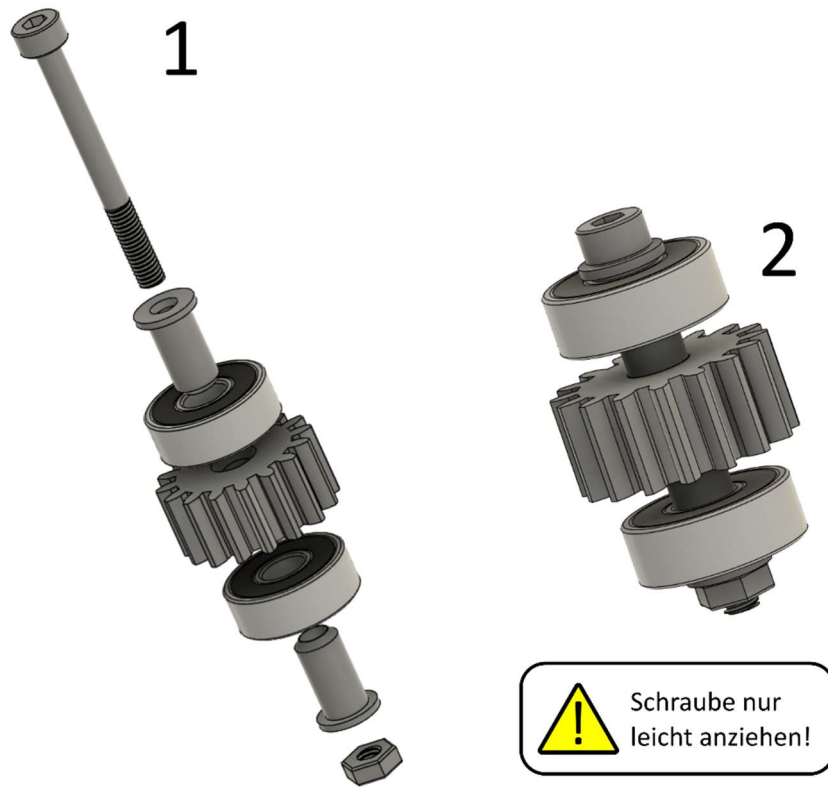
2



Das Motorritzel
muss bündig mit
der Motorwelle
abschließen!

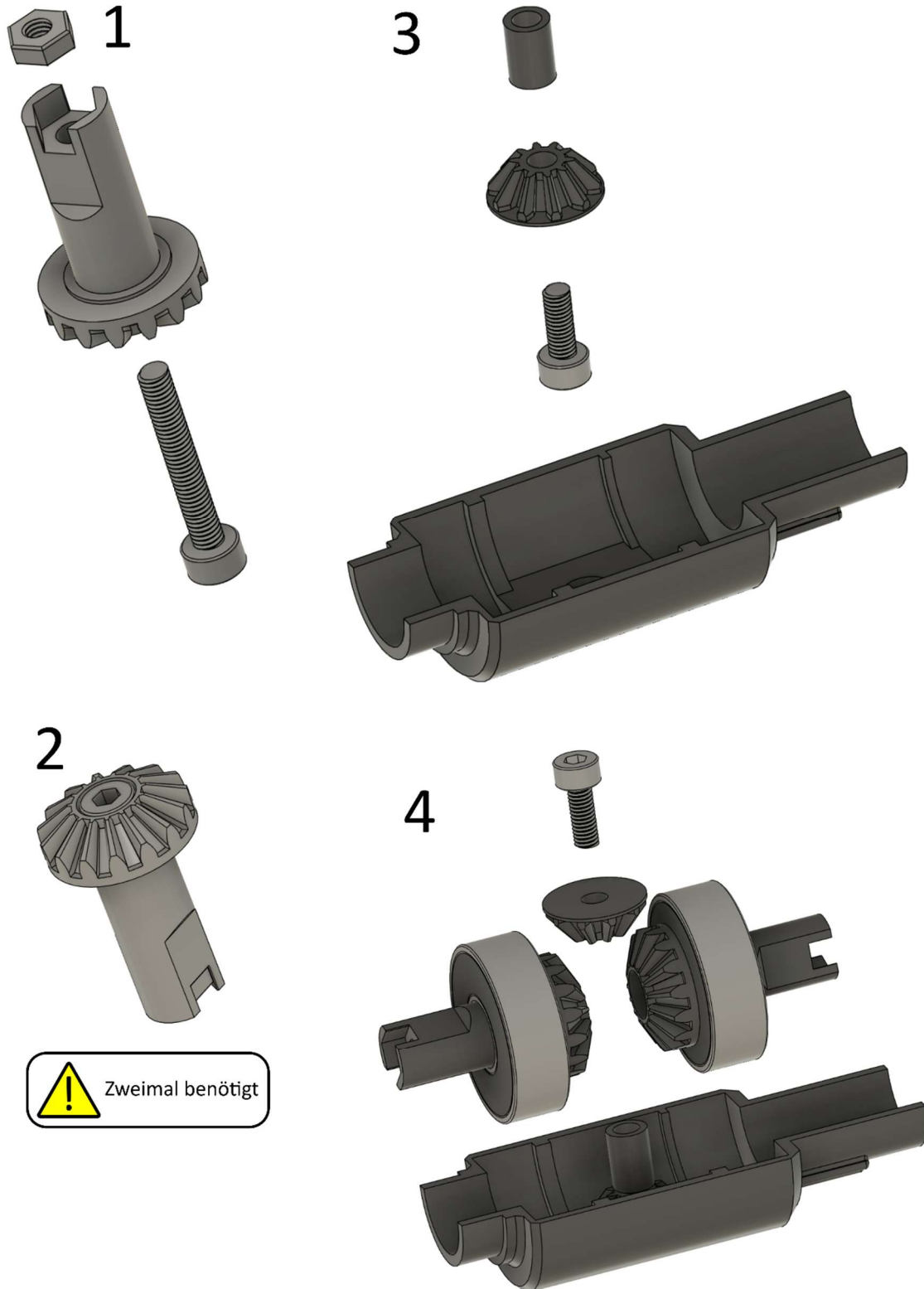
4.2.2 Zwischenzahnrad

Benötigte Bauteile: Zwischenzahnrad mit Lageraufnahme (2x), Kugellager 605 (5x14x5 mm, 2x), M3x30 mm Schraube, M3 Mutter



4.2.3 Differential

Benötigte Bauteile: Differential-Körper (2x), Kronenzahnräder (10 Zähne) mit Abstandshalter (2x), Kronenzahnrad (15 Zähne, 2x), Differentialzahnrad (24 Zähne) mit Abstandshalter (2x), Kugellager 608 (8x22x7 mm, 2x), Kugellager 6802 (15x24x5 mm, 2x), M3x8 mm Schraube (2x), M3x20 mm Schraube (2x), M3 Mutter (2x)

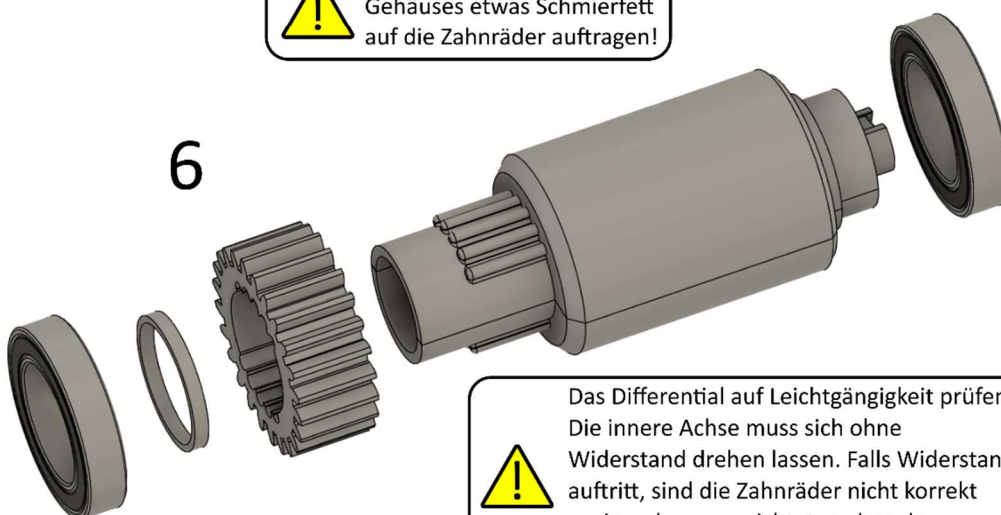


5



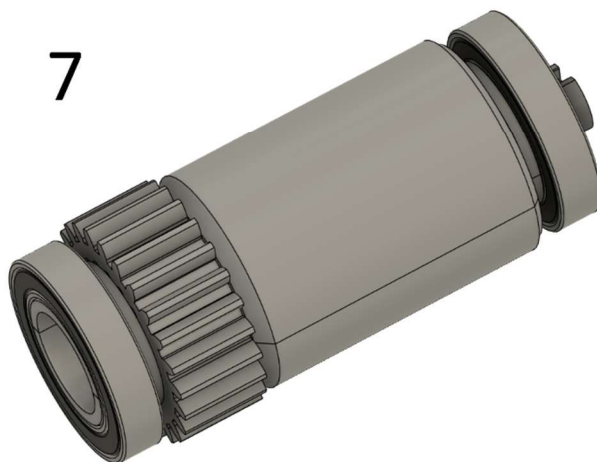
Vor dem Verschließen des Gehäuses etwas Schmierfett auf die Zahnräder auftragen!

6



Das Differential auf Leichtgängigkeit prüfen: Die innere Achse muss sich ohne Widerstand drehen lassen. Falls Widerstand auftritt, sind die Zahnräder nicht korrekt zueinander ausgerichtet, sodass deren Anordnung korrigiert werden muss.

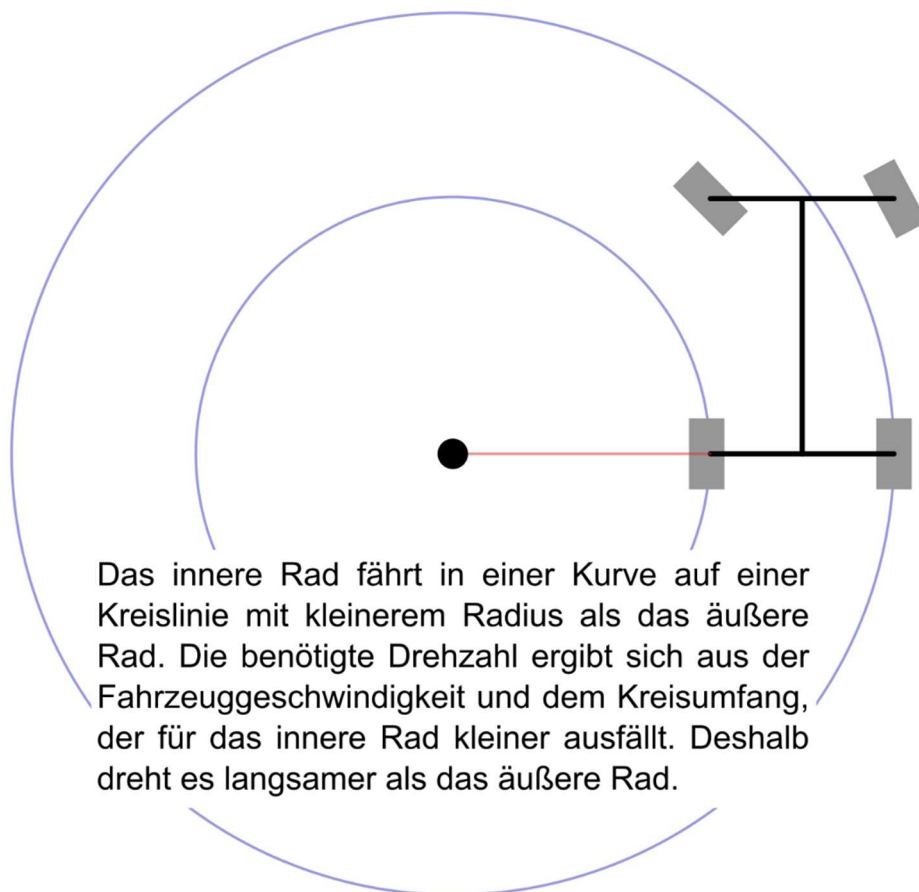
7



Schon gewusst?

Ein Differential ist in jedem Auto verbaut. Ein Differential wird benötigt, um bei einer angetriebenen Achse unterschiedlich schnelle Drehzahlen der Räder zu ermöglichen. Das ist für Kurvenfahrten wichtig: In einer Kurve muss das innere Rad langsamer als das äußere Rad drehen können, damit die Haftung nicht verloren geht. Fahrzeuge mit Hinter- oder Vorderradantrieb benötigen je nur ein Differential zwischen dem linken und rechten angetriebenen Rad. Autos mit Allradantrieb benötigen sogar drei Differenziale (vorn und hinten zwischen den linken und rechten Rädern, sowie ein Mitteldifferential zwischen der vorderen und hinteren Achse).

Der Nachteil eines Differentials wird deutlich, wenn ein Rad die Bodenhaftung verliert. Dann dreht dieses Rad frei, während das andere Rad stillsteht – das Auto steckt fest. Für Straßenfahrzeuge spielt das keine Rolle, da dieser Fall quasi nie auftritt. Für Geländefahrzeuge ist das hingegen ein häufiges Problem. Dafür gibt es Spezialdifferenziale, die selbstsperrend wirken, oder manuell zu betätigende Differentialsperren: Die Differentialfunktion wird mechanisch ausgehebelt, sodass beide angetriebenen Räder gleich schnell drehen müssen. Die Differentialsperre darf aber nur im Gelände aktiviert werden, da im normalen Straßenverkehr zusätzlicher Reifenabrieb, schlechtere Kurvenhaftung und eine unnötig starke Beanspruchung des Antriebsstrangs auftreten.



4.2.4 Hinterräder

Benötigte Bauteile: Hinterradfelge (2x), Reifen (2x), Spacer Hinterradfelge (2x), Linke Hinterradachse, Rechte Hinterradachse, Kugellager 608 (8x22x7 mm, 2x), M3x30 mm Schraube (2x), M3 Mutter (2x)

1



Stege und
Nuten
zueinander
ausrichten!

2



3



Beim Einsetzen der
Schraube nur
drehen, nicht
drücken, um die
Mutter nicht
herauszudrücken!



Tipp zum Einsetzen der
Mutter: Auf eine lange
Schraube setzen und in die
Vertiefung schieben, danach
die Schraube herausdrehen.



Zweimal benötigt

4

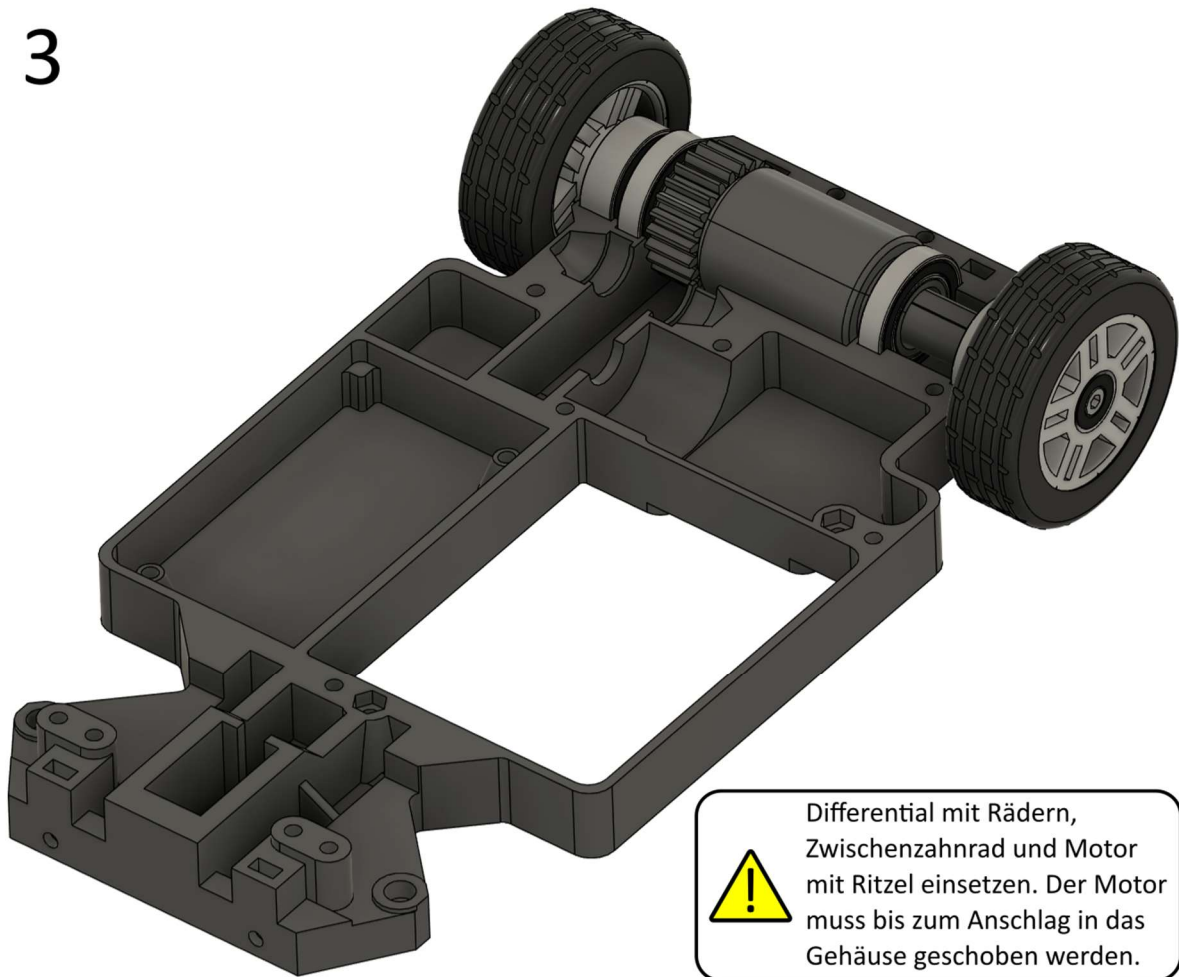


4.2.5 Verbindung Antriebsstrang und Chassis

Benötigte Bauteile: Hinterräder, Differential, Zwischenzahnrad, Motor mit Ritzel (alle bereits vorbereitet), Chassis-Boden, Chassis Deckel, M3x30 mm Schraube (8x), M3 Mutter (14x)

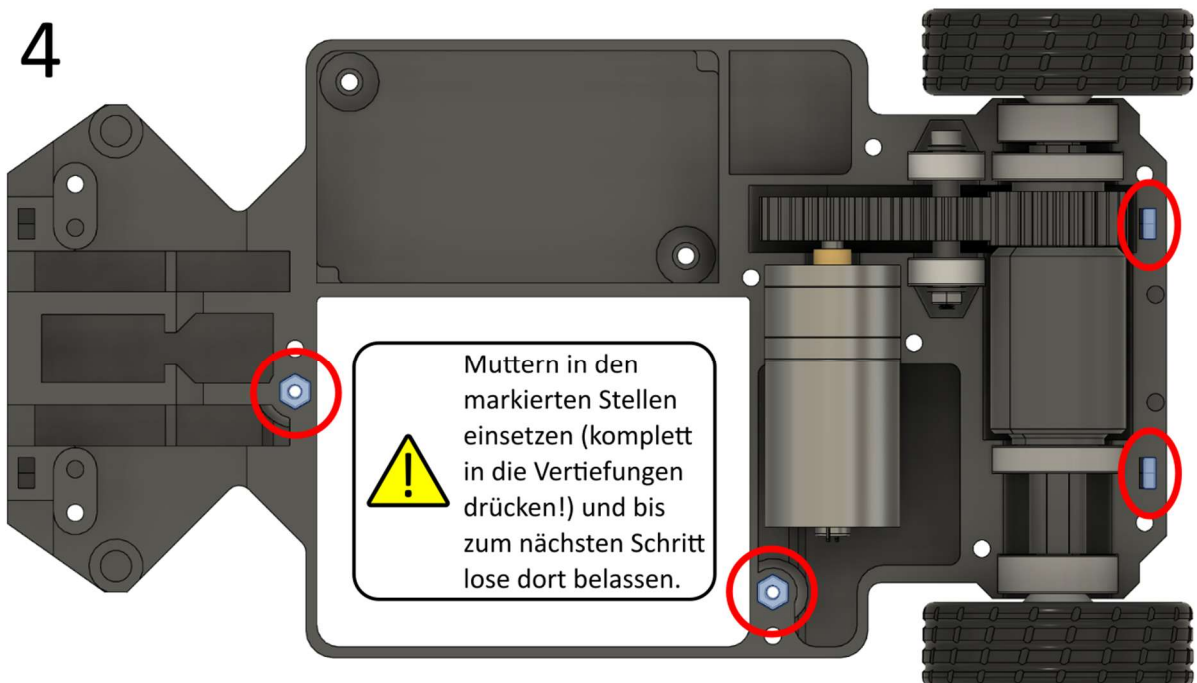


3



! Differential mit Rädern, Zwischenzahnrad und Motor mit Ritzel einsetzen. Der Motor muss bis zum Anschlag in das Gehäuse geschoben werden.

4

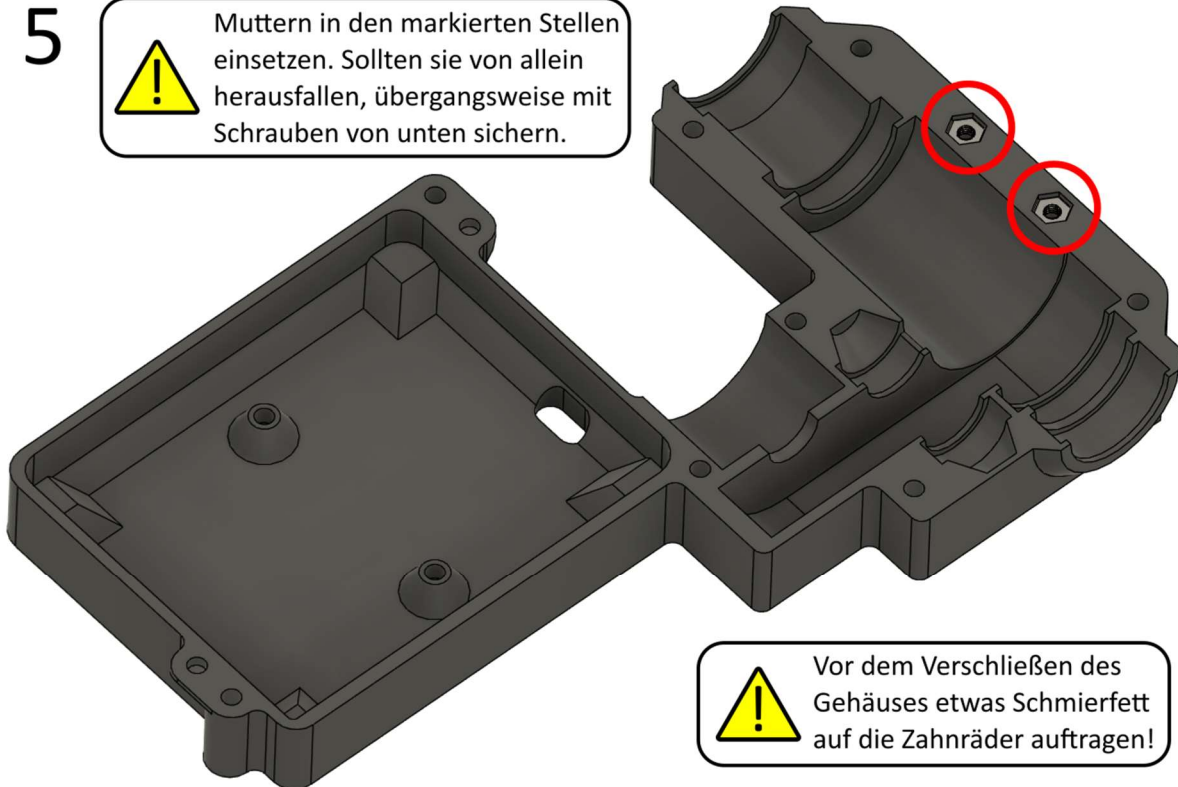


! Muttern in den markierten Stellen einsetzen (komplett in die Vertiefungen drücken!) und bis zum nächsten Schritt lose dort belassen.

5



Muttern in den markierten Stellen einsetzen. Sollten sie von allein herausfallen, übergangsweise mit Schrauben von unten sichern.

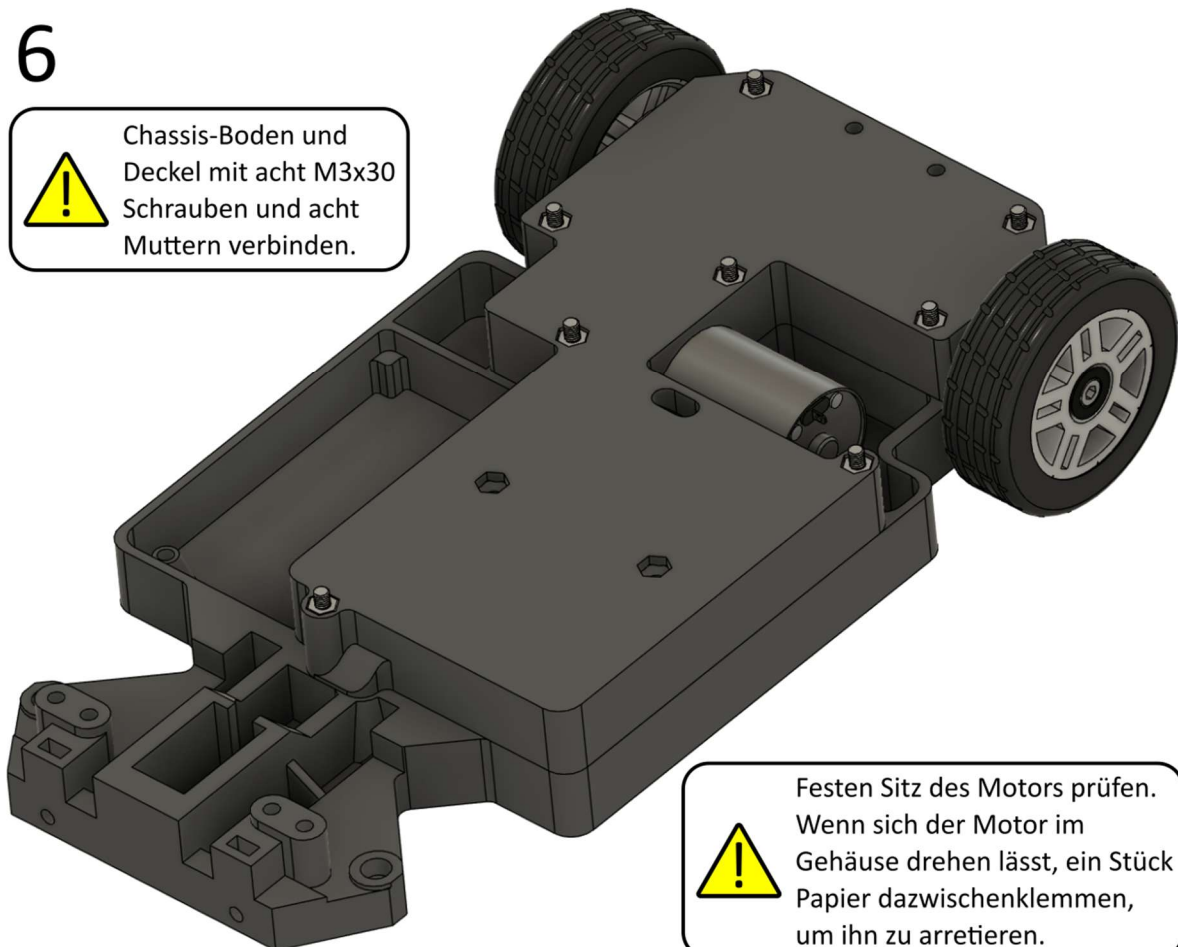


Vor dem Verschließen des Gehäuses etwas Schmierfett auf die Zahnräder auftragen!

6



Chassis-Boden und Deckel mit acht M3x30 Schrauben und acht Muttern verbinden.



Festen Sitz des Motors prüfen. Wenn sich der Motor im Gehäuse drehen lässt, ein Stück Papier dazwischenklemmen, um ihn zu arretieren.

Schon gewusst?

Getriebe sind allgegenwärtig: Nicht nur im Auto, sondern auch in jeder Bohrmaschine, in mechanischen Uhren oder sogar in Salatschleudern sind Getriebe enthalten. Ein Getriebe sorgt dafür, dass die Drehzahl eines mechanischen Antriebs erhöht oder reduziert wird. Im Gegenzug wird das Drehmoment (die Kraft) kleiner oder größer, da sich die Leistung eines mechanischen Antriebs aus Drehmoment $\times$ Drehzahl ergibt. Ein ideales Getriebe hat kaum Reibungsverluste, sodass der Wirkungsgrad meist bei $>90\%$ liegt. Das heißt, dass der Abtrieb eines Motors mit einer 1:10 Übersetzung zwar zehnmal langsamer dreht, aber dafür auch fast zehnmal so stark ist. Die Übersetzung ergibt sich aus dem Verhältnis der Zähnezahls des Zahnradpaars.

Im EasyRC-Modell ist ein Getriebe bereits im Motor integriert, weil die gewünschte 1:10-Übersetzung mit gedruckten Bauteilen nur über ein mehrstufiges Getriebe erreicht werden kann, was viel Platz kosten würde und die Fehleranfälligkeit erhöhen würde. Das gedruckte Getriebe im EasyRC-Modell (25:24-Übersetzung, also annähernd 1:1) dient nur dazu, den Motor weit genug von der Antriebsachse zu entfernen, um Platz für das Differential zu schaffen. Die Anzahl der Zähne des Zwischenzahnradspiels spielt dabei keine Rolle: Entscheidend für die Getriebeübersetzung sind lediglich die Zahnräder der Antriebs- und Abtriebswelle.

4.3 Vorderräder und Lenkung

4.3.1 Vorbereitung Vorderräder

Benötigte Bauteile: Vorderradfelge (2x), Reifen (2x), Vorderrad-Sicherungsring (2x), Vorderrad-Spacer (2x), Vorderradaufhängung (2x, gespiegelte Versionen), Vorderradaufhängung-Spacer (2x), Kugellager 605 (5x14x5 mm, 2x), M3x16 mm Schraube (2x), M3 Mutter (2x)

1

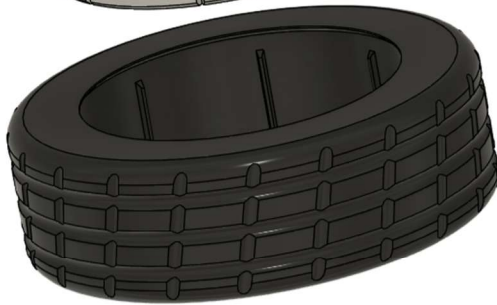
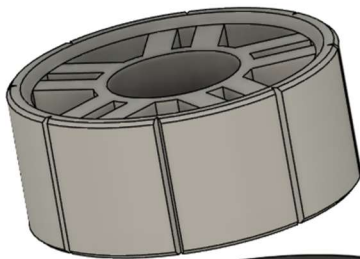


Zweimal benötigt
(spiegelverkehrte
Radaufnahmen)

3



Der Spacer mit
Ausparung für
den Kopf der
Schraube zeigt
nach außen!



2



Der Sicherungsring wird aufgesetzt und mit einem Lötkolben (240°C) mit der Felge verbunden (verschmolzen), indem über die Nahtstelle zur Felge gestrichen wird.

4



4.3.2 Lenkgestänge

Benötigte Bauteile: Vorbereitete Vorderräder, Lenkungsbügel, Lenkgestänge-Verbinder (2x), M3x16 mm Schraube (2x), M3x20 mm Schraube (4x), M3 Mutter (4x)

1



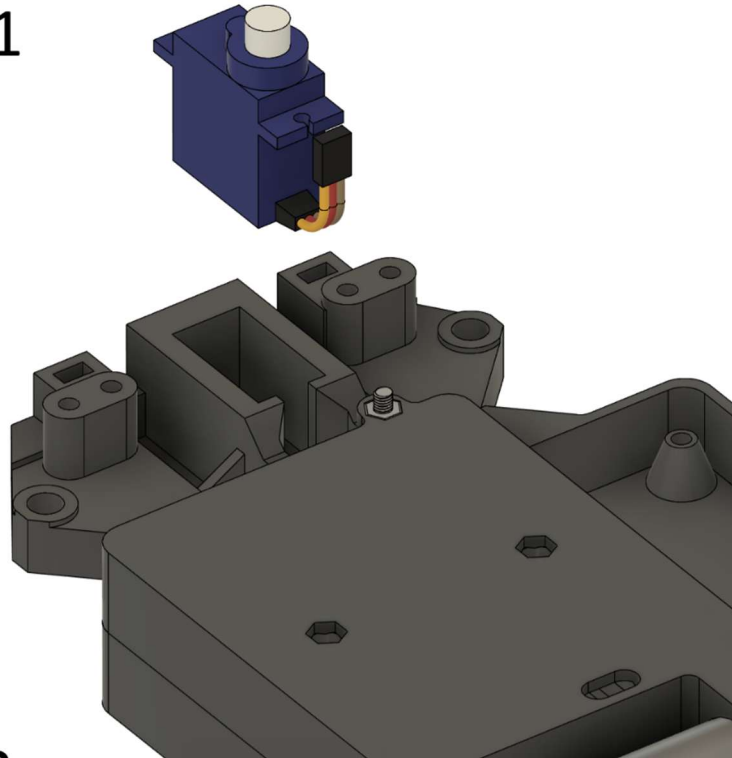
2



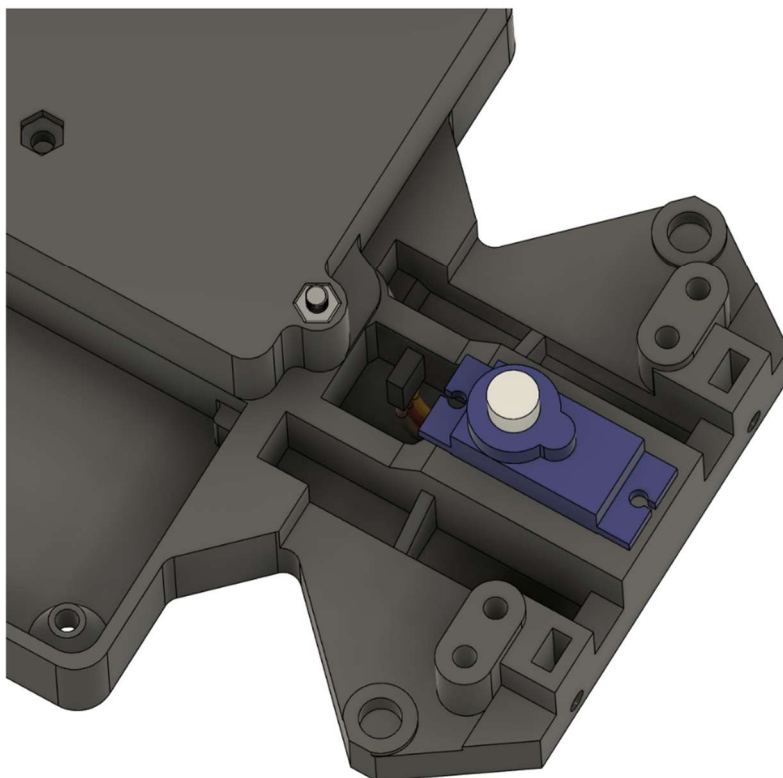
4.3.3 Verbindung von Vorderrädern und Chassis

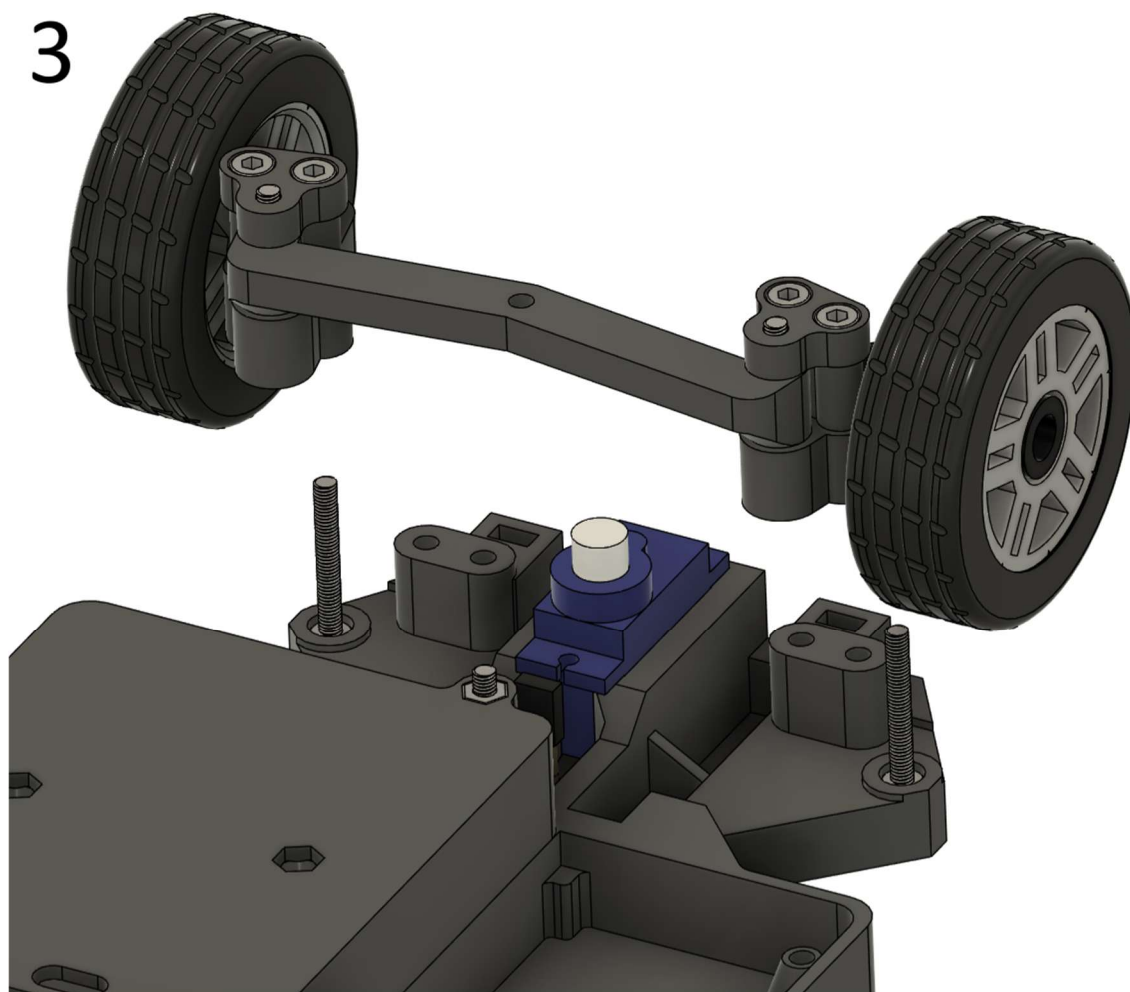
Benötigte Bauteile: Vorbereitetes Chassis, Vorbereitete Vorderräder mit Lenkgestänge, Chassis-Sicherungsbügel, MG90S Servomotor, M3x22 mm Schraube (2x), M3x30 mm Schraube (2x), M3 Mutter (4x)

1

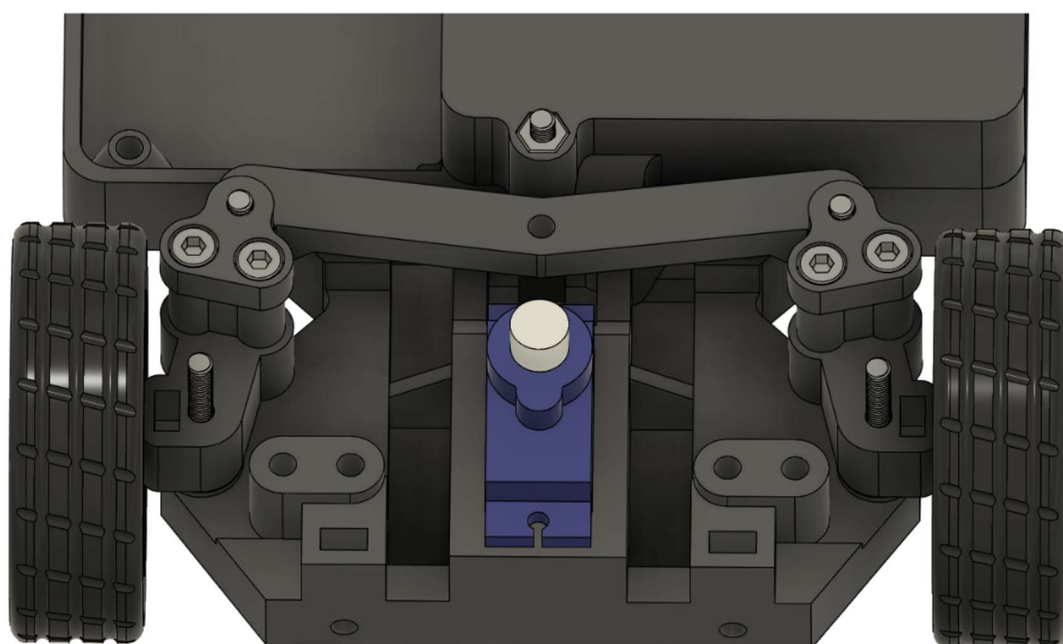


2





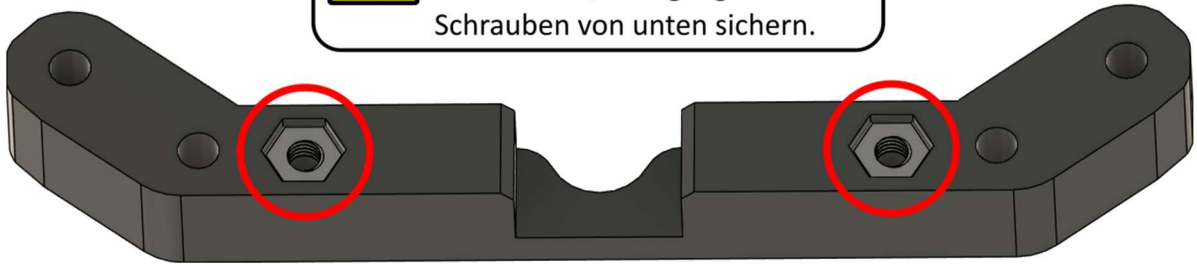
Vorbereitete Vorderräder mit Lenkgestänge auf zwei M3x22 Schrauben aufsetzen. Die Komponenten sitzen aktuell nur lose auf: Bis zum nächsten Schritt in Position halten.



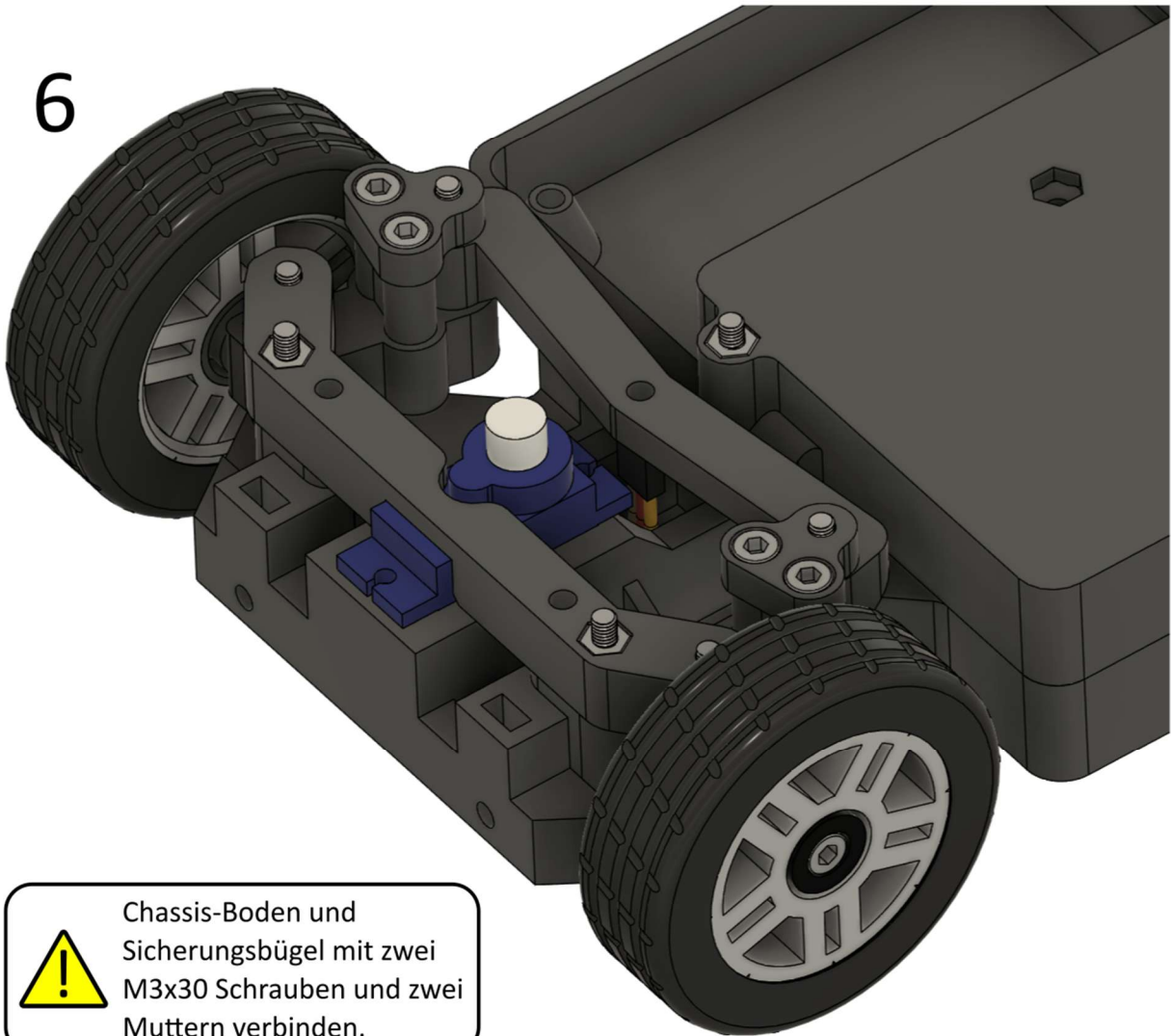
5



Muttern in den markierten Stellen einsetzen. Sollten sie von allein herausfallen, übergangsweise mit Schrauben von unten sichern.



6



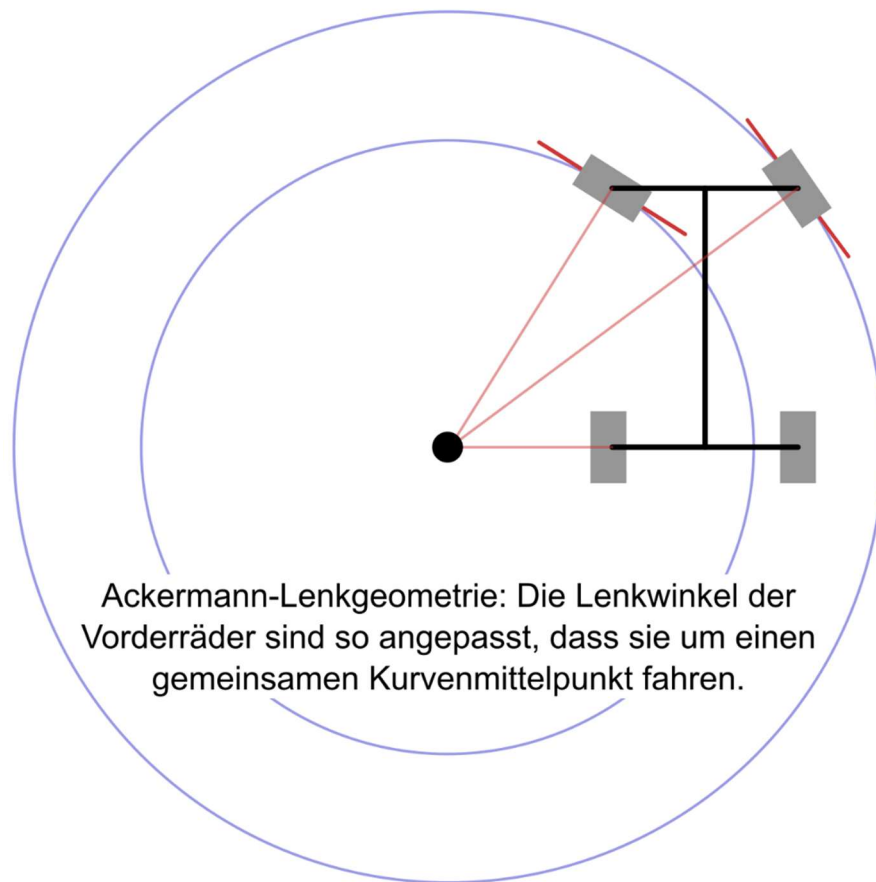
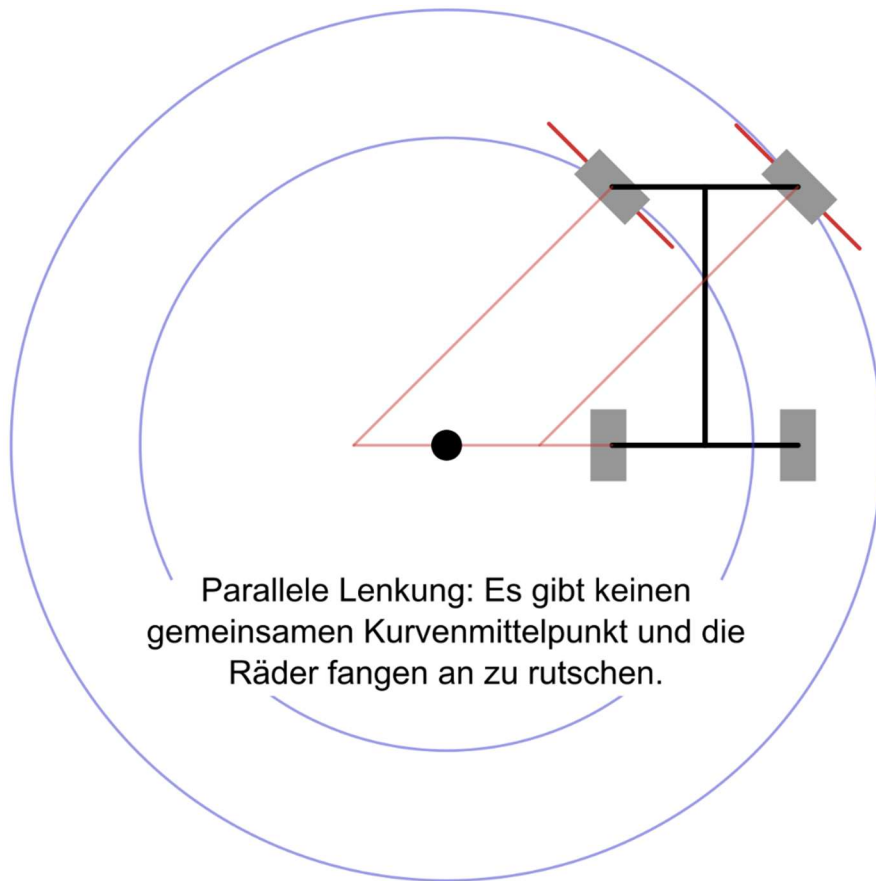
Chassis-Boden und Sicherungsbügel mit zwei M3x30 Schrauben und zwei Muttern verbinden.

Schon gewusst?

Wenn das Lenkrad in einer Kurve eingeschlagen wird, dann bewegt sich das innere Rad weiter als das äußere: Der Lenkungswinkel des inneren Rades ist größer als der des äußeren Rades. Diese Anordnung wird Ackermann-Lenkgeometrie genannt und unterstützt die Haftung des Fahrzeugs in engen Kurven. Da in einem Auto nur die Vorderräder gelenkt werden, muss es in einer Kurve auf einem Kreis fahren, dessen Mittelpunkt eine Verlängerung der Hinterachse ist, damit die Räder nicht anfangen zu rutschen. Das ist auch der Grund, wieso man in engen Parklücken besser rückwärts als vorwärts einparken kann: Der Wendepunkt befindet sich immer auf der Höhe der Hinterachse.

Werden die Vorderräder in dieser Konstellation mit dem exakt gleichen Winkel eingeschlagen, treffen sich ihre Winkel nicht in diesem Mittelpunkt. In der Konsequenz können sie nicht auf einer stabilen Kreisbahn rollen, sondern müssen entgegen ihrer Laufrichtung gezogen werden. Die Ackermann-Lenkgeometrie nähert beide Vorderräder dem idealen Kurvenmittelpunkt an, sodass das Auto in der Kurve so stabil wie möglich fahren kann. Dieses Konzept wurde für Pferdekutschen entwickelt und in Autos übernommen. Allerdings nutzen Autos oftmals keine vollständige Ackermann-Lenkgeometrie, da dieses Konzept vor allem bei langsamem Fahren die Stabilität erhöht. Schnelle Kurvenfahrten sorgen für verschiedene weitere Effekte, wie eine Verlagerung der Fahrzeugmasse auf die kurvenäußeren Räder und die Neigung des Fahrzeugs zur Kurvenaußenseite, was dafür sorgt, dass andere Lenkgeometrien besser funktionieren.

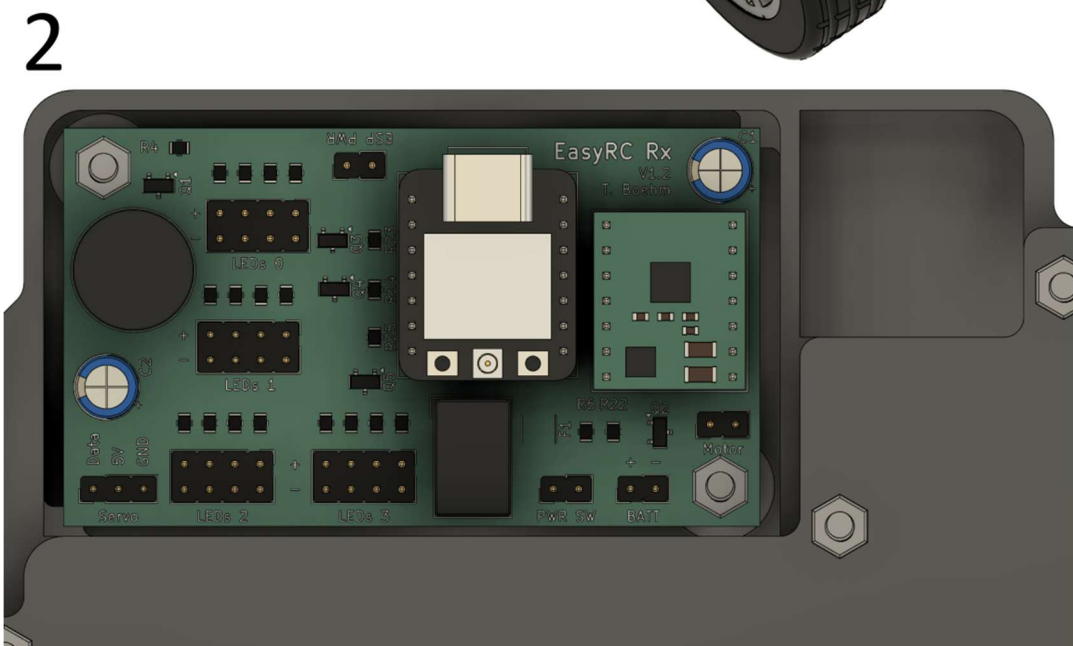
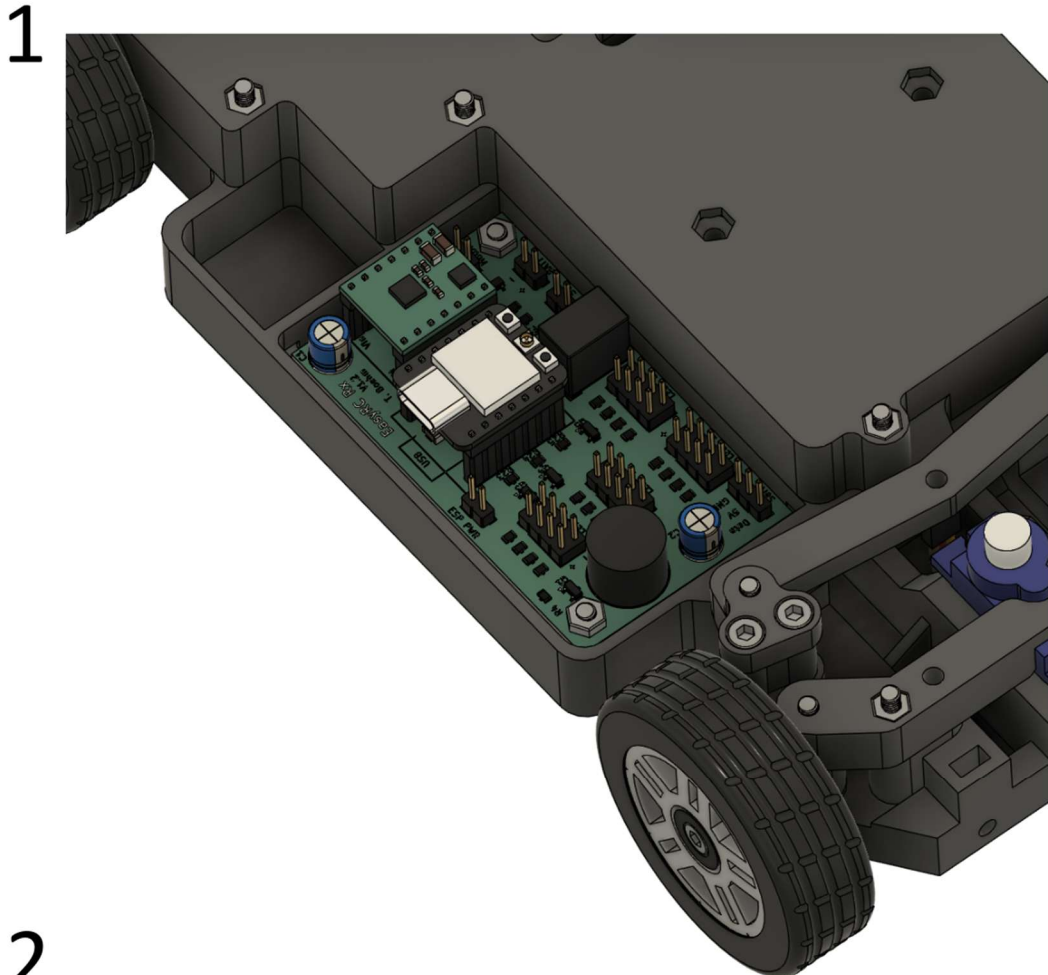
Den Extremfall stellen F1-Autos dar: Sie nutzen sogar anti-Ackermann-Lenkgeometrie, also stärker eingeschlagene kurvenäußere als kurveninnere Räder, um die Reifentemperatur durch zusätzliche Reibung (höheren Schräglaufwinkel) des kurveninneren Rades hochzuhalten. Denn im Hochgeschwindigkeitsbereich ist eine hohe Reifentemperatur notwendig, um beste Traktion zu gewährleisten. Die Nachteile dabei sind ein höherer Reifenabrieb und schlechtere Kurveneigenschaften bei langsamen Fahrten, was für diese Rennfahrzeuge allerdings keine Rolle spielt.



4.4 Finalisierung Chassis

4.4.1 Leiterplatte

Benötigte Bauteile: Vorbereitetes Chassis, EasyRC Rx-Platine, M3x12 mm Schraube (2x), M3 Mutter (2x)



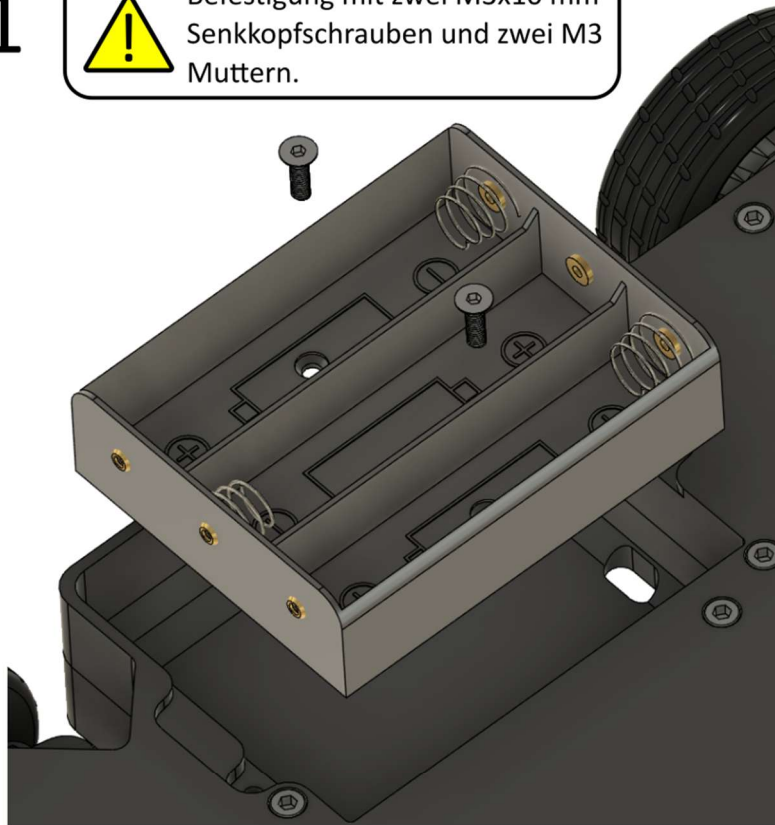
4.4.2 Batteriefach

Benötigte Bauteile: Vorbereitetes Chassis, Batteriefach-Abdeckung, Batteriehalter 3x 18650 Lilon-Zelle, M3x12 mm Schraube (2x), M3x10 mm Senkkopfschraube (2x), M3 Mutter (2x)

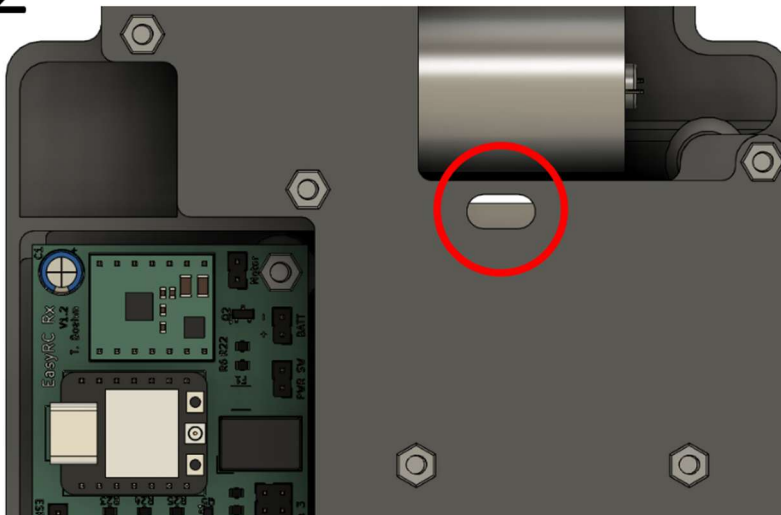
1



Befestigung mit zwei M3x10 mm Senkkopfschrauben und zwei M3 Muttern.



2

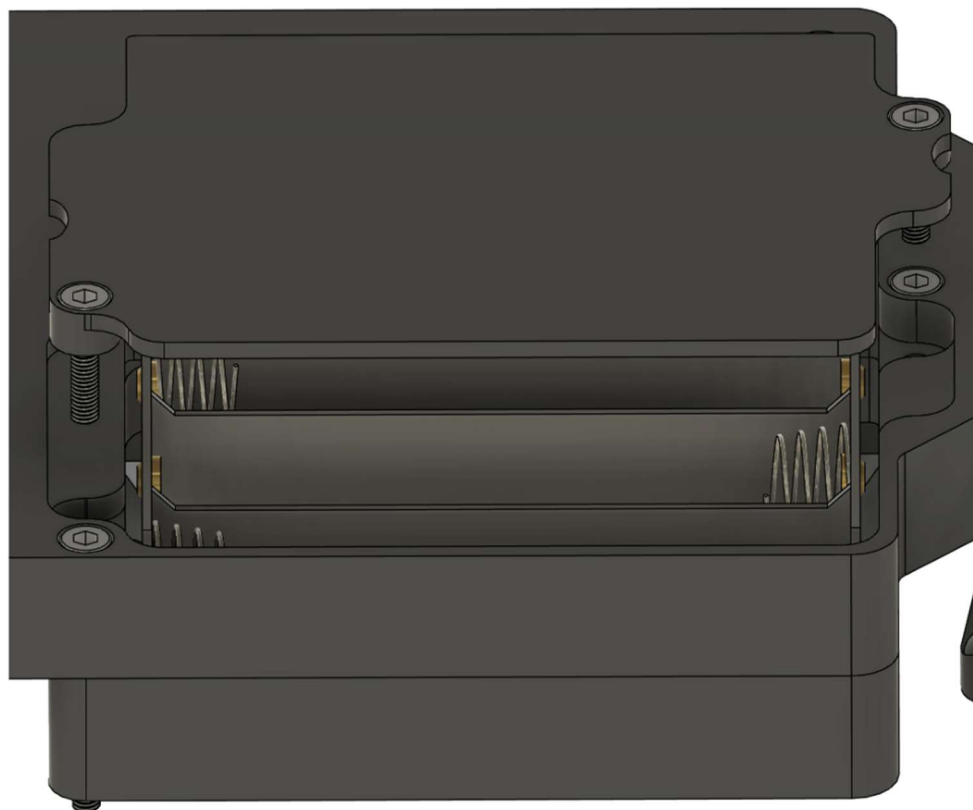


Das Anschlusskabel muss vor dem Verschrauben durch die markierte Öffnung geführt werden.

3



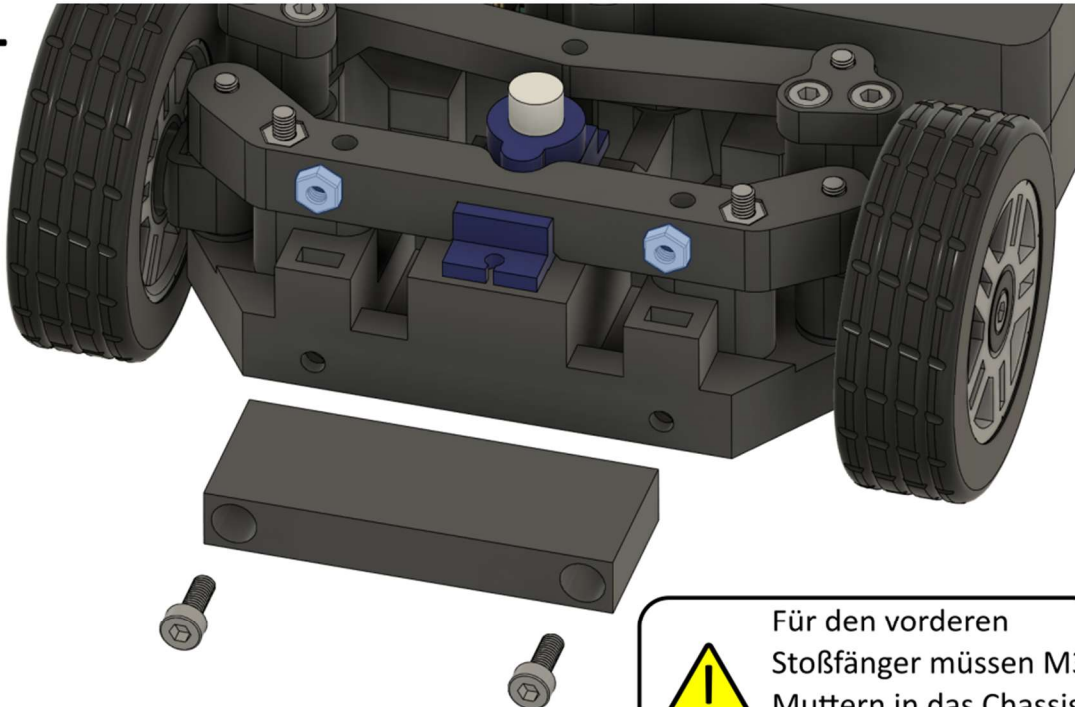
Die Batteriefach-Abdeckung wird mit zwei M3x12 mm Schrauben befestigt.



4.4.3 Stoßfänger

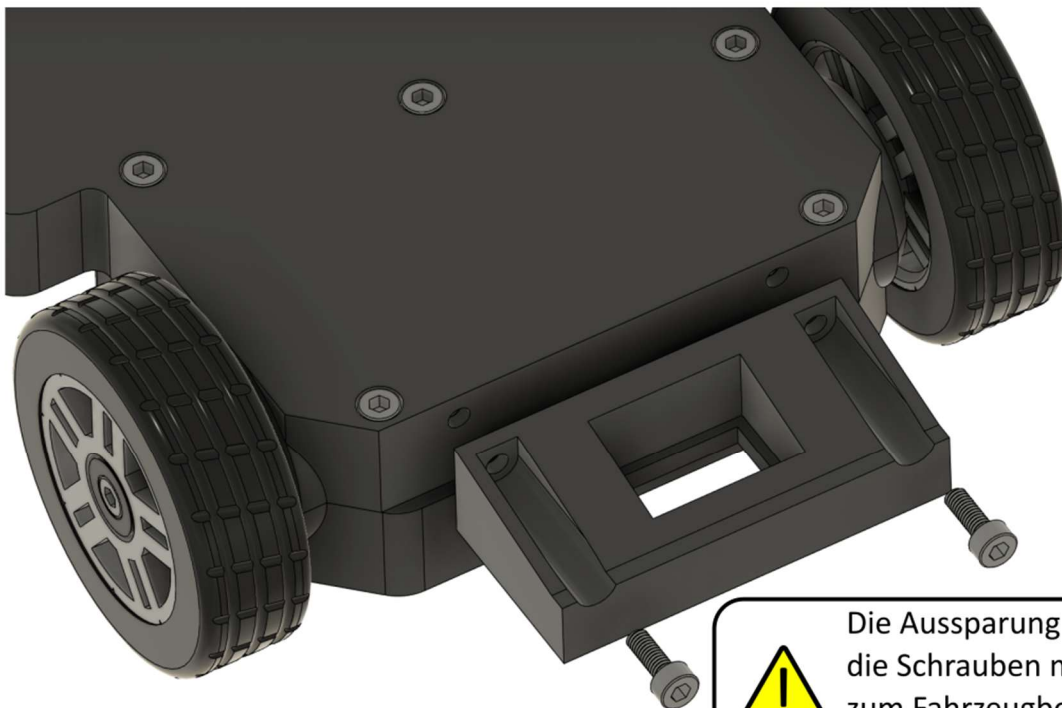
Benötigte Bauteile: Vorbereitetes Chassis, Vorderer Stoßfänger, Hinterer Stoßfänger, An/Aus-Schalter, M3x8 mm Schraube (4x), M3 Mutter (2x)

1



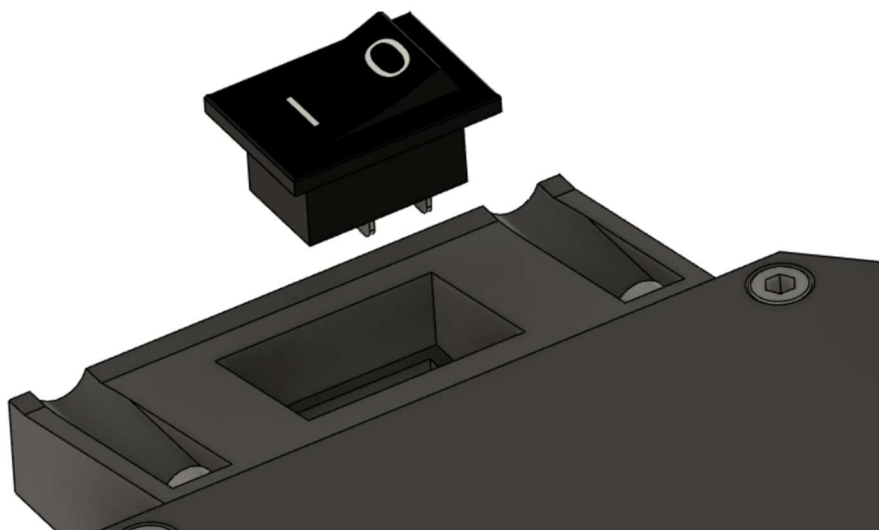
Für den vorderen
Stoßfänger müssen M3
Muttern in das Chassis
eingesetzt werden.

2



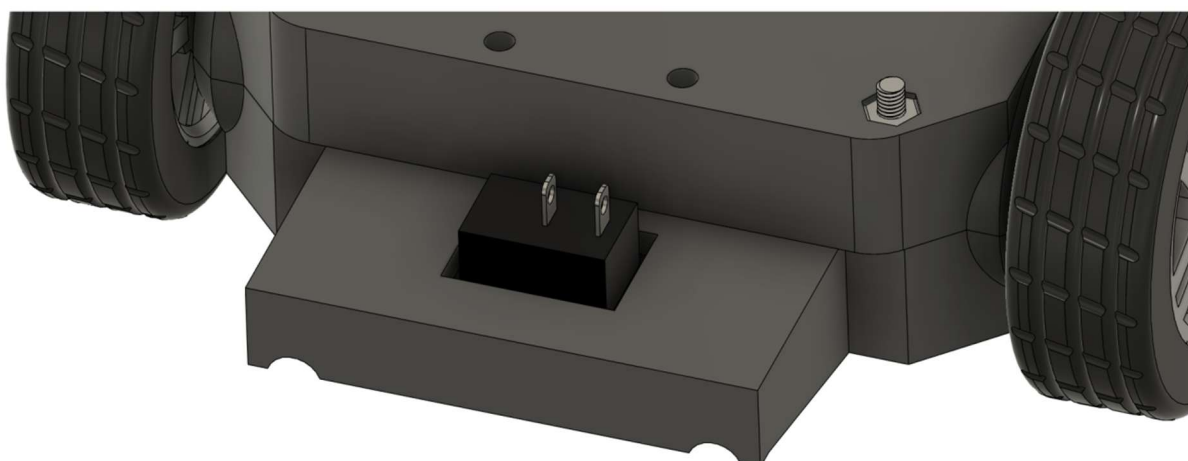
Die Aussparung für
die Schrauben muss
zum Fahrzeugboden
ausgerichtet sein!

3



Der An/Aus-Schalter wird in den hinteren Stoßfänger eingesetzt (mit leichtem Druck, der Schalter schnappt in seine Position).

4

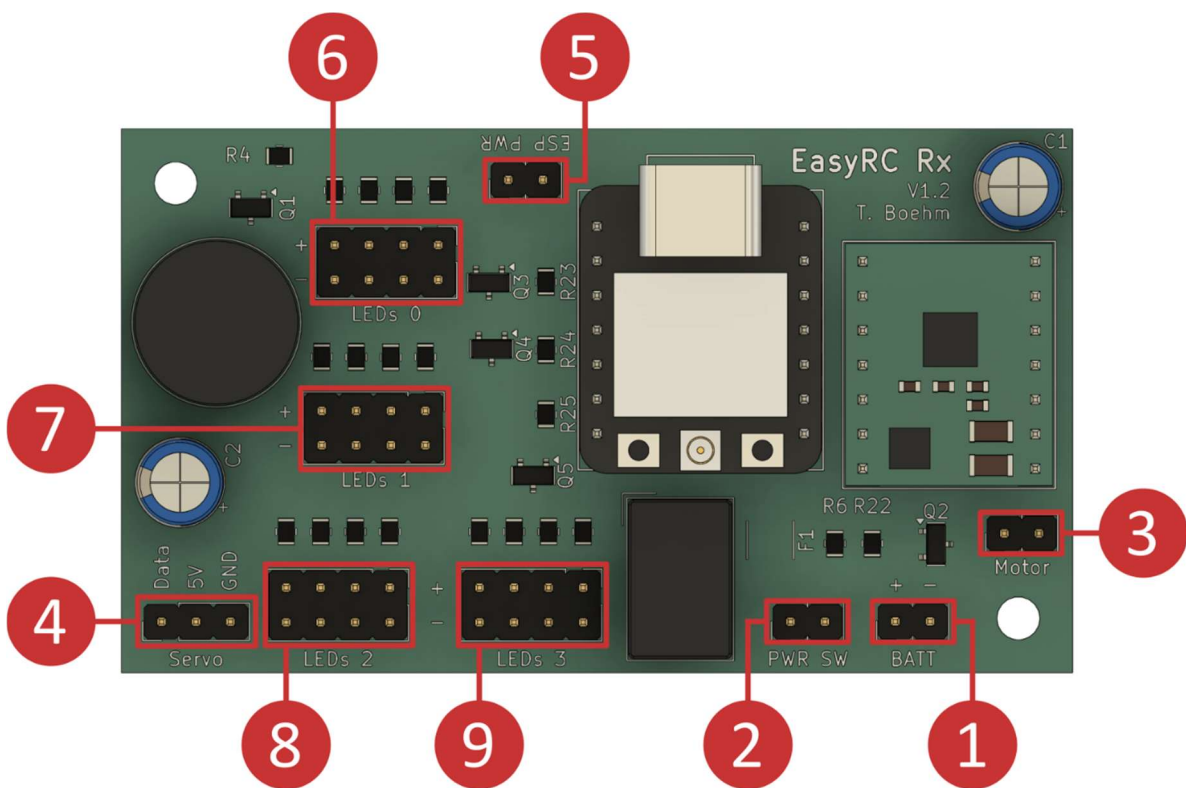


4.4.4 Verkabelung

Die Verkabelung des Chassis beinhaltet die Markierungen 1 bis 4: Batteriefach, An/Aus-Schalter, Antriebsmotor und Lenkung können bereits verkabelt werden. Unbedingt auf korrekte Polarität achten (siehe Tabelle)!

Die Kurzschlussbrücke (Markierung 5) muss vor der Programmierung des Fahrzeugs entfernt werden! Wird der Microcontroller programmiert, während die Kurzschlussbrücke aufgesetzt ist, kann es durch einen zu hohen Stromfluss über den USB-Anschluss zur Platine zu Defekten am PC kommen.

Die LEDs (Markierungen 6 bis 9) werden in Kapitel 5 beim finalen Zusammenbau des Fahrzeugs angeschlossen. Es können bis zu vier LEDs je Kanal angeschlossen werden. Die Standard-Karosserie ist für zwei LEDs je Kanal ausgelegt. Eine LED kann mit einem beliebigen Anschluss eines Kanals verbunden werden (ohne bevorzugte Reihenfolge).



Zahl	Bezeichnung	Polarität
1	BATT (Batterieanschluss)	Rot zu +; Schwarz zu -
2	PWR SW (An/Aus-Schalter)	Egal
3	Motor (Antriebsmotor)	Egal (Drehrichtung wird per Software eingestellt)
4	Servo (Lenkungsmotor)	Gelb zu Data; Rot zu 5V; Braun zu GND
5	ESP PWR (Kurzschlussbrücke)	Egal
6	LEDs 0 (Frontscheinwerfer)*	Rot zu +; Schwarz zu -
7	LEDs 1 (Bremslichter)*	Rot zu +; Schwarz zu -
8	LEDs 2 (Blinker links)*	Rot zu +; Schwarz zu -
9	LEDs 3 (Blinker rechts)*	Rot zu +; Schwarz zu -
*Die LED-Anschlussleisten bestehen aus vier Anschlusspaaren: Es können bis zu vier LEDs je Kanal angeschlossen werden. In dieser Ansicht sind die oberen Kontakte + und die unteren Kontakte -. Die Karosserie ist für zwei LEDs je Kanal ausgelegt.		

4.4.5 Vorbereitung Servohorn

Benötigte Bauteile: Servohorn-Aufsatz, Servohorn, M3x16 mm Schraube, M3 Mutter

1



Der Servohorn-Aufsatz ist in zwei unterschiedlich großen Versionen vorhanden. Die Variante, die der Größe des Servohorns entspricht, muss verwendet werden.

2



Das Servohorn wird erst nach der Programmierung des Fahrzeugs auf den Servomotor gesetzt.

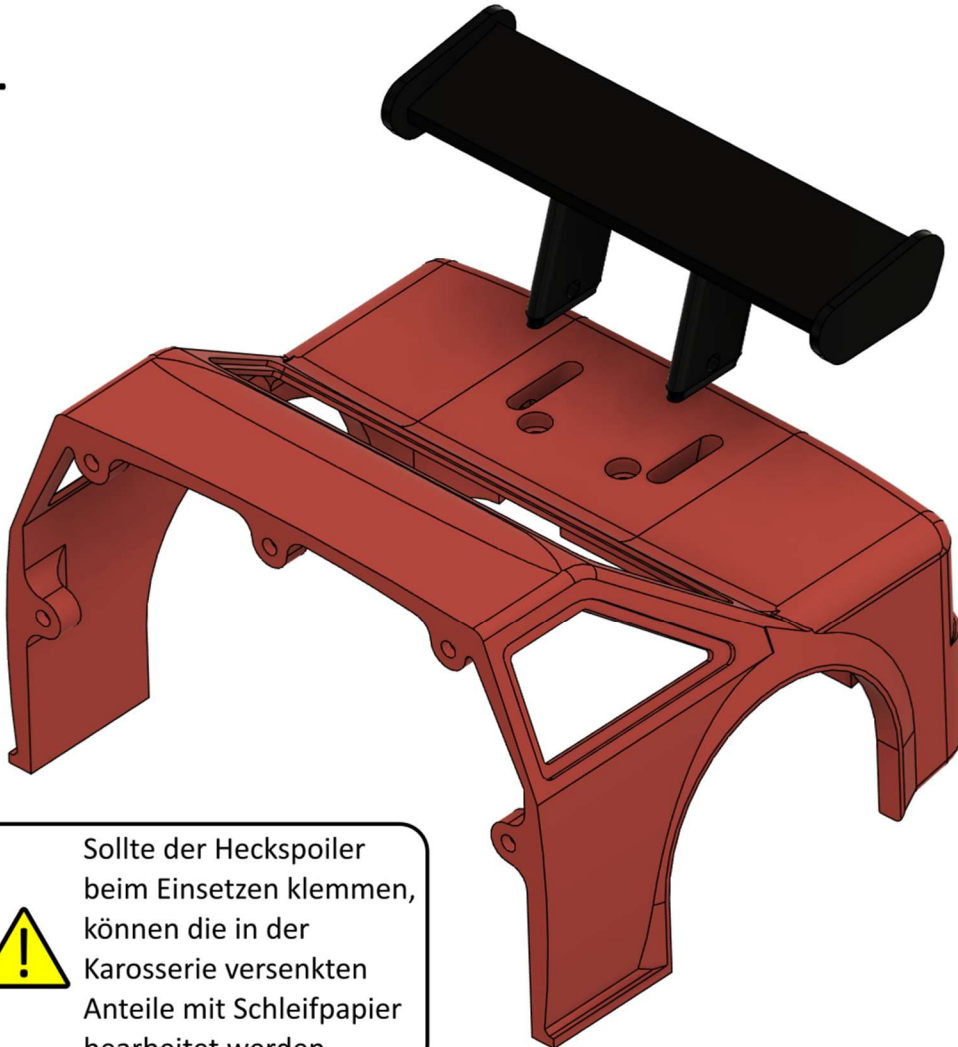
Für den Servohorn-Aufsatz existieren zwei 3D-Modelle, da es die Servohörner der MG90-Servos in verschiedenen Größen gibt. Das kleinere Servohorn hat eine Länge von ca. 32 mm und das größere eine Länge von ca. 37 mm. Es muss der Servohorn-Aufsatz mit der passenden Servohorn-Aussparung gedruckt und eingesetzt werden, um eine spielfreie Lenkung zu ermöglichen.

4.5 Karosserie

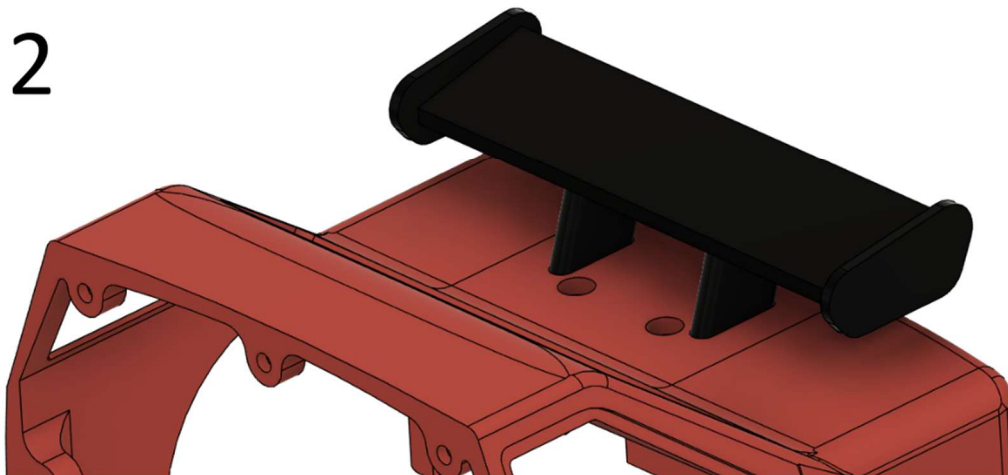
4.5.1 Heckspoiler

Benötigte Bauteile: Karosserie, Heck, Heckspoiler, Heckspoiler-Halterung, M3x8 mm Schraube (2x), M3x16 mm Schraube (2x), M3 Mutter (2x)

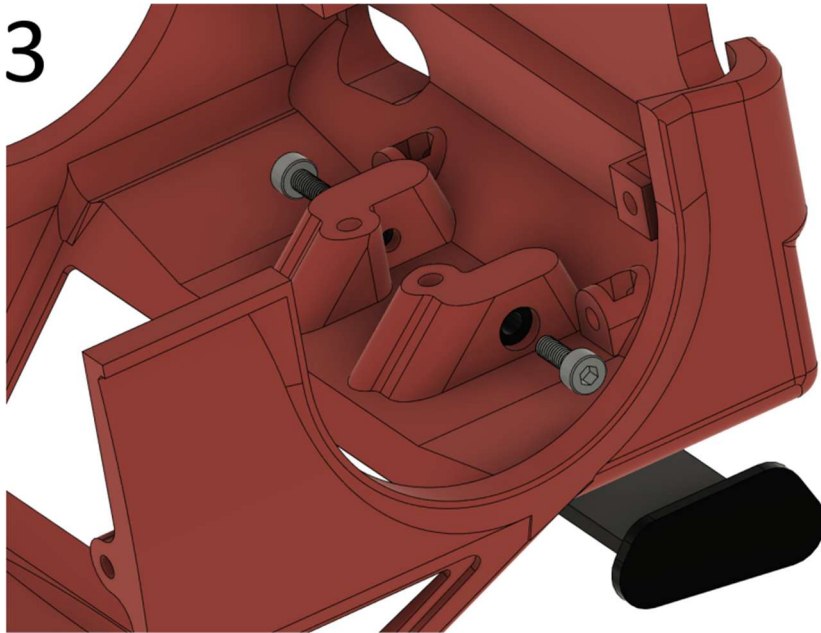
1



2



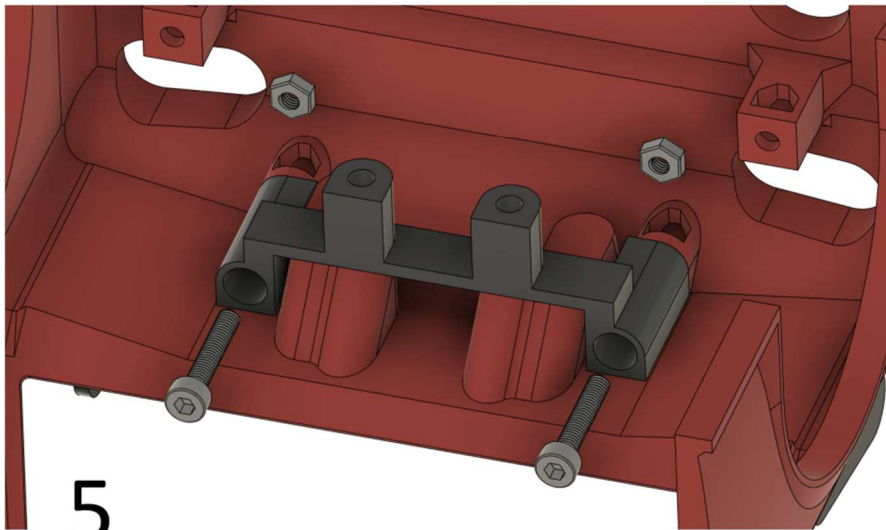
3



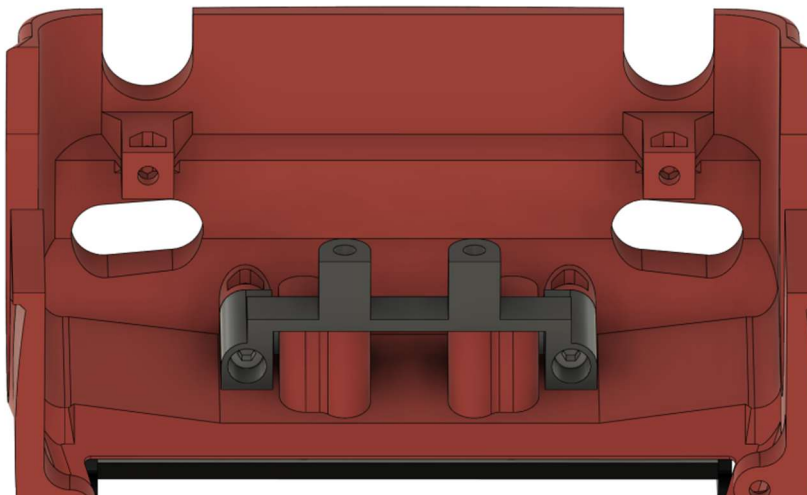
4



Die M3x8 mm Schrauben werden nur eingesteckt und von der Heckspoiler-Halterung fixiert.



5



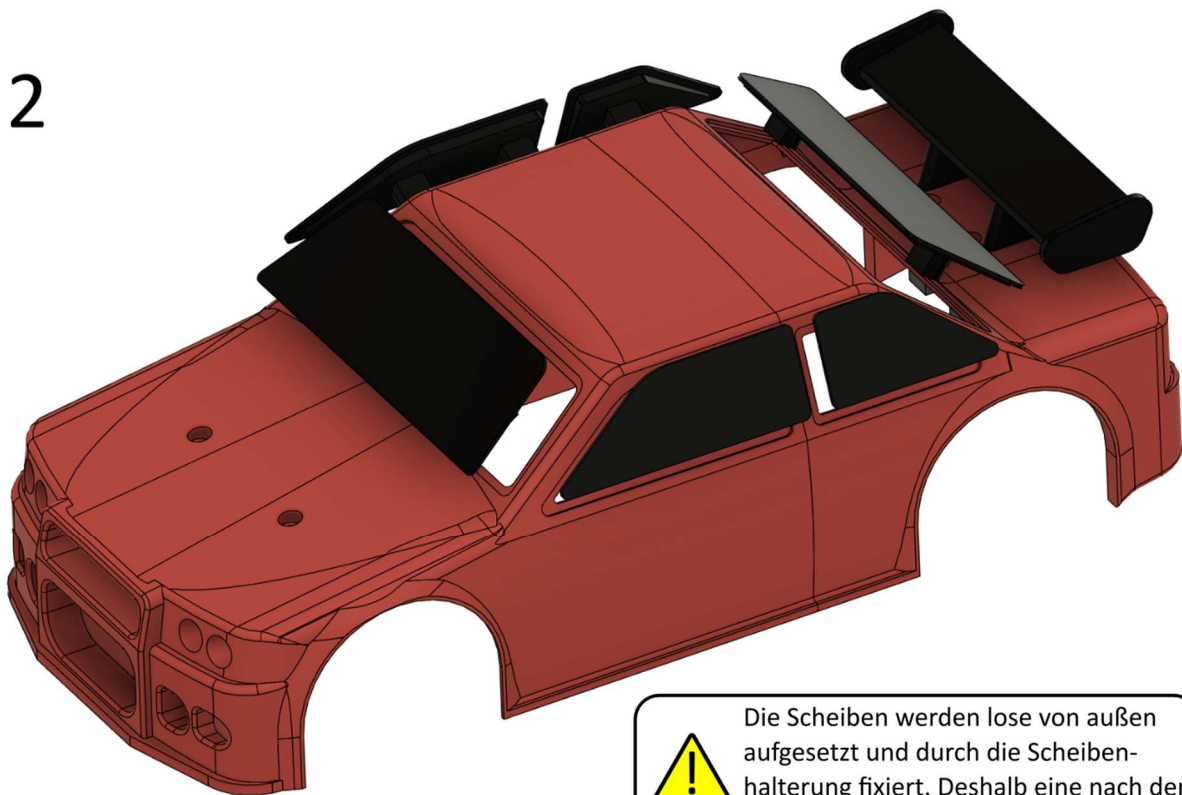
4.5.2 Karosserie-Verbindung und Scheiben

Benötigte Bauteile: Karosserie-Heck (vorbereitet), Karosserie-Front, Scheiben (1x Front, 1x links vorn, 1x links hinten, 1x rechts vorn, 1x rechts hinten, 1x Heck), Scheibenhalterung, M3x8 mm Schraube (15x), M3 Mutter (15x)

1

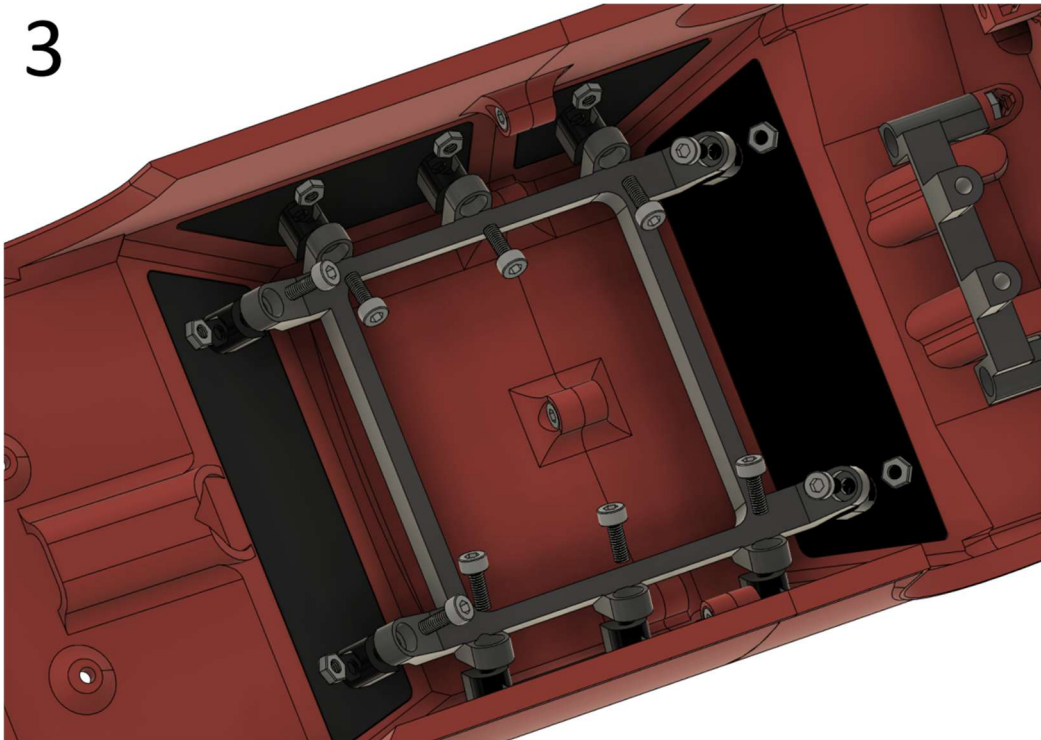


2



Die Scheiben werden lose von außen aufgesetzt und durch die Scheibenhalterung fixiert. Deshalb eine nach der anderen einsetzen und verschrauben.

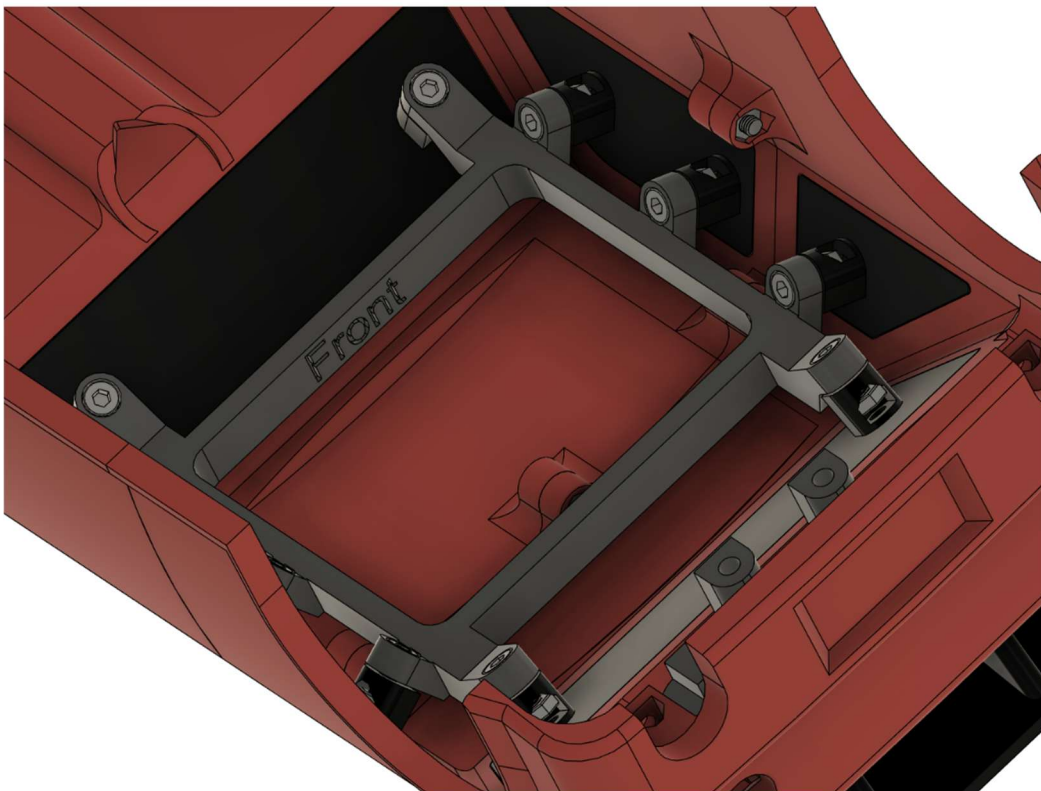
3



4



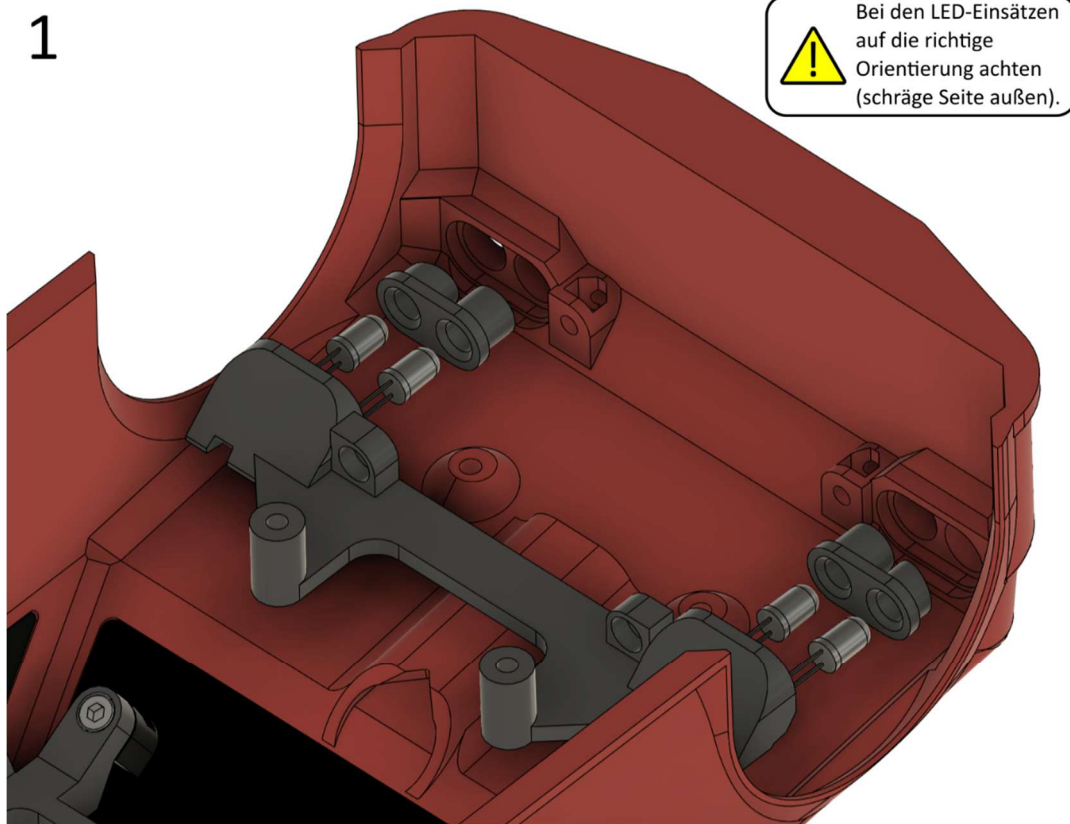
Auf die richtige Orientierung der
Scheibenhalterung achten!



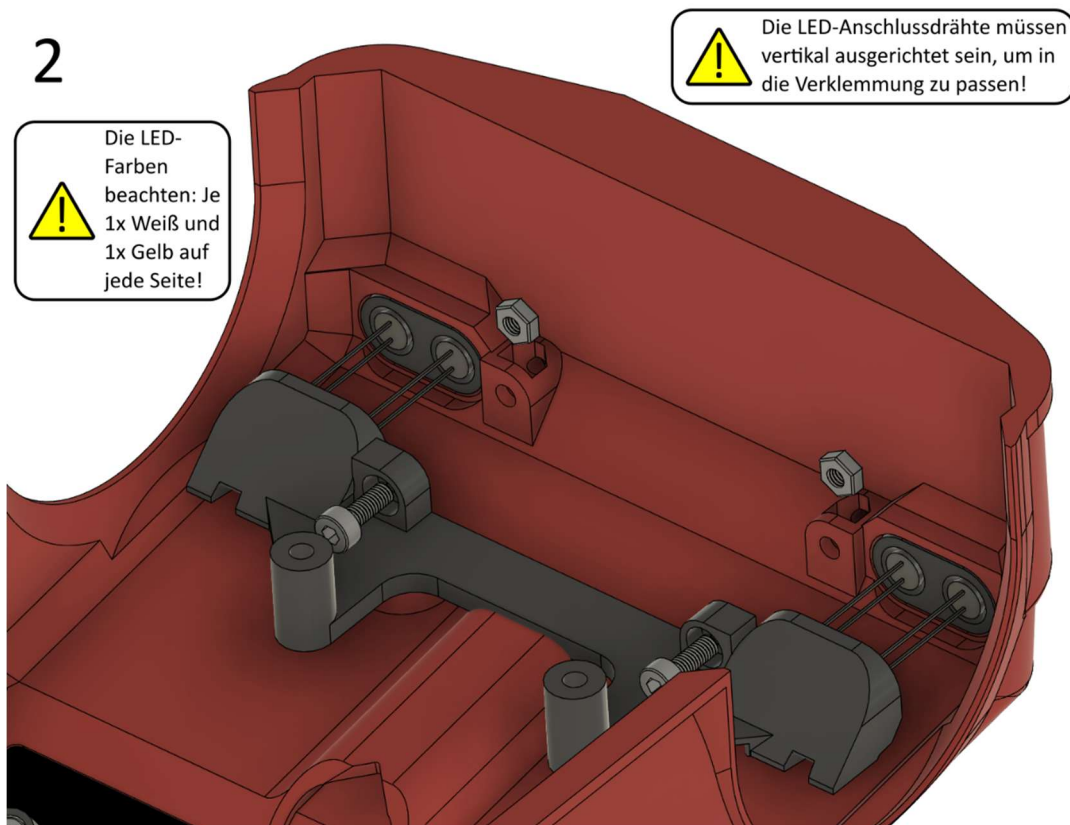
4.5.3 Frontscheinwerfer

Benötigte Bauteile: Karosserie (vorbereitet), Frontlicht-Verklebung, LED-Einsatz vorn (2x), LED weiß (2x), LED gelb (2x), M3x8 mm Schraube (2x), M3 Mutter (2x)

1



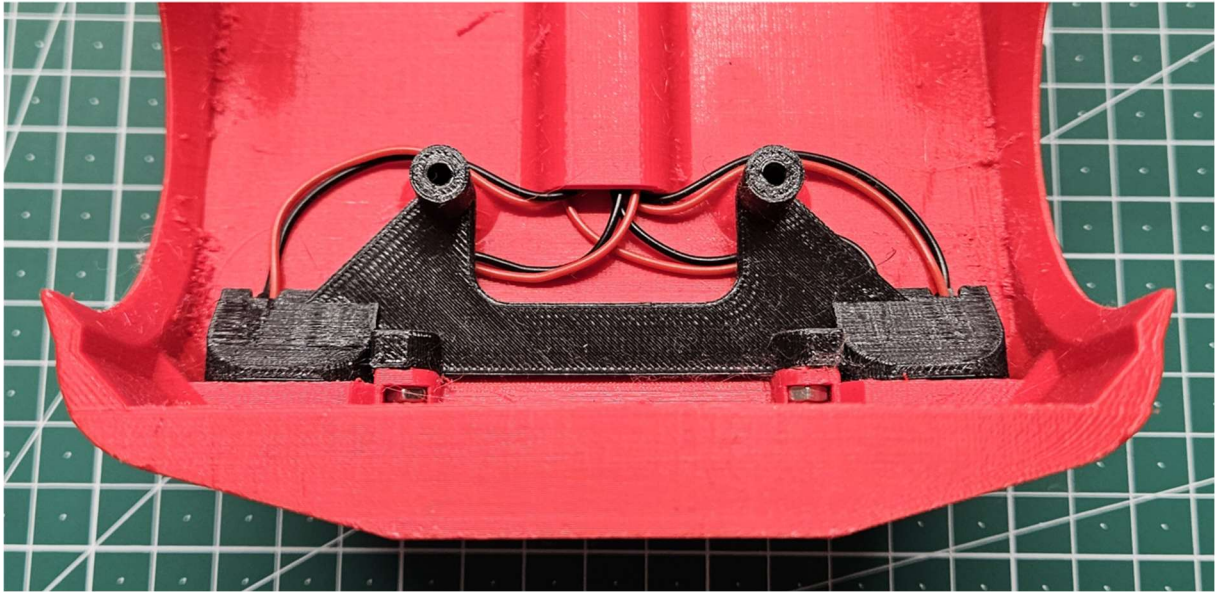
2



3

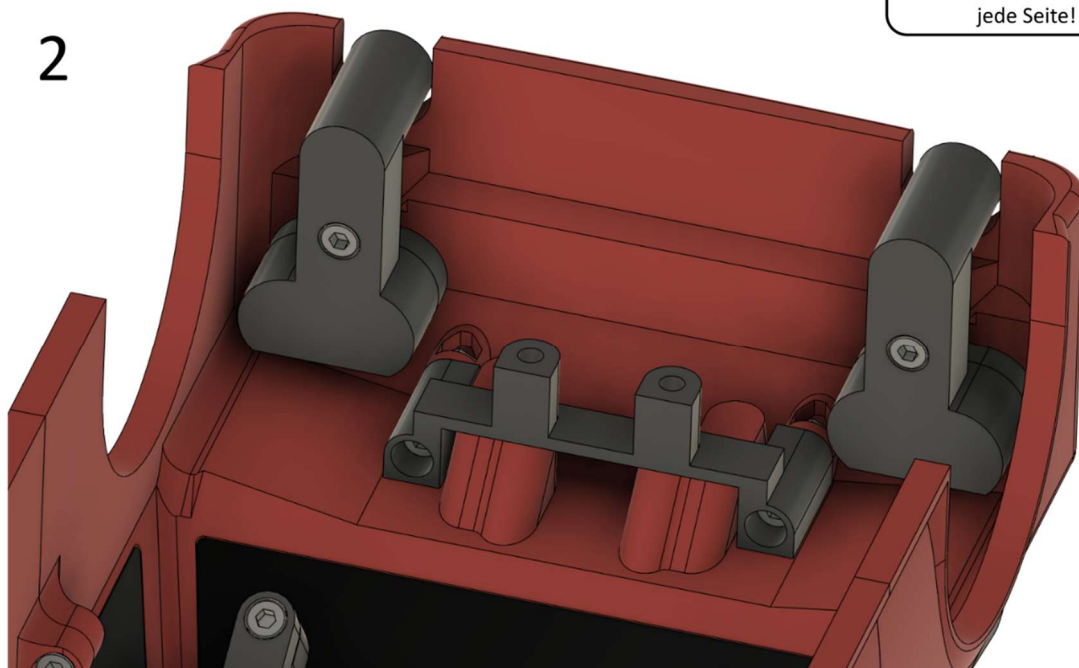
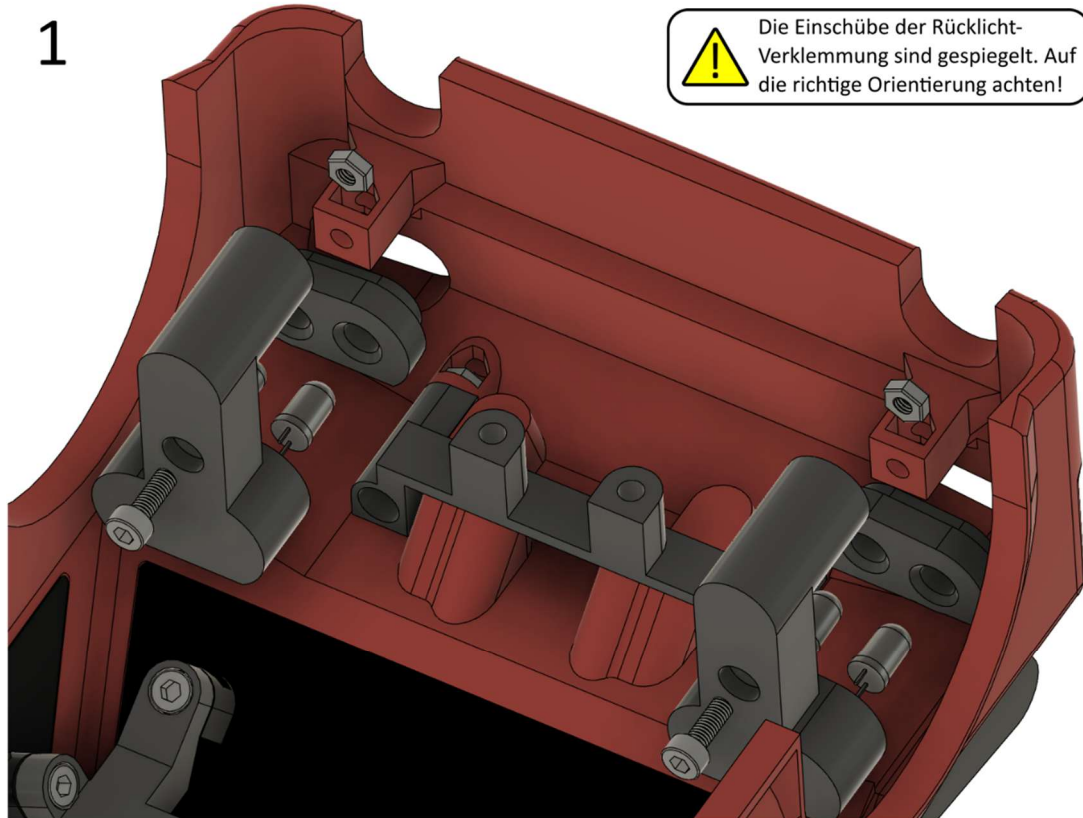


Die LED-Anschlussdrähte
müssen wie auf dem Foto
gezeigt angeordnet werden!



4.5.4 Heckscheinwerfer

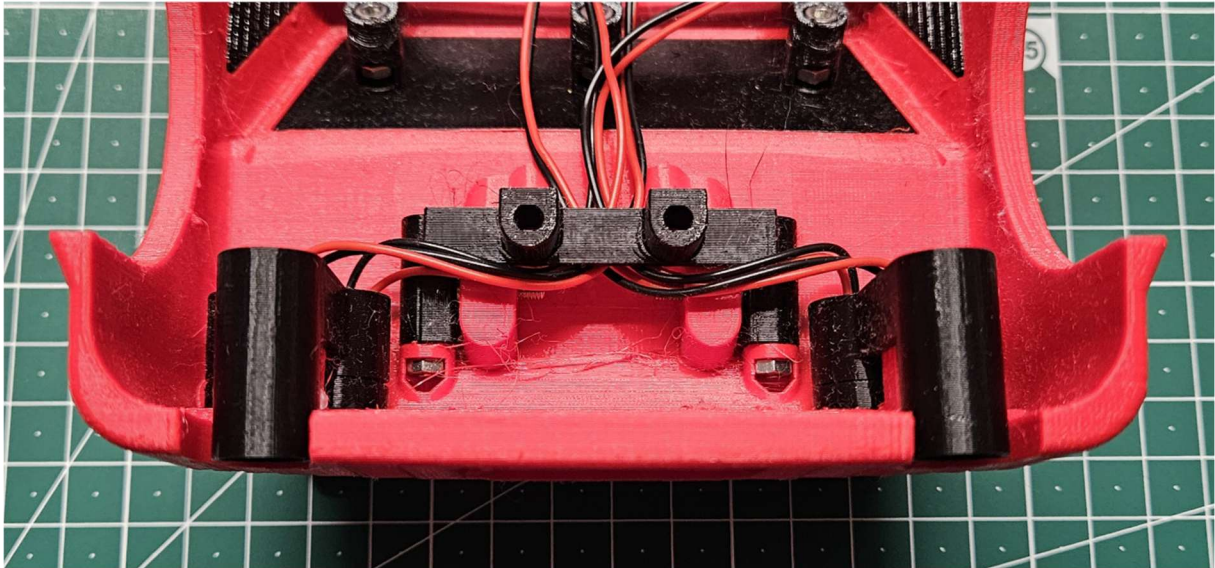
Benötigte Bauteile: Karosserie (vorbereitet), Rücklicht-Verklebung (2x), LED-Einsatz hinten rechts, LED-Einsatz hinten links, LED rot (2x), LED gelb (2x), M3x8 mm Schraube (2x), M3 Mutter (2x)



3



Die LED-Anschlussdrähte
müssen wie auf dem Foto
gezeigt angeordnet werden!



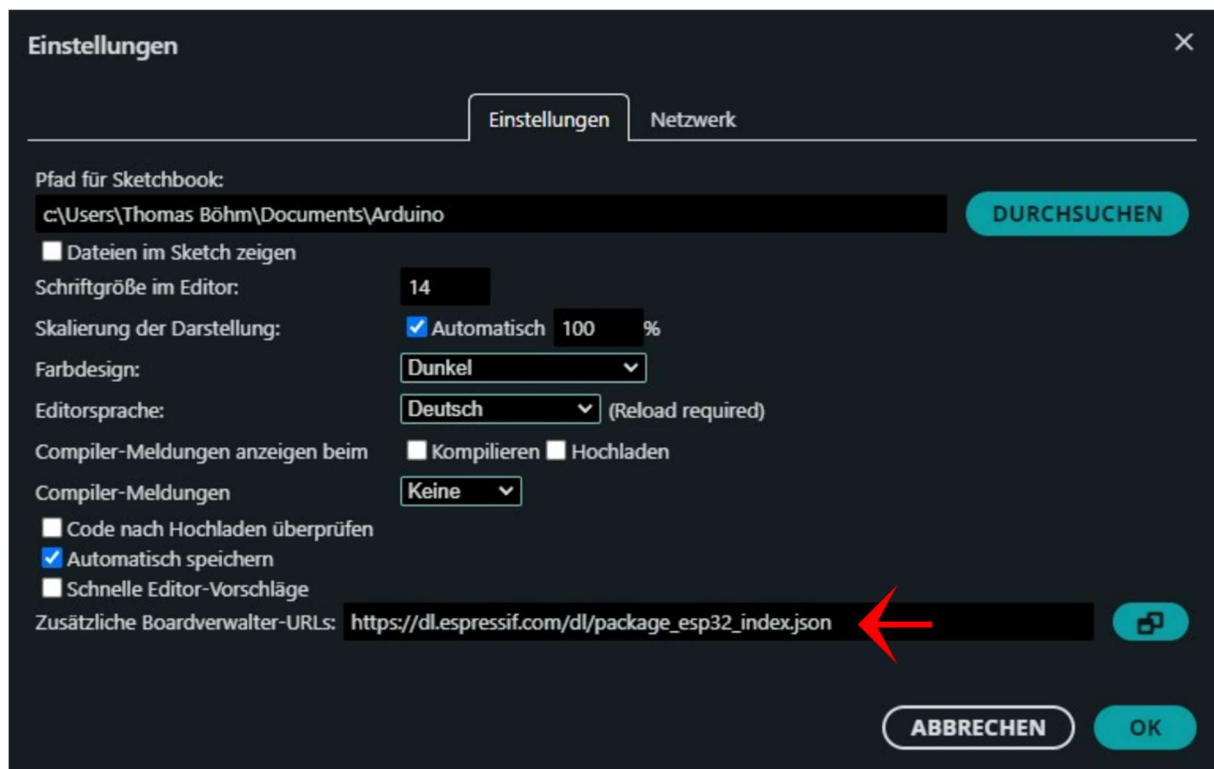
5. Programmierung

5.1 Installation Arduino IDE

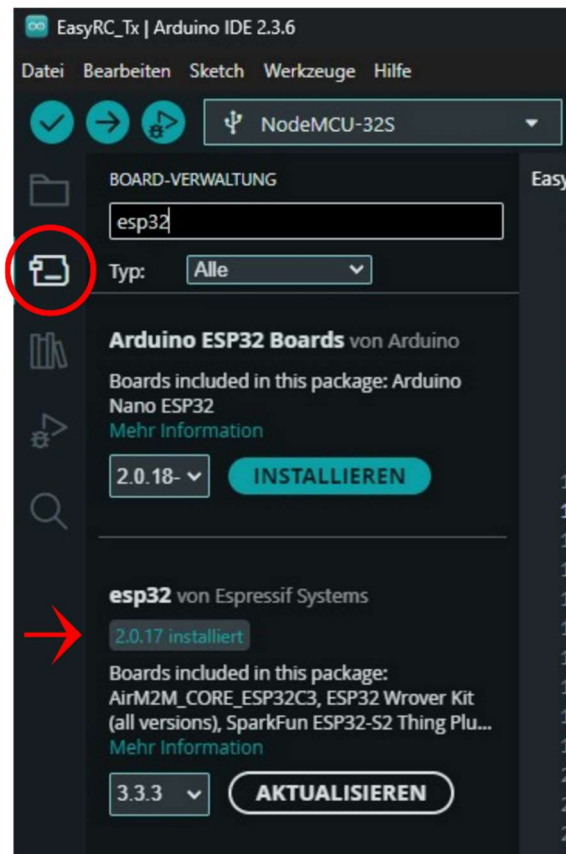
EasyRC verwendet Microcontroller aus der ESP32-Familie der Firma Espressif Systems. Diese Plattform kann frei über die Arduino-Entwicklungsumgebung programmiert werden. Für die Programmierung wird deshalb die Software Arduino IDE (getestet mit Version 2.3.6) verwendet:

<https://www.arduino.cc/en/software/>

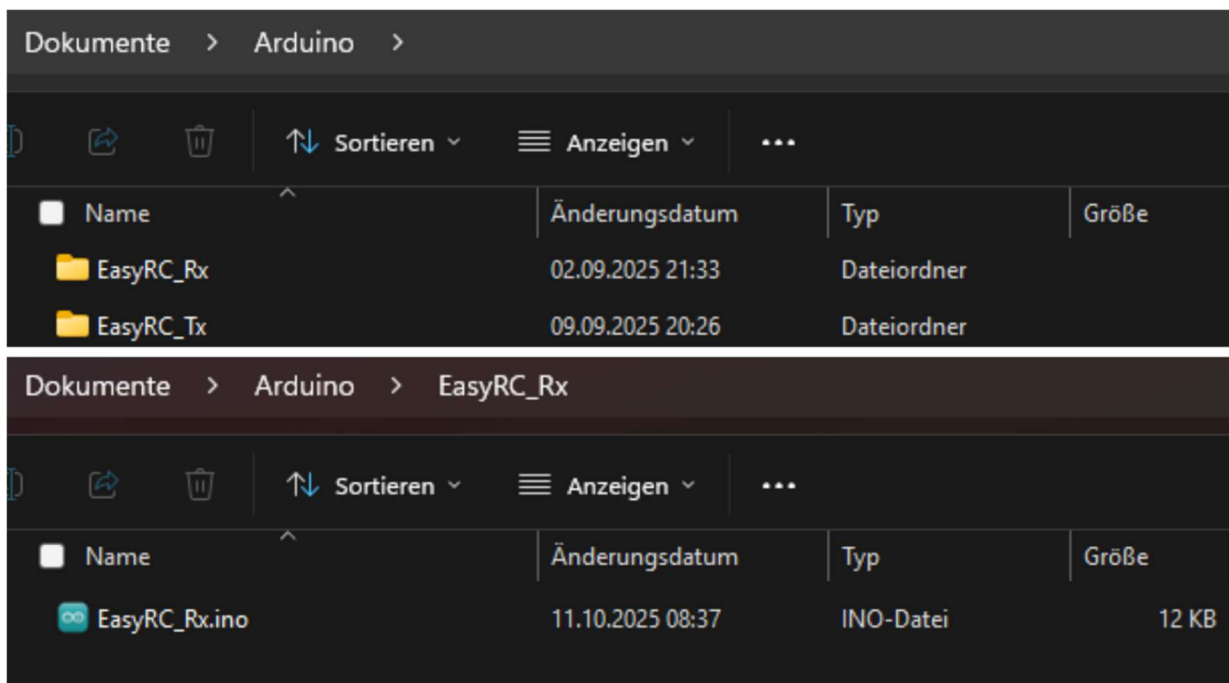
Nach der Installation der Software muss sie für die Verwendung der ESP32-Microcontroller eingerichtet werden. Unter Datei -> Einstellungen muss unter „Zusätzliche Boardverwalter-URLs“ die Adresse https://dl.espressif.com/dl/package_esp32_index.json eingegeben werden.



Als nächstes muss der ESP32-Boardverwalter installiert werden. Dazu wird in der Symbolspalte am linken Rand des Anwendungsfensters das zweitoberste Symbol angeklickt. Dadurch öffnet sich die Board-Verwaltung als Sidebar. Hier muss „esp32“ eingegeben werden und im Anschluss „esp32 von Espressif Systems“ ausgewählt und installiert werden. Dabei muss die Version 2.0.17 gewählt werden (nicht die aktuelle Version!). Der Hintergrund dafür ist, dass bestimmte Programmbibliotheken im Generationswechsel von V2 auf V3 eine andere Syntax verwenden. V3 ist aber nicht so stabil wie V2, weshalb EasyRC für die ältere Version programmiert worden ist.



Um eine Code-Datei in Arduino IDE zu öffnen, muss sie in einem Ordner abgelegt sein, der denselben Namen wie die Datei trägt (ohne die .ino-Endung der Datei). Der Standard-Pfad für Arduino IDE ist in Windows Dokumente -> Arduino. Sofern keine Änderungen daran vorgenommen werden sollen, können einfach in diesem Ordner die zwei notwendigen Ordner für die EasyRC-Codes der Fernsteuerung und des Codes erstellt werden und anschließend die eigentlichen Codes dort abgelegt werden. Die Codes sind frei zum Download verfügbar unter <https://github.com/TRD-B/EasyRC>.



5.2 Mögliche und benötigte Code-Modifikationen

EasyRC ist so konzipiert, dass der Code des Empfängers (des Modellfahrzeugs) (EasyRC_Rx.ino) unverändert auf den Microcontroller hochgeladen werden kann, sofern keine Änderung der Geräte-Adresse (MAC-Adresse) notwendig ist. Die Geräte-Adresse muss angepasst werden, wenn mehr als ein EasyRC-System am gleichen Ort betrieben werden soll. Die Geräte-Adresse legt fest, welche Fernsteuerung welches Fahrzeug steuert (siehe dazu Kapitel 9.1). Der Code des Senders (der Fernsteuerung) (EasyRC_Tx.ino) beinhaltet einige Variablen, die modifiziert werden müssen, um das Fahrzeug zu kalibrieren. Diese sind unter dem Abschnitt „//calibration data“ im Code zu finden:

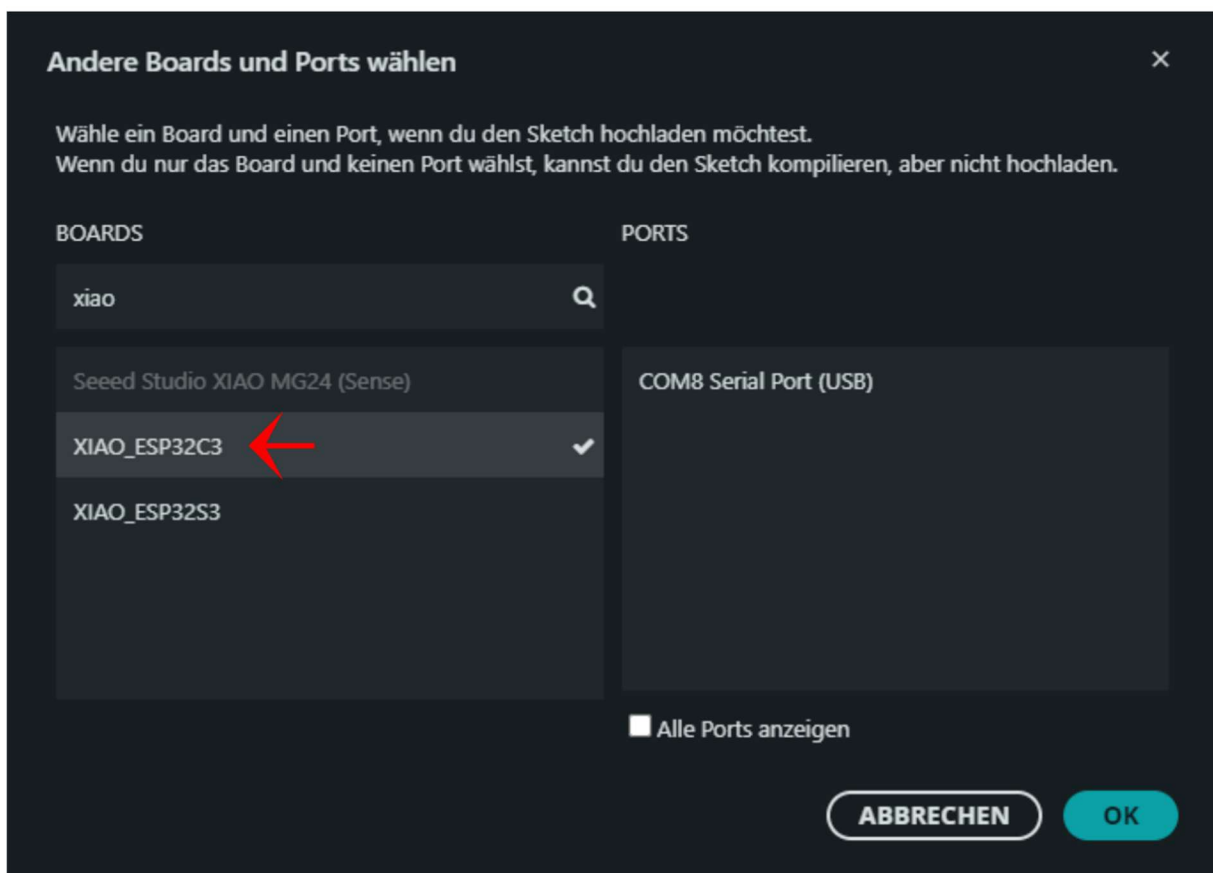
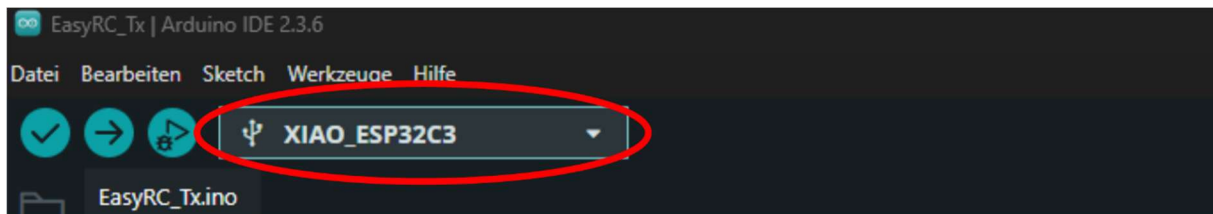
```
EasyRC_Tx.ino
1 //Libraries
2 #include <WiFi.h>
3 #include <esp_now.h> //library for bidirectional wireless communication between two ESP32
4 #include <esp_wifi.h> //needed to adjust the MAC address
5 #include <Wire.h>
6
7 //Pin definitions
8 #define vsense 2
9 #define joyx 3
10 #define joyy 4
11 #define button 5
12 #define led 20
13
14 //calibration data
15 bool motordirection = false; //change this variable to false to invert the motor spin direction
16 int motorint = 10; //sets the delay time between motor speed changes (in ms). Higher values result in
17 //better traction but slower responses. Values between 5 and 10 are recommended.
18 int speedlimit = 127; //this variable sets the maximum speed of the car (values between 0 and 255)
19 int leftlimit = 358; //steering limit calculation: leftlimit = x/20ms*2^12 (default: 205 (x = 1ms))
20 int rightlimit = 258; //steering limit calculation: rightlimit = x/20ms*2^12 (default: 410 (x = 2ms))
21 //RC servos run at 50 Hz (20 ms intervals), with the center at 1.5 ms and a range of +/-0.5 ms around the center
22 //starting values should be 205 to 410 (left to right or right to left, depends on the servo)
23 //adjust these values to a smaller or larger interval depending on the actual steering angles
24 //individually decrease/increase one of the two values to trim the center of the steering
25
26 //MAC addresses
27 uint8_t CarMAC[] = {0xAA, 0xA0, 0xBE, 0xEF, 0xFE, 0xED}; //we will set this MAC address for the car
28 uint8_t RemoteMAC[] = {0xB0, 0xAD, 0xBE, 0xEF, 0xFE, 0xEF}; //we will set this MAC address for the remote control
29 uint8_t broadMAC[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}; //using this MAC address as a receiver for sending
30 //packages means broadcasting (ignoring if the data has been received)
```

Zeile	Code	Mögliche Werte	Erklärung
15	bool motordirection = false;	true, false	Diese Variable kehrt die Drehrichtung des Motors um. Sollte das Fahrzeug beim Bewegen des Joysticks nach vorn rückwärtsfahren, kann hier true gegen false (oder umgekehrt) ausgetauscht werden.
16	int motorint = 10;	1 - 20	Diese Variable verändert, wie schnell das Fahrzeug Änderungen der Geschwindigkeit basierend auf der Joystickposition umsetzt. Ein niedrigerer Wert sorgt für eine höhere Reaktionsfreudigkeit. Ein höherer Wert verhindert ein Durchdrehen der Reifen, lässt das Fahrzeug jedoch träger auf Brems- und Beschleunigungseingaben reagieren.

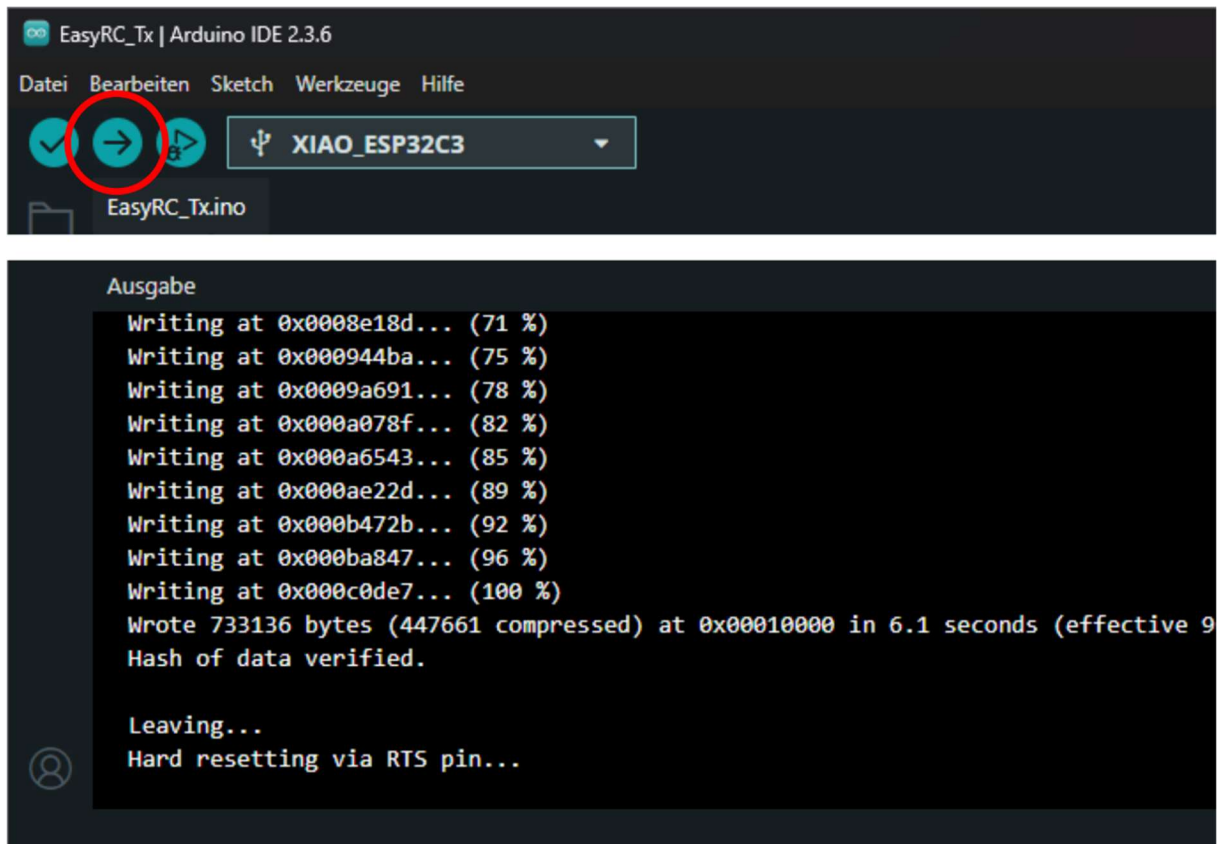
18	int speedlimit = 127;	51 - 255	Diese Variable limitiert die maximale Geschwindigkeit des Fahrzeugs. 255 stellt die Maximalgeschwindigkeit dar. Werte kleiner als 51 (20% der Maximalgeschwindigkeit) sind zwar möglich, aber nicht zu empfehlen, da der Motor so nicht genug Kraft hat, um das Fahrzeug tatsächlich zu bewegen.
19	int leftlimit = 358;	308 - 410	Diese Variable definiert den maximalen Lenkungswinkel des Fahrzeugs nach links. Der Wert ergibt sich aus der Ansteuerung des Servomotors. Ein Wert von 308 entspricht der neutralen Position (Zentrum) des Servomotors. Bei den getesteten Servomotoren entspricht ein höherer Wert einem Ausschlag der Lenkung nach links; das kann je nach Servomotor-Typ allerdings auch umgekehrt sein. Ein Wert von 410 entspricht dem theoretischen Maximalausschlag des Servomotors. Es sollte nicht blind der Maximalwert eingetragen werden, sondern stufenweise der maximale Ausschlag der Lenkung ermittelt werden. Der maximale Lenkwinkel ist der Winkel, bei dem die Räder noch nirgendwo an der Karosserie streifen.
20	int rightlimit = 258;	205 - 308	Diese Variable definiert den maximalen Lenkungswinkel des Fahrzeugs nach rechts (analog zum Lenkungswinkel nach links). Die neutrale Position der Lenkung ergibt sich aus dem Mittelwert des maximalen linken und maximalen rechten Lenkungswinkels. Das bedeutet, dass bei der Erhöhung eines Werts (größer als 308) der andere entsprechend verringert werden muss (kleiner als 308), um den Geradeauslauf des Fahrzeugs nicht zu verändern. Das theoretische Maximalintervall zwischen 205 und 410 für die Servo-Ansteuerung kann je nach Servomotor abweichen. Bei den getesteten Servomotoren war ein kleineres Intervall ausreichend.

5.3 Programmieren der Microcontroller

Zum Hochladen eines Codes auf einen Microcontroller muss dieser per USB an den PC angeschlossen werden. **Wenn der Microcontroller des Fahrzeugs programmiert werden soll, muss dafür der „ESP PWR“ Jumper vorher abgezogen werden (andernfalls kann der USB-Anschluss des PCs beschädigt werden!).** Die Fernsteuerung kann direkt über den von außen zugänglichen USB-Anschluss programmiert werden. Nachdem der Microcontroller mit dem PC verbunden worden ist, muss in Arduino IDE das richtige Modell und der richtige USB-Anschluss ausgewählt werden. Dazu muss in der oberen Leiste in das Feld mit dem USB-Verbindungssymbol geklickt werden. Daraufhin öffnet sich ein Dropdown-Menü, in dem auf „Wähle ein anderes Board und einen anderen Port ...“ geklickt werden muss. Um den passenden Microcontroller zu finden, kann in der linken Spalte nach „Xiao“ gesucht werden. Das richtige Modell ist der „XIAO_ESP32C3“. In der rechten Spalte werden angeschlossene USB-Komponenten angezeigt. Die Nummer des Ports (COM8 im Bild) variiert je nach verwendetem USB-Anschluss des Computers. Sollte hier mehr als ein Port angezeigt werden, obwohl nur ein Microcontroller angeschlossen ist, hilft es, nicht benötigte USB-Geräte vom PC abzustecken und erneut nach aktiven Ports zu suchen.



Sobald das Microcontroller-Modell und der Port ausgewählt sind, kann der Code hochgeladen werden. Dazu wird in der oberen Leiste in Arduino IDE auf den nach rechts zeigenden Pfeil geklickt. Der Computer prüft daraufhin den Code auf Syntaxfehler, kompiliert ihn (Umwandlung in maschinentauglichen Code) und überträgt ihn via USB auf den Microcontroller.



Wenn das Hochladen erfolgreich ist, zeigt das Ausgabefenster schließlich den in der Abbildung gezeigten Text („Leaving... Hard resetting via RTS pin...“) an. Die USB-Übertragung ist allerdings fehleranfällig: Sollte die Übertragung abbrechen oder mit einer Fehlermeldung gar nicht erst starten, sollte der Microcontroller ab- und wieder angesteckt werden sowie geprüft werden, ob der richtige USB-Port ausgewählt worden ist. Besteht der Fehler nach mehreren Versuchen weiterhin, kann auch ein Neustart des PCs Abhilfe schaffen.

6. Code der Microcontroller

Hier folgen kurze Erläuterungen zum Code der beiden Microcontroller, ohne auf die Syntax einzugehen. Es werden keine Codeabschnitte gezeigt, um das Kapitel kompakt zu halten. Die Codes können jederzeit über GitHub abgerufen werden: <https://github.com/TRD-B/EasyRC>

6.1 Fernsteuerung

Der Microcontroller der Fernsteuerung muss den Joystick auslesen, die Batteriespannung prüfen und die verarbeiteten Daten des Joysticks an das Fahrzeug schicken. Zunächst werden Programmbibliotheken eingebunden und die benötigten Variablen definiert. Außerdem wird außerhalb der Funktionen im Code definiert, wie das Datenpaket aussieht, das an das Fahrzeug übertragen wird. Danach folgt die Setup-Funktion, die einmalig beim Start des Microcontrollers ausgeführt wird und Initialisierungen vornimmt. Hier ist die Kalibration der Joystick-Achsen zu erwähnen: Beim Anschalten liest der Microcontroller mehrfach die beiden Achsen aus, um ihren Neutralpunkt zu bestimmen. Dazu werden die Pinfunktionen festgelegt und ESP-NOW initialisiert. In der permanent ablaufenden Loop-Funktion werden Unterfunktionen aufgerufen (Auslesen der Joystick-Achsen, Auslesen der Batteriespannung), geprüft, ob der Joystick-Druckknopf gedrückt ist und alle 50 ms ein Datenpaket an das Fahrzeug verschickt. Zusätzlich wird eine serielle Ausgabe erzeugt, die die aktuellen Sensorzustände zeigt, wenn ein PC angeschlossen ist.

Beim Auslesen der Joystick-Achsen wird ein kleiner Bereich um den Neutralpunkt nicht als Abweichung berücksichtigt, um ein Zittern der Lenkung und des Antriebsmotors durch Messrauschen und Toleranzen des Joysticks zu verhindern. Um die Joystick-Achsen symmetrisch zu halten, wird außerdem berücksichtigt, dass der Neutralpunkt nicht zwingend dem Mittelwert aus 0 V und 3,3 V entsprechen muss, sondern der Maximalausschlag bei einer vorgegebenen Abweichung rund um den Neutralpunkt gekappt. Das Auslesen der X-Achse (Antriebsmotor) enthält ein Segment, das den Ausgabewert invertiert. Dieses Segment wird durch das Setzen der Variable „motordirection“ (Kapitel 5.2) (de)aktiviert. Außerdem gibt die X-Achse dem Fahrzeug auch vor, wie die Helligkeit der Rücklichter (Rücklicht versus Bremslicht) eingestellt werden soll, abhängig davon, ob aktuell vorwärts gefahren/beschleunigt werden soll oder der Joystick in der Neutralposition bzw. rückwärtsgerichtet ist. Für die Y-Achse (Lenkung) wird entsprechend an das Fahrzeug weitergegeben, ob gelenkt und wenn, ob nach links oder rechts gelenkt wird, um die Blinker entsprechend ansteuern zu können.

Das Auslesen der Batteriespannung erfolgt unter Berücksichtigung des eingesetzten Spannungsteilers und lässt beim Unterschreiten eines Grenzwerts (3,4 V) einen internen Zähler ansteigen. Dieser Grenzwert ist verhältnismäßig hoch für eine Lilon-Zelle, die gefahrlos bis 3,0 V entladen werden kann, allerdings versorgt die Batterie einen im Xiao ESP32-C3 integrierten Spannungswandler, der stabile 3,3 V für den Microcontroller zur Verfügung stellen muss. Deshalb wird eine höhere Spannung benötigt. Sobald der Zähler einen vorgegebenen Wert übersteigt, wird die Batteriewarnung (blinkende LED der Fernsteuerung) ausgelöst. Hier wird ein Zähler verwendet, um die Warnung nicht bei einem einzigen Unterschreiten des Spannungsgrenzwerts auszulösen, da Messrauschen oder abrupte Bewegungen der Fernsteuerung und resultierende Spannungsschwankungen (schlechter Kontakt zwischen Batterie und Batteriehalter) so Fehlalarme auslösen könnten.

Die serielle Ausgabe gibt die aktuellen Messwerte der Eingänge des Microcontrollers aus. Dieses Codesegment ist für die Fernsteuerung nicht notwendig, stört aber den Betrieb nicht und kann zum Debuggen genutzt werden.

6.2 Fahrzeug

Der Microcontroller des Fahrzeugs muss die Daten der Fernsteuerung erhalten und darauf basierend den Lenkungsservo und den Antriebsmotor ansteuern. Zusätzlich steuert er den Piezo-Buzzer und die LEDs des Fahrzeugs an und liest die Batteriespannung aus. Im Anfangsbereich des Codes werden ebenfalls Bibliotheken aufgerufen, Variablen definiert und die Verwendung von ESP-NOW festgelegt. In der Setup-Funktion wird die Verwendung der Pins vorgegeben und ein PWM-Timer des Microcontrollers für den Servomotor programmiert (auf 50 Hz eingestellt; das ist die für RC-Servos übliche Steuerungsfrequenz). Anschließend wird ESP-NOW aktiviert und LED-Lichtsequenzen sowie Tonsequenzen des Piezo-Buzzers aufgerufen, die anzeigen, dass das Fahrzeug angeschaltet wird, auf eine Verbindung zur Fernsteuerung wartet oder dass die Verbindung zur Fernsteuerung hergestellt worden ist. In der Loop-Funktion werden Unterfunktionen aufgerufen, um die Motoren und den Piezo-Buzzer zu steuern sowie in einem größeren Codesegment die verschiedenen Alarmzustände abgefragt bzw. entsprechende Warnungen (LEDs blinken und der Piezo-Buzzer piept) beim Erreichen der Alarmzustände ausgegeben. Die Warnzustände sind ein Verbindungsabbruch zur Fernsteuerung, ausbleibende Nutzereingaben für eine längere Zeit sowie das Unterschreiten eines Batteriespannungsgrenzwerts.

Der Verbindungsabbruch zur Fernsteuerung wird detektiert, indem bei jedem erhaltenen Datenpaket ein Timer zurückgesetzt wird (rctimer). Sollte die aktuelle Zeit die Zeit beim Erhalt des letzten Datenpakets um 1 s überschreiten, wird das als Verbindungsabbruch gewertet. Ausbleibende Nutzereingaben werden über eine Unterfunktion detektiert, die einen Timer zurücksetzt, wenn das Fahrzeug bewegt werden soll oder hupen soll. Sollte beides für 60 s lang nicht erfolgen, wird dies als ausbleibende Nutzereingaben gewertet. Die Batteriespannung wird in einer separaten Unterfunktion genau wie bei der Fernsteuerung gemessen, allerdings mit einem an die höhere Batteriespannung des Fahrzeugs angepassten Spannungsteiler und Grenzwert.

Im Loop des Microcontrollers werden die Ansteuerung der Blinker sowie des Bremslichts direkt entsprechend den Eingaben durch die Fernsteuerung übernommen. Das Rücklicht als dunklere Stufe der Bremslichter wird nicht durch eine PWM-Dimmung der LEDs umgesetzt, sondern durch das abwechselnde Aktivieren und Deaktivieren bei jedem dritten Codedurchlauf. Dies ist ohne sichtbares Flackern der Rücklichter möglich, da der Microcontroller sehr schnell durch seinen Code läuft (≤ 1 ms, sodass dies einer Frequenz von mehreren Hundert Hz entspricht). Die Blinker blinken im Sekundentakt (sie leuchten jede Sekunde für eine halbe Sekunde), wobei der Timer bei jedem Richtungswechsel zurückgesetzt wird, sodass der Blinker beim Einschlagen in eine Richtung immer mit einem Leuchten beginnt.

Zudem wird im Loop des Microcontrollers abgefragt, ob die Hupe des Fahrzeugs ausgelöst werden soll und entsprechend über eine Unterfunktion ein PWM-Timer und -Kanal umprogrammiert (setPWMchannel). Theoretisch sollte der ESP32-C3 über sechs voneinander unabhängig programmierbare PWM-Kanäle verfügen, allerdings war dies mit der Arduino IDE nicht umsetzbar: Sobald mehr als zwei Kanäle aufgerufen werden, kommt es zu Fehlfunktionen der PWM-Kanäle. Deshalb wird im Code ein Kanal und Timer für den Lenkungsservo reserviert und der andere je nach Bedarf einer gewünschten Tonfrequenz für den Piezo-Buzzer oder aber dem Antriebsmotor zugeordnet. Das führt zu der Einschränkung, dass das Fahrzeug nicht gleichzeitig fahren und den Piezo-Buzzer auslösen kann, sorgt aber für einen fehlerfreien Betrieb aller Komponenten. Die Tonfrequenz des Piezo-Buzzer wird über die PWM-Frequenz definiert, wobei der Kanal beim Auslösen eines Tons immer mit einem Tastgrad von 50% aufgerufen wird (Code-Unterfunktion

Buzzer). Für den Antriebsmotor wird eine PWM-Frequenz von 20 kHz genutzt. Die hohe Frequenz (im unhörbaren Bereich) unterbindet unerwünschte Fiep- oder Zirp-Geräusche des Motors bei Teillast.

Der Servomotor wird direkt über die eingehenden Befehle angesteuert (ServoControl), da diese bereits für die Verwendung des RC-Servos im Code der Fernsteuerung angepasst werden. Der Servomotor benötigt ein PWM-Signal mit einer Frequenz von 50 Hz, wobei der Tastgrad vorgibt, welchen Winkel der Motor einnimmt. Eine Signaldauer von 1,5 ms entspricht dem Mittelwert und das üblicherweise verwendbare Intervall reicht von 1 ms bis 2 ms. Da der PWM-Kanal für den Servomotor mit 12 bit angesprochen wird, entsprechen die 1,5 ms einem Wert von 308 ($308/(2^{12})$ mal 20 ms (50 Hz Intervall)) und die Limits liegen bei 205 und 410.

Für den Antriebsmotor wird in einer Unterfunktion (setMotorSpeed) die Geschwindigkeitsänderung in Abhängigkeit der aktuellen Geschwindigkeit und angeforderten Geschwindigkeit geändert. Es wird bei hohen Änderungen eine Interpolation durchgeführt, die die tatsächliche Motorgeschwindigkeit langsamer ändert. Dadurch sollen übermäßig abrupte und damit für den Antriebsstrang stark belastende Eingaben verhindert werden. Die Drehrichtung wird nur geändert, wenn die Geschwindigkeit nahe Null liegt, ebenfalls um Belastungen auf dem Antriebsstrang gering zu halten.

7. Erste Schritte und Bedienung des Fahrzeugs

Wenn die Microcontroller der Fernbedienung und des Fahrzeugs programmiert worden sind, kann es schon fast losgehen! Zunächst muss aber noch die Lenkung des Fahrzeugs getrimmt werden und die Karosserie aufgesetzt werden.

7.1 Batterien einsetzen und Geräte anschalten

Als erstes werden Batterien in das Fahrzeug und die Fernbedienung eingesetzt. Die EasyRC-Platinen sind zwar vor Verpolung der Eingangsspannung geschützt, aber dennoch sollte man es nie auf solche Sicherungen ankommen lassen: Beim Einbau bitte immer zweimal prüfen, ob die Batterien richtigerum eingesetzt wurden. Anschließend kann die Fernbedienung angeschaltet werden. Die Status-LED der Fernbedienung leuchtet durchgehend, wenn sie ordnungsgemäß funktioniert. Sollte die LED blinken, ist die Batteriespannung niedrig. Dann muss die Batterie zunächst aufgeladen werden, bevor das Aufsetzen des Fahrzeugs weitergeht. Das ist wichtig: Lilon-Batterien reagieren empfindlich auf Tiefentladung. Sollte die Batteriespannung zu tief fallen, wird die Batterie beschädigt.

Wenn die Fernbedienung angeschaltet wird, kalibriert sich der Joystick automatisch. Die Position, die der Joystick zum Zeitpunkt des Einschaltens innehat, wird als neutrale Position hinterlegt. Deshalb darf der Joystick beim Anschalten der Fernbedienung nicht bewegt werden. Diese Kalibration wird bei jedem Start der Fernbedienung durchgeführt. Sollte man also aus Versehen den Joystick bewegt haben, muss die Fernbedienung einfach nur aus- und wieder angeschaltet werden, um wieder eine korrekte Kalibration zu erhalten. Als zweites wird das Fahrzeug angeschaltet. Die ESP PWR-Kurzschlussbrücke muss davor installiert sein, denn andernfalls bekommt der Microcontroller keinen Strom. Da die LEDs noch nicht angeschlossen sind, wenn die Karosserie noch nicht aufgesetzt ist, gibt zunächst nur Feedback zum Status des Fahrzeugs über den Piezo-Buzzer. Er piept dreimal nach dem Anschalten (der dritte Ton ist etwas länger und höher als die ersten beiden Töne). Anschließend piept der Buzzer alle zwei Sekunden, sofern das Fahrzeug nicht mit einer Fernsteuerung verbunden ist. Sobald das Fahrzeug „seine“ Fernsteuerung findet, folgt eine Signalsequenz mit einem längeren und einem kürzeren Piepton.

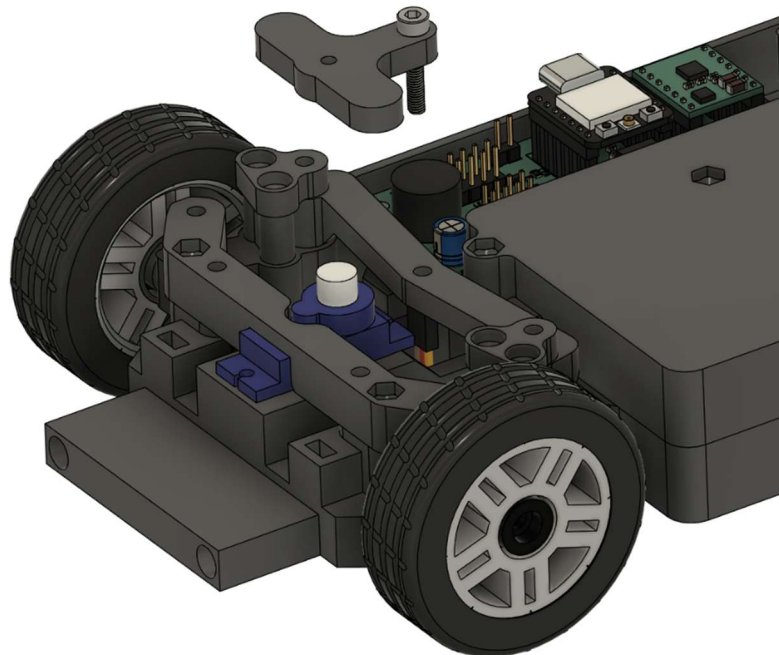
7.2 Lenkung fertig montieren

Zunächst zur Lenkung: Wie im Abschnitt zur Programmierung (Kapitel 5.2) erklärt worden ist, gibt es im Code zwei Variablen, die den maximalen Lenkausschlag nach links und rechts festlegen. Diese müssen angepasst werden, sodass das Fahrzeug bei neutraler Joystick-Position (bezogen auf die Achse von links nach rechts) auch tatsächlich geradeaus fährt und außerdem beim Einschlagen der Lenkung die Vorderräder nicht am Rahmen streifen (der maximale Lenkausschlag darf nicht zu groß sein).

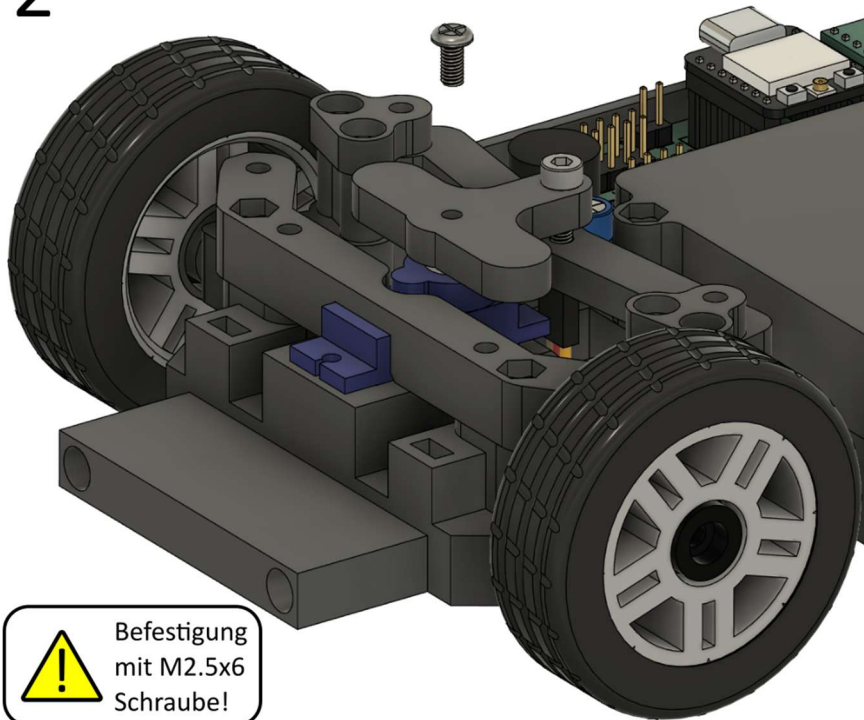
Nun muss das Servohorn auf den Servomotor gesetzt und mit dem Lenkgestänge verbunden werden. Dieser Schritt wird erst jetzt durchgeführt, da zunächst der Servomotor seine neutrale Position einnehmen muss. Der Servomotor fährt automatisch seine neutrale Position an, wenn der Joystick der Fernbedienung nicht bewegt wird und der Mittelwert zwischen linkem und rechtem maximalen Lenkwinkel bei 308 liegt. Das ist die Ausgangslage, wenn der Code der Fernsteuerung von GitHub unverändert übernommen worden ist.

Das Servohorn wird so auf den Servomotor gesetzt, dass die Linie zwischen dem Kontaktpunkt zum Lenkgestänge und der Motorwelle möglichst parallel zur Längsachse des Fahrzeugs ist (entsprechend der roten Linie in Bild 3 der folgenden Abbildung). Da die Abtriebswelle des Servomotors ein kleines Zahnrad ist und das Servohorn in dieses Zahnrad greifen muss, kann es sein, dass das Servohorn nicht in dieser Idealline aufgesetzt werden kann. Das ist kein Problem; es sollte nur so nahe wie möglich an dieser Idealline ausgerichtet sein. Die bereits vorinstallierte M3x16 mm Schraube des Servohorn-Aufsatzes muss in die entsprechende Aussparung des Lenkgestänges geschoben werden. Schließlich muss das Servohorn noch mit einer 6 mm langen Schraube auf dem Motor fixiert werden. Die Anleitung gibt dafür eine M2.5 Schraube an, wobei der Durchmesser je nach Servotyp auch abweichen kann. Nun ist die erste Grundeinstellung der Lenkung abgeschlossen. Die feine Trimmung wird ausschließlich über die Software der Fernsteuerung durchgeführt, nachdem das Fahrzeug mit Karosserie fertig montiert ist.

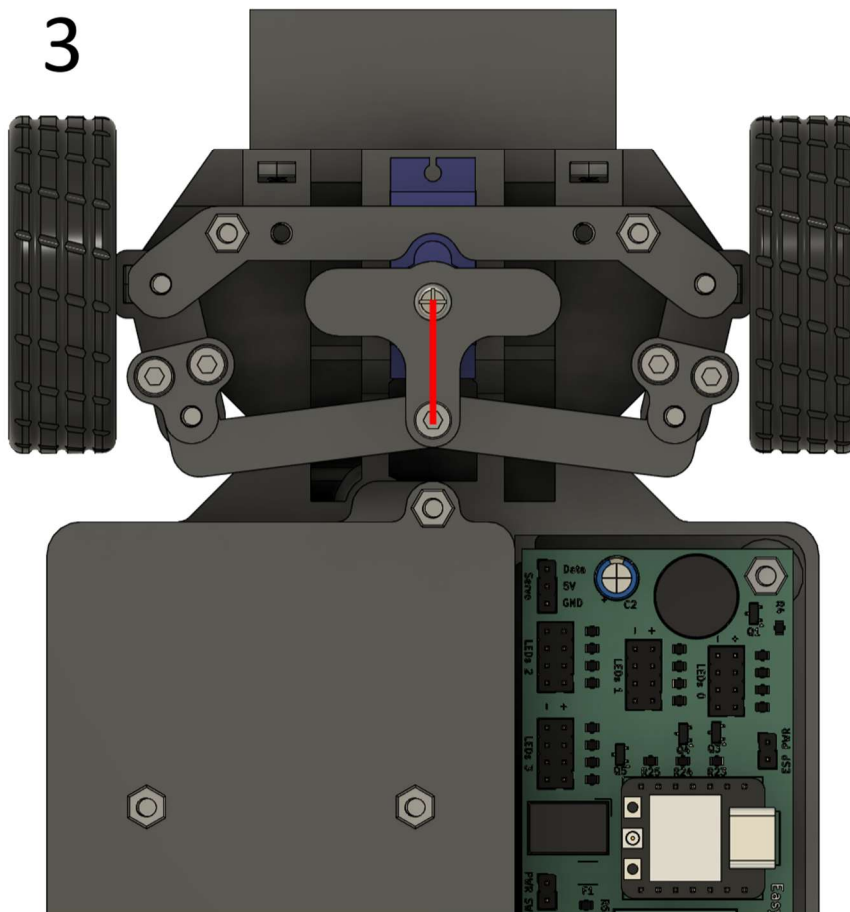
1



2



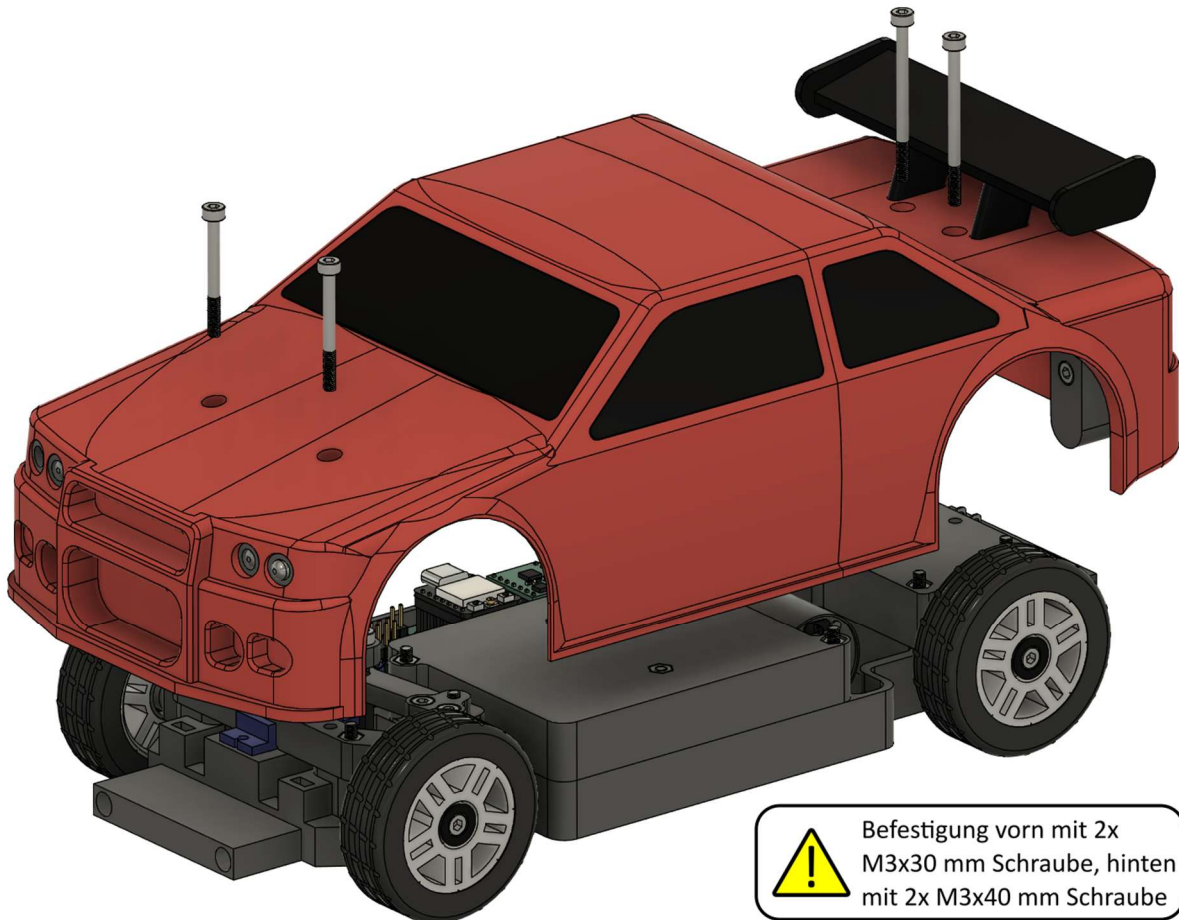
3



Nach der Einstellung der Neutralposition des Servos muss das Servohorn so aufgesetzt werden, dass es möglichst parallel zur roten Linie ausgerichtet ist.

7.3 Fahrzeug fertig montieren

Nun werden die LEDs der Karosserie mit der Platine verbunden und anschließend die Karosserie auf das Chassis aufgesetzt. Die LEDs werden wie in Kapitel 4.4.4 beschrieben angeschlossen. Wichtig: Es dürfen keine Kabel beim Verschrauben der Karosserie eingeklemmt werden. Außerdem müssen die Kabel so verlegt werden, dass sie nicht an den Rädern schleifen – dabei bitte auch den Lenkausschlag der Vorderräder beachten.



7.4 Steuerungsprinzip und Funktionen des Fahrzeugs

Das Fahrzeug wird über den einzelnen Joystick der Fernsteuerung gesteuert. Wenn der Joystick nach vorn bewegt wird, beschleunigt das Fahrzeug vorwärts. Wird der Joystick nach hinten bewegt, fährt das Fahrzeug rückwärts. Sollte diese Zuordnung vertauscht sein, muss die Variable „motordirection“ im Code der Fernsteuerung modifiziert werden (siehe Kapitel 5.2).

Wenn der Joystick nach links bewegt wird, lenkt das Fahrzeug nach links; analog lenkt das Fahrzeug nach rechts, wenn der Joystick nach rechts bewegt wird. Sollte das Fahrzeug in die falsche Richtung lenken, müssen die Werte der Variablen „leftlimit“ und „rightlimit“ im Code der Fernsteuerung vertauscht werden (siehe Kapitel 5.2). Bevor die Lenkung korrekt getrimmt ist, kann es sein, dass die neutrale Lenkposition nicht korrekt ausgerichtet ist. Das wird im nächsten Kapitel eingestellt.

Die weißen Frontscheinwerfer-LEDs leuchten durchgehend, sobald das Fahrzeug angeschaltet wird. Die roten Rücklichter leuchten mit voller Intensität, solange das Fahrzeug steht oder rückwärtsfährt. Sie leuchten mit reduzierter Intensität, wenn das Fahrzeug vorwärtsfährt. Die Blinker blinken automatisch, wenn der Joystick nach links bzw. nach rechts bewegt wird. Sollte die Zuordnung der Lichter nicht passen, sollte geprüft werden, ob alle LEDs korrekt angeschlossen sind.

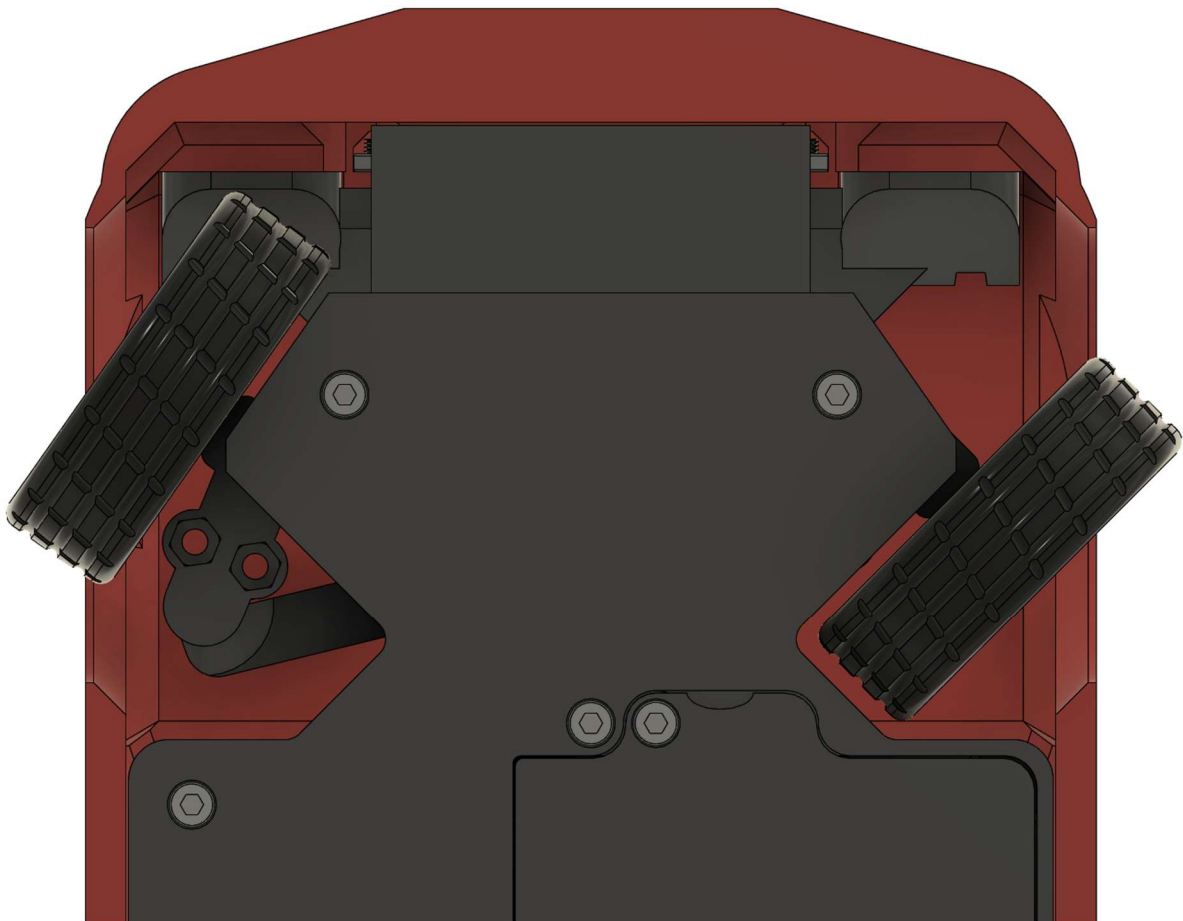
Der Piezo-Buzzer des Fahrzeugs fungiert als akustischer Signalgeber für Systemwarnungen und außerdem als Hupe. Wird der Joystick gedrückt, ertönt ein Signalton, solange der Joystick gedrückt bleibt. Das Fahrzeug hupt allerdings nur, während es stillsteht. Die Funktion ist deaktiviert, sobald das Fahrzeug fährt.

Die Hupe und die Blinklichter dienen außerdem zur Statusanzeige des Fahrzeugs. Neben der initialen Ton- und Lichtsequenz beim Anschalten des Fahrzeugs und der Verbindung mit der Fernbedienung (siehe Kapitel 7.1) ertönt außerdem eine Tonsequenz kombiniert mit dem Leuchten der Blinker, wenn die Batteriespannung zu niedrig ist, das Fahrzeug keine Verbindung mehr zur Fernsteuerung hat, oder aber seit längerer Zeit keine Nutzereingaben über die Fernbedienung mehr erfolgt sind. In allen drei Fällen wird der Antriebsmotor deaktiviert und kein Datensignal mehr an den Servomotor der Lenkung übertragen. Die letzten beiden Fälle sind reversibel, der Alarmzustand kann also durch eine erneute Nutzereingabe bzw. ein Anschalten/in-Reichweite-bringen der Fernbedienung deaktiviert werden. Die Warnung zur niedrigen Batteriespannung kann nur durch einen Neustart des Fahrzeugs zurückgesetzt werden.

Ursache	Ton- und Blinkfrequenz
Batteriespannung sinkt unter 10,2 V	Blinker leuchten und Hupe ertönt alle 3 s für 0,5 s
Keine Nutzereingaben für wenigstens 60 s mehr erfolgt	Blinker leuchten und Hupe ertönt alle 3 s für 1,0 s
Signalverlust zur Fernbedienung für wenigstens 1 s	Blinker leuchten und Hupe ertönt alle 3 s für 1,5 s

7.5 Trimmung der Lenkung

Zum Einstellen der Lenkung wird das Fahrzeug geradeaus bewegt, indem der Joystick nach vorn geschoben wird. Wenn das Fahrzeug auf eine Seite zieht, müssen die Variablen „leftlimit“ und „rightlimit“ angepasst werden. Der maximale Lenkerausschlag wird angepasst, indem die Werte für „leftlimit“ und „rightlimit“ gleichzeitig weiter vom Servo-Mittelwert entfernt werden. Der maximale Lenkerausschlag darf nicht mehr als 45° für das kurveninnere Rad betragen (siehe Abbildung). Solange der maximale Lenkerausschlag noch nicht erreicht ist, kann ein Drall nach links ausgeglichen werden, indem der Wert für „rightlimit“ weiter vom Servo-Mittelwert (308) entfernt wird (analog für einen Rechtsdrall mit „leftlimit“). Wenn der maximale Lenkerausschlag bereits erreicht ist, wird ein Linksdrall durch eine Annäherung des Werts für „leftlimit“ an den Servo-Mittelwert (308) ausgeglichen (analog für einen Rechtsdrall mit „rightlimit“). Die Modifikation der Lenkung kann bei aktivem Fahrzeug direkt im Code der Fernbedienung erfolgen (siehe auch Kapitel 5.2). Das erlaubt eine schnelle Anpassung ohne etwas zerlegen zu müssen.



7.6 Feintuning der Fahreigenschaften

Wenn die Lenkung eingestellt ist, ist das Fahrzeug grundsätzlich vollständig fahrbereit. Es gibt aber noch weitere Variablen, die das Fahrverhalten beeinflussen. Zunächst ist hier die Maximalgeschwindigkeit zu nennen. Diese wird ebenfalls im Code der Fernsteuerung eingestellt, über die Variable „speedlimit“ (siehe Kapitel 5.2). Der Maximalwert liegt bei 255, was einer Geschwindigkeit von bis zu ca. 10 km/h entspricht. Der Wert kann theoretisch bis auf 0 reduziert werden, allerdings skaliert die Geschwindigkeit im niedrigen Bereich nicht mehr linear mit der Vorgabe, da diese die maximale Motorspannung (eingestellt über Pulsweitenmodulation) definiert. Ein Wert von ca. 120 bis 150 stellt einen sinnvollen Kompromiss aus hoher Maximalgeschwindigkeit und guter Kontrollierbarkeit dar: Zu hohe Geschwindigkeiten bedeuten, dass Kurven und Bremsen schwierig zu meistern sind.

Eine weitere Modifikationsmöglichkeit ist die Empfindlichkeit, mit der der Motor auf Eingaben durch den Joystick reagiert. Die Variable „motorint“ im Code der Fernsteuerung (siehe Kapitel 5.2) gibt vor, wie viele Millisekunden zwischen Änderungen der Sollgeschwindigkeit vergehen. Ein niedrigerer Wert bedeutet, dass das Fahrzeug auf abrupte Joystickeingaben sehr schnell reagiert. Dieses sehr responsive Verhalten birgt den Nachteil, dass die Räder zum Durchdrehen neigen, was die Kontrollierbarkeit, insbesondere beim Bremsen, deutlich verschlechtert. Eine Erhöhung des Werts lässt den Motor hingegen träger auf veränderte Eingaben reagieren, sodass die Traktion besser wird, das Fahrzeug aber länger zum Beschleunigen und Bremsen benötigt. Als Startwert ist ein Wert von 10 voreingestellt. Dieser kann schrittweise erhöht oder reduziert werden, bis der individuell bevorzugte Kompromiss aus Reaktionsfreudigkeit und Kontrollierbarkeit gefunden ist.

Die Einstellung der Werte erfolgt in der Fernbedienung, sodass das Fahrzeug wie bei der Trimmung der Lenkung nicht abgeschaltet oder zerlegt werden muss, wenn die Änderungen vorgenommen werden.

8. Mögliche Fehler und Lösungsansätze

Fehlerbeschreibung	Ursache	Lösungsmöglichkeiten
Status-LED der Fernbedienung leuchtet nach dem Anschalten nicht.	Batterie falschherum eingesetzt.	Auf korrekte Polarität achten. Anschlüsse zwischen Platine und Batteriehälter ebenfalls prüfen!
Status-LED der Fernbedienung blinkt zweimal pro Sekunde.	Batteriespannung niedrig (<3,4 V).	Batterie laden.
Status-LED der Fernbedienung flackert unruhig oder mit reduzierter Helligkeit.	Batteriespannung zu niedrig für korrekten Betrieb der Fernsteuerung (<3,0 V).	Batteriespannung prüfen. Sollte die Batterie weniger als 2,5 V haben, wurde sie tiefentladen und muss ersetzt werden. Andernfalls laden.
Es ertönt kein Signalton/es leuchtet keine LED des Fahrzeugs nach dem Anschalten.	Wenigstens eine der drei Batterien falschherum eingesetzt.	Auf korrekte Polarität achten. Anschlüsse zwischen Platine und Batteriehälter ebenfalls prüfen!
Es ertönt kein Signalton/es leuchtet keine LED des Fahrzeugs nach dem Anschalten.	Die Batterien sind defekt (alle drei zusammen liefern <7 V, also weniger als 2,3 V pro Zelle).	Batterien einzeln prüfen und tiefentladene/defekte Zellen ersetzen.
Die Tonsequenz für die Verbindung mit der Fernbedienung ertönt nicht (das Fahrzeug piept alle 2 s, obwohl eine angeschaltete Fernbedienung daneben ist).	Geräte-Adressen zwischen Fernbedienung und Fahrzeug nicht korrekt (gleich) definiert.	Die Codezeilen 27 und 28 in der Fernsteuerung und 23 und 24 im Fahrzeug müssen identisch sein. Siehe Kapitel 9.1.
Die Tonsequenz für die Verbindung mit der Fernbedienung ertönt nicht (das Fahrzeug piept alle 2 s, obwohl eine angeschaltete Fernbedienung daneben ist).	Die Antennen sind nicht ordnungsgemäß in den Anschlüssen der Microcontroller eingesteckt.	Die Antennenanschlüsse auf korrekten Sitz prüfen. Die Funkreichweite ohne die externen Antennen ist extrem gering.
Die Tonsequenz für die Verbindung mit der Fernbedienung ertönt nicht (das Fahrzeug piept alle 2 s, obwohl eine angeschaltete Fernbedienung daneben ist).	Starke Interferenzen durch WLAN.	Funkkanal der EasyRC-Plattform wechseln. Siehe Kapitel 9.2.
Das Fahrzeug fährt, obwohl der Joystick der Fernsteuerung nicht bewegt wird.	Joystick nicht korrekt kalibriert.	Fernsteuerung aus- und wieder anschalten. Dabei den Joystick nicht aus der Ruheposition bewegen.
Das Fahrzeug fährt eine Kurve, obwohl der Joystick nur nach vorn/hinten bewegt wird.	Lenkung nicht korrekt getrimmt.	Anweisungen in Kapitel 7.5 zur Trimmung der Lenkung folgen.
Das Fahrzeug fährt eine Kurve, obwohl der Joystick nur nach vorn/hinten bewegt wird und/oder lenkt nicht bei Joystick-Eingaben.	Servomotor defekt (z.B. durch eine zuvor erfolgte Kollision mit den Vorderrädern).	Servomotor austauschen und die Montage und Trimmung (Kapitel 7.2, 7.5) der Lenkung ausführen.

9. Anhang

9.1 Änderung der Geräte-Adresse der EasyRC-Plattform

Die Geräte-Adresse der EasyRC-Plattform kann geändert werden, um mehrere EasyRC-Systeme gleichzeitig betreiben zu können. Wichtig ist, dass immer ein Paar aus Sender und Empfänger die gleichen Geräte-Adressen hinterlegt hat: Das Fahrzeug hat die Geräte-Adresse der zugehörigen Fernsteuerung hinterlegt und prüft beim Erhalt neuer Datenpakete, ob diese von einer Fernsteuerung mit der korrekten Geräte-Adresse verschickt worden sind. Deshalb müssen die Variablen CarMAC und RemoteMAC sowohl im Code des Microcontrollers der Fernsteuerung als auch des Fahrzeugs geändert werden, wenn ein zweites EasyRC-System eingesetzt werden soll.

Die Geräte-Adressen sind in der Fernsteuerung in den Codezeilen 27 und 28 hinterlegt. Im Fahrzeug sind sie in den Zeilen 23 und 24 zu finden. Die Geräte-Adresse besteht aus sechs Zeichenpaaren, die sich aus Zahlen und Buchstaben zusammensetzen. Die Abbildung zeigt die Geräte-Adressen in der Fernsteuerung. Zur Änderung der Geräte-Adressen reicht es bereits aus, lediglich ein Zeichen eines Zeichenpaars zu ändern.

```
26 //MAC addresses
27 uint8_t CarMAC[] = {0xAA, 0xA0, 0xBE, 0xEF, 0xFE, 0xED}; //we will set this MAC address for the car
28 uint8_t RemoteMAC[] = {0xB0, 0xAD, 0xBE, 0xEF, 0xFE, 0xEF}; //we will set this MAC address for the remote control
29 uint8_t broadMAC[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}; //using this MAC address as a receiver for sending
30 //packages means broadcasting (ignoring if the data has been received)
31
```

Hier ist eine kleine Auswahl an möglichen Geräteadressen, die die Zeilen im Code direkt ersetzen können. Das oberste Paar entspricht den Geräte-Adressen im Auslieferungszustand. Das ausgewählte Paar muss sowohl in der Fernsteuerung als auch im Fahrzeug hinterlegt werden.

uint8_t CarMAC[] = {0xAA, 0xA0, 0xBE, 0xEF, 0xFE, 0xED}; //we will set this MAC address for the car
uint8_t RemoteMAC[] = {0xB0, 0xAD, 0xBE, 0xEF, 0xFE, 0xEF}; //we will set this MAC address for the remote control
uint8_t CarMAC[] = {0xA0, 0xA0, 0xBE, 0xEF, 0xFE, 0xED}; //we will set this MAC address for the car
uint8_t RemoteMAC[] = {0xBA, 0xAD, 0xBE, 0xEF, 0xFE, 0xEF}; //we will set this MAC address for the remote control
uint8_t CarMAC[] = {0xAA, 0xAA, 0xBE, 0xEF, 0xFE, 0xED}; //we will set this MAC address for the car
uint8_t RemoteMAC[] = {0xB0, 0xAA, 0xBE, 0xEF, 0xFE, 0xEF}; //we will set this MAC address for the remote control
uint8_t CarMAC[] = {0xA0, 0xAA, 0xBE, 0xEF, 0xFE, 0xED}; //we will set this MAC address for the car
uint8_t RemoteMAC[] = {0xBA, 0xAA, 0xBE, 0xEF, 0xFE, 0xEF}; //we will set this MAC address for the remote control

9.2 Änderung des Funkkanals der EasyRC-Plattform

Der Funkkanal wird im Code in den Zeilen 90 und 92 (Fernbedienung, in der Abbildung gezeigt) sowie 124 und 126 (Fahrzeug) eingestellt, indem die rot markierte Zahl verändert wird. Die Voreinstellung der Plattform ist Kanal 1. Es kann ein Wert zwischen 1 und 13 ausgewählt werden. Wichtig: Ein verbundenes Paar aus Fahrzeug und Fernsteuerung muss denselben Kanal nutzen. Andernfalls ist keine stabile Verbindung möglich!

```
83 // Activate ESP-NOW
84 WiFi.mode(WIFI_STA); //Set device as a Wi-Fi Station
85 esp_wifi_set_mac(WIFI_IF_STA, RemoteMAC); //Overwrite h
86 esp_wifi_set_protocol(WIFI_IF_STA, WIFI_PROTOCOL_LR); /
87 esp_now_init(); // Initialize ESP-NOW
88
89 esp_wifi_set_promiscuous(true); //set WiFi channel
90 esp_wifi_set_channel(1, WIFI_SECOND_CHAN_NONE);
91 esp_wifi_set_promiscuous(false);
92 peerInfo.channel = 1;
93 peerInfo.encrypt = false;
```

10. Noch umzusetzende Verbesserungen

- Anschluss für einen externen (beliebigen) Motortreiber: Die Fahrzeug-Leiterplatte soll um Ausgang erweitert werden, der GND sowie die zwei Pins zur Steuerung von Motor-Drehrichtung und Geschwindigkeit zur Verfügung stellt. So kann ein externer Motortreiber mit höherer Leistung genutzt werden. Alternativ könnte auch ein Ausgang mit 5 V, einem PWM-Pin sowie GND integriert werden, um beliebige Motortreiber aus dem RC-Bereich nutzen zu können. Hier besteht allerdings die Einschränkung, dass diese über eine eigene Logik verfügen, um Steuerungssignale in Drehzahl und Drehrichtung umzusetzen, die je nach Modell nur eingeschränkt (wenn überhaupt) verändert werden kann.
- Diode statt Jumper zur Verhinderung von Rückstrom aus der USB-Buchse zur 5V-Leitung der Fahrzeug-Leiterplatte. Dadurch soll die Sicherheit und Benutzerfreundlichkeit der Platine erhöht werden.
- Platinen neu und fehlerfrei annotieren.
- Alternative Revisionen von Fahrzeug und Fernsteuerung entwickeln, die größer sind und mehr Leistungsreserven bieten.

11. Impressum

EasyRC ist eine open-source-Plattform, entwickelt von Thomas Böhm. Alle relevanten Informationen sowie Updates zum Projekt sind im zugehörigen GitHub-Repository zu finden: <https://github.com/TRD-B/EasyRC>

Der Autor übernimmt keine Haftung für Schäden oder Verletzungen, die durch die Nutzung der zur Verfügung gestellten Hard- und Software entstehen können!